Linux BSP 42.0 User Manual for S32G2 platforms

30-Sep-2024

Contents

1 About this Document **3**
1.1 References . . . 3
1.2 Terms and abbreviations . . . 3

2 Introduction **4**
2.1 Arm Trusted Firmware (TF-A) . . . 4
2.2 Linux Kernel Image . . . 4
2.3 Root File System . . . 4

3 Build the Linux BSP **5**
3.1 Building the Linux BSP using YOCTO . . . 5
3.1.1 Building default YOCTO . . . 5
3.1.2 BSP Yocto Build . . . 6
3.1.3 BSP Ubuntu Build . . . 7
3.1.4 Building OP-TEE-Enabled Image . . . 8
3.1.5 Building Xen-Hypervisor-Enabled Image . . . 8
3.1.6 Building Images with M7 as Boot Target . . . 8
3.1.7 Building Images with HSE Support . . . 9
3.1.8 Additional instructions for developers . . . 9
3.2 Manually building Linux BSP components . . . 16
3.2.1 Obtaining a GCC toolchain for Arm 64-bit . . . 16
3.2.2 Setting up and building U-Boot bootloader . . . 16
3.2.3 Setting up and building the Linux kernel . . . 18
3.2.4 Building the Arm Trusted Firmware . . . 18
3.2.5 Building the optee_os component of OP-TEE . . . 22

4 How to boot **23**
4.1 Writing images . . . 23
4.1.1 Setup an SD/MMC card . . . 23
4.1.2 Setup QSPI memory . . . 26
4.1.3 Setup eMMC . . . 27
4.2 Board set-up and boot . . . 28
4.2.1 Booting from SD . . . 28
4.2.2 Booting from eMMC . . . 30
4.2.3 Booting from TFTP . . . 30
4.2.4 Booting from NFS (root filesystem) . . . 31
4.2.5 Booting from QSPI . . . 32
4.3 Boards . . . 35
4.3.1 S32G2 EVB . . . 35
4.3.2 S32G274A RDB2 . . . 38

5 SMP Boot **40**
5.1 TF-A SMP Boot . . . 40
5.2 Linux Kernel SMP . . . 40

6 Lockstep Operation **41**
6.1 Enablement flow . . . 41
6.2 Errors handling . . . 42

7 Ethernet Information **43**
7.1 Prerequisites . . . 43
7.2 Ethernet Hardware Support on S32G2 . . . 43
7.2.1 Ethernet support on S32G2 SoC . . . 43
7.2.2 SerDes configuration for SGMII in U-Boot . . . 43
7.2.3 SerDes configuration for SGMII in Linux . . . 45
7.2.4 Ethernet support on the S32G2 EVB board(s) . . . 46
7.2.5 Ethernet support on the S32G274A RDB2 board . . . 47
7.3 Ethernet Software support on S32G2 . . . 47
7.3.1 U-Boot Ethernet support . . . 47
7.3.2 Ethernet fixup procedure . . . 53
7.3.3 Linux Ethernet support . . . 54
7.3.4 Build by Auto Linux BSP . . . 56
7.4 Setting up DHCP client in Linux . . . 56
7.5 SSH support in Linux . . . 56
7.6 Setting up SJA1105 switch . . . 56
7.6.1 SJA1105 Out-of-tree Linux driver . . . 57
7.6.2 SJA1105 Linux DSA driver . . . 57
7.7 Setting up SJA1110 switch . . . 59
7.7.1 Building firmware image for SJA1110 switch . . . 59
7.7.2 Setting up SJA1110 switch over SPI . . . 59
7.7.3 Enabling SJA1110 DSA driver . . . 59

8 PTP Information **60**
8.1 Precision Time Protocol - IEEE 1588 . . . 60
8.2 802.1Qav . . . 61

9 PCIe Support **63**
9.1 PCIe in U-Boot . . . 63
9.1.1 PCIE PHY CRSS, SRNS and SRIS support . . . 65
9.2 PCIe in Linux . . . 65
9.2.1 Enable PCIe manually . . . 65
9.2.2 Rescan PCIe Bus . . . 66
9.2.3 PCIe MSIs . . . 66
9.3 PCIe Clock Configuration . . . 67

10 HSE Security Support **68**
10.1 Prerequisites . . . 68
10.2 Building HSE Features with Yocto . . . 68
10.3 HSE Userspace Driver . . . 70
10.3.1 Prerequisites . . . 70
10.3.2 Configuration . . . 70
10.4 PKCS11 Support . . . 71
10.4.1 Prerequisites . . . 71
10.4.2 Building OpenSSL 3.0.5 for aarch64 . . . 72
10.4.3 Building libp11 0.4.11 for aarch64 . . . 72
10.4.4 Building OpenSC's pkcs11-tool . . . 72
10.4.5 Building the HSE PKCS11 module . . . 73
10.4.6 Building and running the libp11 PKCS11 HSE example . . . 74
10.4.7 Running the pkcs11-tool PKCS11 HSE examples . . . 74
10.4.8 Building the libhse examples . . . 75

10.4.9 Building the HSE Secure Boot demo application . . . 75
10.5 HSE Secure Boot . . . 75
10.6 Manually Building TF-A and Kernel for Authentication . . . 75
10.6.1 Configure TF-A's BL2 & BL3x Authentication . . . 75
10.6.2 Configure Kernel Authentication . . . 80
10.7 HSE Linux Driver . . . 84
10.7.1 Prerequisites . . . 84
10.7.2 Supported Algorithms . . . 85
10.7.3 Configuration . . . 85
10.8 HSE Crypto Driver in OP-TEE . . . 86
10.8.1 Build Instructions . . . 86
10.8.2 Configure HSE OP-TEE Crypto Driver . . . 86
10.8.3 Enable the OP-TEE Crypto Driver . . . 88
10.8.4 Features . . . 88

11 DDR **90**
11.1 Integration of DDR Tool generated code into TF-A . . . 90
11.2 Checking if Inline ECC Feature is Enabled . . . 90

12 LLCE Support **92**
12.1 Yocto Integration . . . 92
12.2 Manual compilation . . . 93
12.3 CAN . . . 93
12.3.1 Usage example . . . 93
12.3.2 Timestamping . . . 94
12.3.3 Multi-Host . . . 94
12.3.4 Statistics . . . 97
12.3.5 Filters and Routing . . . 98
12.3.6 Advanced parameters . . . 101
12.4 CAN Logger . . . 103
12.4.1 Using the Driver . . . 103
12.5 LINFlexD . . . 104
12.6 LPSPI . . . 104

13 CAN Information **106**
13.0.1 Example of sending a CAN message . . . 106

14 SD/eMMC high speed modes support **107**
14.1 SD/eMMC high speed modes support . . . 107
14.1.1 Enabling high speed modes in U-Boot . . . 107
14.1.2 Enabling high speed modes in Linux . . . 107
14.2 eMMC HS400 Enhanced Strobe support . . . 108
14.3 eMMC Performance enhancements . . . 108
14.4 'mmc_test' Kernel Driver Usage and Enablement . . . 108

15 Temperature Monitoring Unit **110**
15.1 Reading Temperatures from the Thermal Monitoring Unit . . . 110

16 SoC Level Time Source **111**
16.1 General Information . . . 111
16.2 Kernel Configuration . . . 111
16.3 Userspace Interface . . . 111
16.4 Supported commands . . . 111
16.5 Accessing Time Source and Control from Outside Linux . . . 111

17 Switch the Linux System Timer from Generic Timer to PIT timer **112**

18 SPI Target Support **113**
18.1 SPI Target Support in Linux . . . 113

19 SAR-ADC Driver **114**
19.1 Using the SAR-ADC in U-Boot . . . 114
19.2 Using the SAR-ADC in Linux . . . 114

20 Controlling GPIOs in Linux **118**
20.1 About libgpiod . . . 118
20.2 Using libgpiod-tools . . . 118
20.2.1 gpiodetect . . . 118
20.2.2 gpioinfo . . . 118
20.2.3 gpioget . . . 119
20.2.4 gpioset . . . 119
20.2.5 gpiofind . . . 119
20.2.6 gpiomon . . . 119
20.3 Devicetree Example of using a GPIO external interrupt (EIRQ) . . . 120

21 Clocking **120**
21.1 Read clock frequency at runtime . . . 120

22 Setting up SIUL2 pads **121**

23 Power Management **122**
23.1 Platform Reset . . . 122
23.2 CPU Hotplug . . . 122
23.3 Suspend to RAM . . . 122
23.3.1 Wakeup Sources . . . 123
23.3.2 Suspend to RAM with RTC as Wakeup Source . . . 123
23.3.3 Suspend to RAM with GPIO as Wakeup Source . . . 124
23.4 Dynamic Frequency Scaling . . . 125
23.4.1 Via sysfs filesystem . . . 126
23.4.2 Via cpufrequtils package . . . 126

24 Virtualization **128**
24.1 Linux Containers User Guide . . . 128
24.1.1 Build, Installation and Configuration . . . 128
24.1.2 LXC How To's (Getting Started with a BusyBox System Container) . . . 129
24.2 Docker Containers . . . 132
24.2.1 What is Docker? . . . 132
24.2.2 Enabling Docker in Auto Linux BSP . . . 132
24.2.3 Docker Compose . . . 133
24.3 Xen Hypervisor . . . 133

24.3.1 What is Xen? . . . 133
24.3.2 Dom0less Virtual Machines . . . 134
24.3.3 How to enable Xen in S32 Linux BSP build . . . 134
24.3.4 Building Custom Xen Setups . . . 135
24.3.5 Booting Xen, Dom0 and DomU . . . 136
24.3.6 Networking in DomUs . . . 137
24.3.7 VirtIO PV Drivers . . . 138

25 Arm Trusted Firmware **140**

26 OP-TEE **141**
26.1 RPMB Secure Storage . . . 141
26.2 TrustZone Key Provisioning . . . 143
26.2.1 Terminology . . . 143
26.2.2 Timeline of Actions . . . 143
26.2.3 Build and Run Instructions . . . 144
26.3 Opaque Keys . . . 145
26.3.1 Plain Keys vs Opaque Keys . . . 145
26.3.2 Opaque Key Format . . . 147
26.3.3 Configuration and Examples . . . 147
26.3.4 PBKDF2 Support for Opaque Keys . . . 148

27 Optional Features **150**
27.1 Unrestricted userspace access to hardware registers . . . 150
27.2 SysVInit setup . . . 150
27.3 DM-Verity . . . 150
27.4 Root Hash Signing . . . 151
27.5 Changing UART baud rate . . . 151
27.5.1 Changing UART baud rate from Yocto . . . 151
27.5.2 Changing UART baud rate per component . . . 152

28 Demo Applications **154**
28.1 Multicore Sample Application . . . 154
28.2 Networking Sample Applications . . . 155
28.2.1 One to One Chat Application . . . 155
28.2.2 One to Many Echo Application . . . 157
28.3 GPIO Libgpiod Application . . . 158
28.4 PCIe Sample Applications . . . 158
28.4.1 PCIe Shared Memory Sample Application . . . 159
28.4.2 *pcitest* Application . . . 161

29 SoC Errata **164**
29.1 Thermal Monitoring Unit (TMU) . . . 164
29.1.1 ERR052243 . . . 164
29.2 Ultra Secured Digital Host Controller (uSDHC) . . . 164
29.2.1 ERR052198 . . . 164

1 About this Document

1.1 References

Reference	File Name	Version
S32G2 Reference Manual	S32G2RM.pdf	Rev. 8
GMAC Subsystem Reference Manual	GMACSUBSYSRM.pdf	Rev. 2
S32G2 and S32R SerDes Subsystem Reference Manual	S32G2RSERDESRM.pdf	Rev. 6
PFE Firmware User Manual	PFE_Firmware_S32G_UserManual.pdf	1.8.0
PFE Driver User Manual	PFE_S32G_A53_LNX_UserManual.pdf	1.7.0
PFE Health Monitoring User Manual	PFE_SW_HM_S32G_UserManual.pdf	1.4.1
PFE MCAL 4.4 Driver	AUTOSAR_MCAL_ETH_43_PFE_UM.pdf	1.3.0
LLCE Firmware User Guide	LLCE_firmware_user_guide.pdf	1.0.9
LLCE Firmware Getting Started Guide	LLCE_getting_started_guide.pdf	1.0.9
HSE Service API Reference Manual	S32G2XX_HSE_Service_API_Reference_Manual.pdf	0.2.51.0
S32G-VNP-EVB User Guide	S32G-VNP-EVB-UG.pdf	Rev. 1
S32G-VNP-EVB Quick Start Guide	S32G-VNP-EVB-QSG.pdf	N/A
S32G-VNP-RDB2 User Guide	S32G-VNP-RDB2UG.pdf	N/A
S32G-VNP-RDB2 Reference Manual	S32G-VNP-RDB2-RM.pdf	Rev. 0.4
S32G-VNP-RDB2 Ethernet Enablement Guide	VNPRDBEEUG.pdf	Rev. 1

1.2 Terms and abbreviations

Acronym	Definition
BSP	Board Support Package
CoT	Chain of Trust
CQE	Command Queuing Engine
EL0,1,2,3	Exception Level 0,1,2,3 for A53 core
GCC	GNU C Compiler Suite
MTD	Memory Technology Device
NFS	Network File System
SCP	System Control Processor
SD	Secure Digital
S32CC / S32GEN1	Sub-group of S32 processor family containing S32G and S32R processors
TBBR	Trusted Board Boot Requirements
TF-A	Arm Trusted Firmware for Cortex-A cores
TZ	Arm TrustZone

2 Introduction

The Linux BSP is a collection of source code that can be used to create U-Boot boot loader, Linux kernel image, a root filesystem and an Arm Trusted Firmware (TF-A) images for the supported boards.

2.1 Arm Trusted Firmware (TF-A)

The Linux BSP delivery package contains the following Arm Trusted Firmware binary: *auto_linux_bsp42.0/<board>-/fip.s32-<sdcard/qspi>*.

This TF-A firmware supports SD/MMC/eMMC (*fip.s32-sdcard*) and QSPI (*fip.s32-qspi*).

2.2 Linux Kernel Image

This Linux BSP contains a pre-built kernel image based on the v6.6.32 of the Linux kernel. The kernel image and dtb file is located at the following path: auto_linux_bsp42.0_<soc>/<board>

2.3 Root File System

The package contains the following rootfs file system: *binaries_auto_linux_42.0_<soc>/<board>/fsl-image-[auto/base/flash]-<board>.tar.gz*. The *rootfs.tar.gz* file system includes NXP specific libraries. Its contents can be mounted as a NFS share or can be stored on a boot media such as Secure Digital (SD) card.

3 Build the Linux BSP

This Linux BSP has been built and tested using the GCC 11.4.0 for ARM64 cross-compiler included in the Yocto 4.0.18 "Kirkstone" distribution.

All the steps described below have been run and validated on Ubuntu-20.04 LTS (native or through a virtual machine). It is then recommended to install Ubuntu-20.04 LTS before going through the following sections.

3.1 Building the Linux BSP using YOCTO

3.1.1 Building default YOCTO

Update the package manager:

```
sudo <pkg-mgr> update
```

Install dependencies:

- python 2.x - 2.6 or newer:

  ```
  sudo <pkg-mgr> install python
  ```

- git 1.8.3 or newer:

  ```
  sudo <pkg-mgr> install git
  ```

- curl:

  ```
  sudo <pkg-mgr> install curl
  ```

  where <pkg-mgr> is the package manager for your distribution (apt-get or apt for Debian/Ubuntu, yum or dnf for CentOS/Fedora, zypper for SUSE). On more recent Linux distributions, the python package has been replaced by python2 or python3.

To get the BSP you need to have **repo** installed. Please install it using the following commands (this only needs to be done once):

```
mkdir ~/bin
curl https://storage.googleapis.com/git-repo-downloads/repo-2.32 > ~/bin/repo
chmod a+x ~/bin/repo
PATH=${PATH}:~/bin
```

Configure your git environment (you may skip this option if you have git already configured):

```
git config --global user.email "you@example.com"
git config --global user.name "Your Name"
```

Next, download the Yocto Project Environment into your directory:

```
mkdir fsl-auto-yocto-bsp
cd fsl-auto-yocto-bsp
repo init -u https://github.com/nxp-auto-linux/auto_yocto_bsp -b release/bsp42.0
repo sync
```

This will download the sources for the latest NXP Auto Linux BSP (from the branch release/bsp42.0), structured on top of the Yocto Kirkstone release and upstream NXP QorIQ SDK.

Optional manifests options:
The repository provides more manifest files, particularized for different use cases. The desired manifest file is selected by specifying it in the repo init command, using parameter `-m <manifest file>`, e.g:

```
repo init -u https://github.com/nxp-auto-linux/auto_yocto_bsp \
    -b release/bsp42.0 \
    -m <manifest file>.xml
```

The manifests are delivered in two formats: *<manifest>.xml* and *<manifest>-bitbucket.xml*. The latter would allow fetching internal bitbucket development repositories.

Enter the directory fsl-auto-yocto-bsp and follow below instructions (available in the *README* file).



A Yocto build needs at least 50GB of free space and takes a lot of time (2 to 10 hours, depending on the system configuration). It is recommended to use a powerful system with many cores and a fast storage media (SSD is recommended). The recommended RAM size is 8 GB.

3.1.2 BSP Yocto Build

To build the Linux BSP, please follow the steps:

1. First time setup (you need to have rights to execute *sudo apt-get <cmd>* without password).

```
sudo apt update
./sources/meta-alb/scripts/host-prepare.sh
```

2. Creating Build Directories and Testing the Installation. Now you can create a build directory in the SDK root with:

```
source nxp-setup-alb.sh -m <machine>
```

For example, if targeting S32G2 EVB, machine is `s32g274aevb`:

```
source nxp-setup-alb.sh -m s32g274aevb
```

3. When this is done, a `bitbake <imagename>`, e.g.

```
bitbake fsl-image-base
```

would be enough to completely build U-Boot, kernel, modules, the TF-A and a rootfs ready to be deployed. Look for a build result in *<builddirectory>/tmp/deploy/images/*.

In rare cases, when building using python 3.X on the host OS, the following error: "OSError: Cannot initialize new instance of inotify, Errno=Too many open files" may occur. In this case, the script below shows which processes consume inotify watches. The processes that got stuck need to be manually terminated:

```
find /proc/*/fd -lname anon_inode:inotify 2> /dev/null |
sed 's/fd\//fdinfo\//' |
xargs -I '{}' -- grep -H -c 'inotify.*ino' '{}' |
awk -F'[/:]' '{map[$3] += $6} END \
{ for (key in map) print key"\t"map[key]"\t"key }' |
awk '$2 > 3' |
sort -rn -k2 |
awk '{ cmd="cat /proc/"$3"/comm"; \
cmd | getline $3; print "PID="$1"\t NR="$2"\tName="$3}'
```

If the machine chosen was s32g274aevb, after the successful finalization of the build, the results will be placed in the *build_s32g274aevb/tmp/deploy/images/s32g274aevb* directory. Results for other machines will be placed in similar directories, but named according to the machine name. This release includes support for:

- Machines: s32g274aevb, s32g254aevb, s32g233aevb, s32g274ardb2, s32g274aevb3, s32g254aevb3, s32g233aevb3
- Images: fsl-image-base, fsl-image-auto

After deploying the Yocto image and booting the platform, please use the following credentials to log in:

- The user root with no password

3.1.3 BSP Ubuntu Build

In case of building Ubuntu target images, this release includes support for versions 22.04.1 LTS and 20.04.1 LTS. To build the Linux BSP, please follow the steps:

1. First time setup (you need to have rights to execute *sudo apt-get <cmd>* without password).

```
sudo apt update
./sources/meta-alb/scripts/host-prepare.sh
```

2. Creating Build Directories and Testing the Installation. Now you can create a build directory in the SDK root with:

```
source nxp-setup-alb.sh -m <machine>ubuntu
```

For example, if targeting S32G2 EVB, machine is s32g274aevbubuntu:

```
source nxp-setup-alb.sh -m s32g274aevbubuntu
```

3. When this is done, a bitbake <imagename>, e.g.

```
bitbake fsl-image-ubuntu-base
```

would be enough to completely build U-Boot, kernel, modules, the TF-A and a rootfs ready to be deployed. Look for a build result in *<builddirectory>/tmp/deploy/images/*.

This release includes support for:

- Machines: `s32g274aevbubuntu, s32g274ardb2ubuntu, s32g254aevbubuntu, s32g233aevbubuntu, s32g274aevb3ubuntu, s32g254aevb3ubuntu, s32g233aevb3ubuntu`;
- Images: fsl-image-ubuntu-base, fsl-image-ubuntu, fsl-image-ubuntu-ros.

In case of Ubuntu images, by default Ubuntu target version is 22.04.1.
If targeting Ubuntu-20.04 images, add the following line in *<builddirectory>/conf/local.conf*.

```
UBUNTU_TARGET_VERSION = "20.04.1"
```

Then run `bitbake <imagename>`, where <imagename> is any of the supported Ubuntu images specified above. After deploying a Ubuntu image and booting the platform, please use the following credentials to log in:

- user: *bluebox*
- password: *bluebox*



Ubuntu root file system content can be customized (adding or removing apt-get Ubuntu packages) by updating APGET_EXTRA_PACKAGES variable, instead of IMAGE_INSTALL mechanism, as follows:

```
APTGET_EXTRA_PACKAGES += "<package_name>"
APTGET_EXTRA_PACKAGES:remove += "<package_name>"
```

3.1.4 Building OP-TEE-Enabled Image

OP-TEE can be built and deployed by editing *<builddirectory>/conf/local.conf* and appending the following line:

```
DISTRO_FEATURES:append = " optee"
```

3.1.5 Building Xen-Hypervisor-Enabled Image

Xen Hypervisor is an Open-Source Type-1 (Bare-metal) hypervisor. The support for Xen is an optional feature that can be enabled by appending the following line to *<builddirectory>/conf/local.conf*:

```
DISTRO_FEATURES:append = " xen"
```

When Xen support is enabled, the BSP components are built with Xen support (where needed): U-Boot loads Xen at EL2 with Xen-specific booting commands, Linux Kernel image is built with support for paravirtualization, and Xen tools and libraries are installed into the root filesystem.

Please refer to Chapter "Building Custom Xen Setups" for more information about how to build and use different Xen configurations.

3.1.6 Building Images with M7 as Boot Target

Cortex-M7 booting flow is provided as an example for enabling lockstep operation mode. It is enabled by editing *<builddirectory>/conf/local.conf* and appending the following line:

```
DISTRO_FEATURES:append = " m7boot"
```

When Cortex-M7 booting flow is enabled, the generated SD card or flash images can be used in the same way as for default Cortex-A53 booting flow. These images include all the needed changes: Image Vector Table (IVT) updates and addition of Cortex-M7 software. In case IVT files needs to be deployed manually, when Cortex-M7 booting flow is enabled the files with **.m7* (e.g. *fip*.m7*) extensions should be used.

3.1.7 Building Images with HSE Support

To enable PKCS11 Support & the HSE Linux Driver, update the *conf/local.conf* file in the Yocto build directory with these lines:

```
DISTRO_FEATURES:append = " hse"
NXP_FIRMWARE_LOCAL_DIR = "/path/to/firmware/folder"
```

To enable SECBoot Support, update the *conf/local.conf* file in the Yocto build directory with these lines:

```
DISTRO_FEATURES:append = " secboot"
NXP_FIRMWARE_LOCAL_DIR = "/path/to/firmware/folder"
```

`NXP_FIRMWARE_LOCAL_DIR` must point to the the location of the folder containing the HSE Firmware deliverables.

To enable Verified Boot Support, update the *conf/local.conf* file in the Yocto build directory with this line:

```
DISTRO_FEATURES:append = " verifiedboot"
```

Verified boot can be enabled for both SD card and QSPI images.

For more information on the build configuration of HSE features in Yocto, please consult the Building HSE Features with Yocto section.

3.1.8 Additional instructions for developers

This chapter contains instructions for developers and assumes that the reader is familiar with the Yocto build process as well as the Kbuild framework used by U-Boot and the Linux kernel.

3.1.8.1 How to locate U-Boot, Linux and other sources in the Yocto tree

The following paragraphs assume you are in *<Yocto folder>/build_<machine>* build directory after you have sourced `nxp-setup-alb.sh` (so that `bitbake` is found in $PATH).

Locate the source for any packages, use the command below run from the build directory:

```
bitbake -e <package> | grep "^S="
```

This command will go through the package environment and find out information about the location of the source. Example:
U-Boot source:

```
bitbake -e u-boot | grep "^S="
S="<yocto root>/build_<machine>/tmp/work/<machine>-fsl-linux/u-boot-s32/2022.04-r0/git"
```

Linux source:

```
bitbake -e virtual/kernel | grep  "^S="
S="<yocto root>/build_<machine>/tmp/work/<machine>-fsl-linux/linux-s32/<kernel_version>-r0/git"
```

Where *<machine>* is the name used as Yocto machine, e.g. *s32g274aevb*, and *<kernel_version>* is the version of your built kernel, e.g. *6.6.32*.

3.1.8.2 Customizing, building and deploying kernel using YOCTO

3.1.8.2.1 Patch the kernel

In this example, we create a patch for Linux SAR-ADC driver. We should do the steps as below:

1. Go to the SAR-ADC driver folder:

   ```
   cd tmp/work/<machine>-fsl-linux/linux-s32/<kernel_version>-r0/git
   ```

   and edit the source file drivers/iio/adc/s32cc_adc.c

2. Add source that you changed before to git

   ```
   git add drivers/iio/adc/s32cc_adc.c
   git commit -s -m <title of your commit>
   ```

3. Create the patch

   ```
   git format-patch -1
   ```

   Check in *<builddirectory>/tmp/work/<machine>-fsl-linux/linux-s32/<kernel_version>-r0/git* folder, there is a patch file created, e.g. *0001-test-sar-adc-repackage.patch*

4. Copy the patch file to meta-alb-bsp/recipes-kernel/linux/linux-s32 folder

5. Edit *linux-s32_<kernel_major>.bb* file in meta-alb-bsp/recipes-kernel/linux to include the patch file, e.g.

   ```
   SRC_URI += "file://0001-test-sar-adc-repackage.patch"
   ```

6. Return to the build directory and run command as below to clean the previous sstate:

```
source ./SOURCE_THIS
bitbake virtual/kernel -c cleansstate
```

7. Re-build and deploy linux

```
bitbake virtual/kernel
```

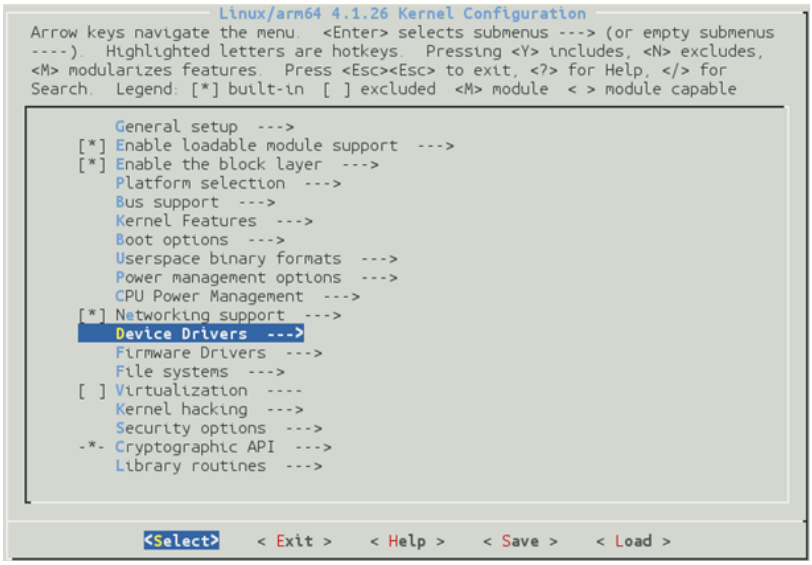
Where <*kernel_version*> is the full version of your built kernel, e.g. *6.6.32*, and <*kernel_major*> is the major version of your built kernel, e.g. *6.6*.

3.1.8.2.2 Invoke menuconfig to set up a specific configuration for Linux

This chapter contains instructions for modifying a particular kernel compile-time option, using the Yocto build framework.

1. From the build folder, run the following command:

```
bitbake virtual/kernel -c menuconfig
```



The user can then navigate the configuration options exposed in the `menuconfig` interface by using the arrow keys, and select/unselect individual configs using the 'y' or 'n' keys, or Space to toggle selection. The user can also search for a specific config option using the slash '/' key from anywhere in `menuconfig`. This opens a search dialog and the user can enter a (sub)string or regexp and jump to the listed findings by pressing the associated numbers: (1), (2),... The user has the option to save or discard the changes before exiting the `menuconfig` interface.

2. Rebuild kernel

```
bitbake -c compile -f virtual/kernel
```

3. Deploy the images

```
bitbake virtual/kernel
```

After the deploy process is finished, check in the folder, the latest Linux Image is deployed.

3.1.8.2.3 Build standalone kernel modules

To build a standalone kernel module, the following environment variables should be set in order to use the toolchain and the kernel build folder:

```
export ARCH=arm64
export CROSS_COMPILE=<builddir>/tmp/work/<machine>-fsl-linux/linux-s32/<kernel_version>-r0/\
recipe-sysroot-native/usr/bin/aarch64-fsl-linux/aarch64-fsl-linux-
export <Your_kernel_build_dir>=<builddir>/tmp/work/<machine>-fsl-linux/linux-s32/<kernel_version>-r0/build
```



- `<Your_kernel_build_dir>` should be the variable used in the module's Makefile, passed as argument to the make invocation via the `-C` parameter. For example, if the module's Makefile invokes the kernel's top Makefile using a similar command:

  ```
  make -C ${KDIR} M=${PWD} modules
  ```

  then, `<Your_kernel_build_dir>` should be `KDIR`, and should contain the path to the kernel build folder, as mentioned above.
- `<machine>` is one of the supported machines listed above.
- `<kernel_version>` is the full version of your built kernel, e.g. `6.6.32`.

Then build the kernel module invoking `make`.

To simplify the procedure to export the toolchain and kernel build folders, the following script can be used to compute these variables automatically:

```
#!/bin/sh

if test $# -lt 1 ;then
  YOCTO_BUILD=`pwd`
else
  YOCTO_BUILD=$1
fi

BB_SOURCE="$YOCTO_BUILD/SOURCE_THIS"
```

```
if test -f $BB_SOURCE
then
  . ${BB_SOURCE}
  KERNEL_SETTINGS=$(bitbake -e virtual/kernel | \
  grep "^STAGING_BINDIR_TOOLCHAIN\|^TARGET_PREFIX=\|^KBUILD_OUTPUT=\|export ARCH=")
else
  echo "Usage : $0 <yocto_build_dir>"
  echo "\n\tyocto_build_dir:  The path of Yocto build"
  echo "\tExample:/build_<machine>"
  echo "\nERROR:$YOCTO_BUILD isn't an yocto build folder !"
  exit 1
fi

get_env_var()
{
  echo $KERNEL_SETTINGS | sed -E "s/.*$1=\"([^\"\]+)\".*/\1/"
}

STAGING_BINDIR_TOOLCHAIN=$(get_env_var "STAGING_BINDIR_TOOLCHAIN")
TARGET_PREFIX=$(get_env_var "TARGET_PREFIX")
KDIR=$(get_env_var "KBUILD_OUTPUT")
ARCH=$(get_env_var "ARCH")

echo KDIR=$KDIR
echo ARCH=$ARCH
echo CROSS_COMPILE=$STAGING_BINDIR_TOOLCHAIN/$TARGET_PREFIX
```

3.1.8.2.4 Building with a 5.15.158 kernel

From BSP 42.0, the default version of kernel is v6.6.32. In case of need, the kernel can be changed to 5.15.158 by adding the following line into the *conf/local.conf* file of your 'build_<machine>' directory:

```
PREFERRED_VERSION_linux-s32 = "5.15.158"
```

Afterwards, any build that uses linux-s32 will use the 5.15.158 one.

3.1.8.3 Customizing, building and deploying U-Boot using YOCTO

3.1.8.3.1 Invoke menuconfig to set up a specific configuration for U-Boot

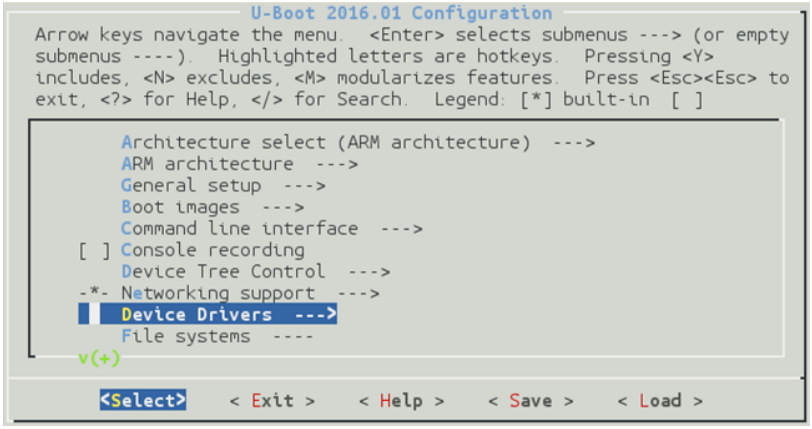
1. From the build folder, run the following command:

```
bitbake u-boot -c configure
bitbake u-boot -c devshell
```

The following commands must be executed in the newly opened window:

```
cd ../build/<defconfig>
make menuconfig
exit
```

where <*defconfig*> is the selected configuration, e.g. *s32g2xxaevb_defconfig*. For the complete list of configurations for each board refer to section"Building the U-Boot bootloader"



The `menuconfig` interface for U-Boot is the same as the one for the Linux kernel (see Invoke menuconfig to set up a specific configuration for Linux).

2. Rebuild U-Boot

```
bitbake -c compile -f u-boot
```

3. Deploy U-Boot

```
bitbake u-boot
```

After the deploy process is finished, check the deploy folder for the U-Boot binaries, which should include: `u-boot-nodtb.bin-sdcard` `u-boot-nodtb.bin-qspi`

3.1.8.4 Applying custom patches in Yocto

There are certain patches not included by default for various reasons, but which may be useful in certain use-cases. Here are the steps required for including such a patch into a Yocto build.

1. Create a .bbappend file for the appropriate recipe:

```
mkdir -p sources/meta-alb/recipes-patched/<package>
touch sources/meta-alb/recipes-patched/<package>/<package_recipe_name>.bbappend
```

More information about .bbappend files can be found at: https://docs.yoctoproject.org/4.0.18/ref-manual/terms.html#term-Append-Files

2. Identify the required .patch file, and add a reference to it in the .bbappend:

```
SRC_URI += "<remote_package_location>/<desired-patch-file>.patch"
```

More information about patching recipes can be found at: https://docs.yoctoproject.org/4.0.18/dev-manual/new-recipe.html

3. Rebuild and redeploy

Here is an example:

```
mkdir -p sources/meta-alb/recipes-patched/<package>
touch sources/meta-alb/recipes-patched/<package>/<package>_git.bbappend
echo "SRC_URI += \" <remote_package_location>/<package_patch>.patch\"" >> sources/meta-alb/\
recipes-patched/<package>/<package>_git.bbappend
```

3.1.8.5 Configuration files, logs and other useful items from the Yocto file structure

S32 configuration files are stored in the meta-alb-bsp/conf/machine folder, you can find details here.

Yocto saves the output log from bitbake in *<builddirectory>/tmp/log/cooker*, kept under *<builddirectory>/tmp/*. When an error occurs, the exact information listed in the log file is also printed on the console output.

At the following address you can find some useful commands when working with Yocto: https://community.nxp.com/docs/DOC-94953

3.1.8.6 Customize Image Size of Yocto Build

This chapter contains information related to customizing the image size of a Yocto build.

1. How to change the size of boot partition (FAT)

   In `conf/local.conf` set the `BOOT_SPACE` to desired value.

   For example: Set boot partition size to 64MB

   ```
   BOOT_SPACE = "65536"
   ```

2. How to change the size of the extra free space in the root file system partition (EXT4)

   The `IMAGE_ROOTFS_EXTRA_SPACE` variable should be set to the desired amount of free space in Kbytes in the root file system partition.

   Example: Allow 1GB of free space.

   ```
   IMAGE_ROOTFS_EXTRA_SPACE = "1048576"
   ```

   The variable can be set in `conf/local.conf` or directly in the recipe of the image.

3.1.8.7 Building Linux BSP Yocto images offline

This chapter contains information relating to building Linux BSP images offline, following the steps below:

1. Fetch all necessary packages for the image target e.g: `fsl-image-auto`. The following operation should be executed with network access.

```
bitbake fsl-image-auto --runall=do_fetch
```

2. Add the following lines in `conf/local.conf` file from the build directory.

```
BB_NO_NETWORK = "1"
BB_SRCREV_POLICY = "cache"
```

`BB_SRCREV_POLICY` variable is used to instruct the fetcher to retain the value the system previously obtained rather than querying the source control system each time.

3. The `fsl-image-auto` Yocto image can be built without network access.

```
bitbake fsl-image-auto -c clean && bitbake fsl-image-auto
```

3.2 Manually building Linux BSP components

3.2.1 Obtaining a GCC toolchain for Arm 64-bit

This Linux BSP has been built and tested using the GCC 11.4.0 for ARM64 toolchain.
The link for a compatible pre-built GCC 11.3.0 for ARM64 toolchain, as delivered by Arm at:

https://developer.arm.com/-/media/Files/downloads/gnu/11.3.rel1/binrel/arm-gnu-toolchain-11.3.rel1-x86_64-aarch64-none-linux-gnu.tar.xz?rev=8d05006a68d24d929d602804ec9abfb4&hash=E8A66D3B9FF0EFC60A5C041AC3C5EE476349817B

Once you have downloaded the toolchain package, in order to install it, you just need to untar it in a directory of your choice.

3.2.2 Setting up and building U-Boot bootloader

Prerequisites:

- Install bison and flex, which are required in the build step. This is a one-time operation. On Ubuntu-based hosts, you should:

```
sudo apt-get install bison flex
```

(note that you might need to *sudo apt-get update* first, in order to update the host's package list).

3.2.2.1 Downloading the U-Boot bootloader source code

There are two ways of obtaining the source for this component, each described below. Choose the one which is appropriate for your situation.

1. Cloning the GIT repository:
   In a Linux terminal window, type in the following commands

```
git clone https://github.com/nxp-auto-linux/u-boot
cd u-boot
git checkout bsp42.0-2022.04
ls
```

   The contents of the U-Boot source code should appear here.

2. If you already have a clone of the repository, you can run the following commands in the root directory of the existing repository:

```
git fetch origin bsp42.0-2022.04
git checkout bsp42.0-2022.04
ls
```

   The contents of the U-Boot source code should appear here.

3.2.2.2 Building the U-Boot bootloader

In the same Linux terminal window as above, type in the following commands:

```
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- <board>_defconfig
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu-
```

These commands should generate the U-Boot image with Program data (*u-boot-nodtb.bin*).



The values of <board> are given as following:

- For `s32g274aevb` using Memory Card, board is `s32g2xxaevb`
- For `s32g274aevb` using QSPI, board is `s32g2xxaevb_qspi`
- For `s32g254aevb` using Memory Card, board is `s32g2xxaevb`
- For `s32g254aevb` using QSPI, board is `s32g2xxaevb_qspi`
- For `s32g233aevb` using Memory Card, board is `s32g2xxaevb`
- For `s32g233aevb` using QSPI, board is `s32g2xxaevb_qspi`
- For `s32g274ardb2` using Memory Card, board is `s32g274ardb2`
- For `s32g274ardb2` using QSPI, board is `s32g274ardb2_qspi`
- For `s32g274aevb3` using Memory Card, board is `s32g2xxaevb3`
- For `s32g274aevb3` using QSPI, board is `s32g2xxaevb3_qspi`
- For `s32g254aevb3` using Memory Card, board is `s32g2xxaevb3`
- For `s32g254aevb3` using QSPI, board is `s32g2xxaevb3_qspi`
- For `s32g233aevb3` using Memory Card, board is `s32g2xxaevb3`
- For `s32g233aevb3` using QSPI, board is `s32g2xxaevb3_qspi`

3.2.3 Setting up and building the Linux kernel

3.2.3.1 Downloading the Linux kernel source code

There are two ways of obtaining the source for this component, each described below. Choose the one which is appropriate for your situation.

1. Cloning the GIT repository:
   In a Linux terminal window, type in the following commands

   ```
   git clone https://github.com/nxp-auto-linux/linux
   cd linux
   git checkout bsp42.0-6.6.32-rt
   ls
   ```

   The contents of the linux kernel source code should appear here.

2. If you already have a clone of the repository, you can run the following commands in the root directory of the existing repository:

   ```
   git fetch origin bsp42.0-6.6.32-rt
   git checkout bsp42.0-6.6.32-rt
   ls
   ```

   The contents of the Linux kernel source code should appear here.

3.2.3.2 Building the Linux Kernel

In the same Linux terminal window as above, type in the following commands:

```
make ARCH=arm64 CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- \
    <soc_name>_defconfig
make ARCH=arm64 CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu-
```

For `s32g274aevb`, `s32g254aevb`, `s32g233aevb`, `s32g274ardb2`, `s32g274aevb3`, `s32g254aevb3`, `s32g233aevb3`: the defconfig is `s32cc_defconfig`.

This command should generate the kernel binary (*Image*) in arch/arm64/boot and the board device tree blobs (e.g. *s32g2xx-evb.dtb*) in arch/arm64/boot/dts/freescale/.

3.2.4 Building the Arm Trusted Firmware

Prerequisites:

- (Optional) If choosing to include OP-TEE in the TF-A boot flow, one must also build optee_os according to Building the optee_os component of OP-TEE. The optee_os build will be excuted as part of a boot stage in TF-A.
- Install libssl-dev and openssl for headers required by *fiptool*. This is a one-time operation. On Ubuntu-based hosts, you should:

  ```
  sudo apt-get install libssl-dev openssl
  ```



You might need to run *sudo apt-get update* first, in order to update the host's package list.

- Check your host's dtc version. If you have a dtc version at least 1.4.6, you can skip the apt-get install step following this one:

```
dtc --version
```

- If your dtc is older or you do not have it installed, install/upgrade to dtc version 1.4.6 or above - this is a one-time operation:

```
sudo apt-get install device-tree-compiler
```



The TF-A blob being built is a FIP image that contains both the TF-A boot stages (**BL2**, **BL31**) and the U-Boot binary (**BL33**). Because of that, the TF-A build process depends on U-Boot being available. To that end, the **BL33** parameter to the make command-line must be the path to the *u-boot-nodtb.bin* located in the directory where U-Boot has been built as part of the prerequisites. Optionally, TF-A also depends on OP-TEE if enabled: **BL32** must be the path to the *tee-header_v2.bin* and **BL32_EXTRA1** must be the path to the *tee-pager_v2.bin*, both located in the directory where optee_os has been built as part of the prerequisites.

Follow these steps to build the TF-A:

1. Obtain the arm-trusted-firmware:

```
git clone https://github.com/nxp-auto-linux/arm-trusted-firmware
cd arm-trusted-firmware
git checkout bsp42.0-2.10
```

   If you already have a clone of the repository, you can run the following commands in the root directory of the existing repository:

```
git fetch origin bsp42.0-2.10
git checkout bsp42.0-2.10
```

2. Build the arm-trusted-firmware FIP image. Choose one of the following available options:

   (a) Build the default TF-A FIP image:

```
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- \
    ARCH=aarch64 PLAT=<plat> BL33=<path-to-u-boot-nodtb.bin>
```

   (b) Build the OP-TEE-enabled TF-A FIP image:

```
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- \
    ARCH=aarch64 PLAT=<plat> BL33=<path-to-u-boot-nodtb.bin> \
    BL32=<path-to-tee-header_v2.bin>  \
    BL32_EXTRA1=<path-to-tee-pager_v2.bin> SPD=opteed
```

3. Deploy *build/<plat>/release/fip.s32* to the sdcard. See also this section for indications on partitioning the SD card.

```
# Skip the partition table from the SD card and copy the rest of the fip image
# (TF-A and U-Boot).
# Presumably, if you are doing these steps, you are using a pre-partitioned SD card
# on which you are only replacing the TF-A and/or U-Boot image which you have manually built.
sudo dd if=<path/to/fip.s32> of=/dev/<sdcard_dev> skip=512 seek=512 \
    iflag=skip_bytes oflag=seek_bytes conv=fsync,notrunc
```

Depending on the target board, the value of `<plat>` should be as following:

- `s32g274aevb`: `<plat>` is `s32g2xxaevb`
- `s32g254aevb`: `<plat>` is `s32g2xxaevb`
- `s32g233aevb`: `<plat>` is `s32g2xxaevb`
- `s32g274ardb2`: `<plat>` is `s32g274ardb2`
- `s32g274aevb3`: `<plat>` is `s32g2xxaevb3`
- `s32g254aevb3`: `<plat>` is `s32g2xxaevb3`
- `s32g233aevb3`: `<plat>` is `s32g2xxaevb3`



The above build steps assume that the boot medium is an SD card. However, the same TF-A FIP image can boot from eMMC as well. For QSPI boot, the BL33 parameter should contain the path to a U-Boot binary built specifically for QSPI compatibility. Please consult the Setting up and building U-Boot bootloader section for instructions on building an U-Boot binary for QSPI boot. For more information on how images can be written on any boot medium, please see the How to boot section.

3.2.4.1 Custom Build Parameters

TF-A provides build time parameters that allow TF-A customization for different booting flow scenarios. For example, if the boot target is M7, TF-A can be configured for a different memory layout. TF-A can read FIP image from a configurable location.

- For QSPI boot, configure TF-A to read the FIP image from a defined QSPI offset. Add **FIP_QSPI_OFFSET=<QSPI offset>** make parameter, as in the following example:

```
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- \
    ARCH=aarch64 PLAT=<plat> BL33=<path-to-u-boot-nodtb.bin> \
    FIP_QSPI_OFFSET=0x5400
```

- For SD/eMMC boot, configure TF-A to read the FIP image from a defined SD/eMMC offset. Add **FIP_MMC_OFFSET=<MMC offset>** make parameter, as in the following example:

```
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- \
    ARCH=aarch64 PLAT=<plat> BL33=<path-to-u-boot-nodtb.bin> \
    FIP_MMC_OFFSET=0x5400
```

- Configure TF-A to read the FIP image from a defined memory address instead of reading it from the boot source storage (QSPI or MMC).

  The memory location of the FIP image is set using the **FIP_MEMORY_OFFSET** compilation parameter and can be used in two cases:

  1. Loading the FIP image from a predefined memory location instead of a storage device
  2. BL2 in-place execution from FIP image

In the first case, TF-A uses the parameter to locate the image header and load the following stages from the given location, and in the second, the parameter contributes to in-place execution of the BL2 stage. In both cases, it is assumed the FIP image is copied to the specified memory address from outside TF-A. For example, the FIP image can be copied to specified SRAM address from the M7 bootloader configured as boot target, before starting BL2. This is how **FIP_MEMORY_OFFSET=<memory address>** is used for the first use case:

```
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- \
    ARCH=aarch64 PLAT=<plat> BL33=<path-to-u-boot-nodtb.bin> \
    FIP_MEMORY_OFFSET=0x34520000
```

In this case, BL2 is not executed in-place as part of the FIP image and must be stored at a location that does not overlap with the FIP image, which is expected to be loaded at **FIP_MEMORY_OFFSET**.

This feature can be paired with `BL2_BASE` to prepare the resulting image and BL2 stage for in-place execution. The in-place execution of the BL2 requires a three-step compilation:

1. Compile the TF-A using a custom base address for the BL2 stage if the default value is not suitable. For this purpose, the `BL2_BASE` must be appended to the compilation command.

```
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- \
    ARCH=aarch64 PLAT=<plat> BL33=<path-to-u-boot-nodtb.bin> \
    BL2_BASE=0x34100000
...
Boot Core:      A53_0
IVT Location:   SD/eMMC
Load address:   0x340f8bc0
Entry point:    0x34100000
...
```

2. Obtain the load address of the FIP image. In the above example, it is 0x340f8bc0 (see `Load address`)
3. Recompile the TF-A using `FIP_MEMORY_OFFSET` set to identified load address.

```
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- \
    ARCH=aarch64 PLAT=<plat> BL33=<path-to-u-boot-nodtb.bin> \
    BL2_BASE=0x34100000 \
    FIP_MEMORY_OFFSET=0x340f8bc0
```

It is important to note that using the **FIP_MEMORY_OFFSET** option during TF-A compilation will make the resulting binary larger due to the padding added to the BL2 stage to allow in-place execution.

- For BL33 and Linux to boot at EL2, TF-A compile flag **S32_BL33_AT_EL2** is set. To disable it and configure TF-A to boot BL33 and Linux at EL1, add **S32_BL33_AT_EL2=0** make parameter, as in the following example:

```
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- \+
-
    ARCH=aarch64 PLAT=<plat> BL33=<path-to-u-boot-nodtb.bin> \
    S32_BL33_AT_EL2=0
```

Some of the above-described options may alter the memory layout of the TF-A stages. There is a dedicated make target that can be used to inspect the resulting configuration:

```
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- \
    ARCH=aarch64 PLAT=<plat> BL33=<path-to-u-boot-nodtb.bin> \
    <some additional options> \
    memmap
```

3.2.5 Building the optee_os component of OP-TEE

This section is detailing the required steps for successfully building the optee_os component of the OP-TEE project. The other three components - optee_client, optee_examples and optee_test - are automatically built and deployed through the Yocto build (see Building OP-TEE-Enabled Image) because they should not be modified. For more details on each component, please refer to the OP-TEE chapter.

Prerequisites:

- Certain configurations must be enabled for both U-Boot and Linux so that OP-TEE can communicate with Linux and allow user space applications to call for TAs execution.
  While **building U-Boot**, one must make sure that both **CONFIG_OPTEE=y** and **CONFIG_OPTEE_LIB=y** are set. **Building Linux** requires both **CONFIG_OPTEE=y** and **CONFIG_OPTEE_SHM_NUM_PRIV_PAGES=256** to be set. For more information about obtaining and building any of the two, please see Setting up and building U-Boot bootloader and Setting up and building the Linux kernel.

- Two Python3 packages are mandatory for building OP-TEE. For easier package management and installation, the pip3 package management system for Python3 software should be installed. On Ubuntu-based hosts it is not present by default, so, if it has not been previously installed, run the following command:

  ```
  sudo apt-get install python3-pip
  ```

  Then, the required Python3 packages - pyelftools and pycryptodomex - can be installed using pip3:

  ```
  pip3 install pyelftools pycryptodomex
  ```

Follow these steps to build optee_os:

1. Obtain optee_os:

   ```
   git clone https://github.com/nxp-auto-linux/optee_os
   cd optee_os
   git checkout bsp42.0-4.0
   ```

2. Build optee_os:

   ```
   make CROSS_COMPILE64=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- ARCH=arm  \
   PLATFORM=s32 PLATFORM_FLAVOR=s32g2
   ```

The above steps should create the following two binary images: *out/arm-plat-s32/core/tee-header_v2.bin* and *out/arm-plat-s32/core/tee-pager_v2.bin*. Since OP-TEE can only boot with TF-A support, these two binaries **must be included in the FIP image generated when building the TF-A**. Please refer to the Building the Arm Trusted Firmware section for instructions on building an OP-TEE-enabled FIP image: all the prerequisites must be met and the build steps followed thoroughly.

4 How to boot

This chapter describes the steps to prepare an SD/MMC card and boot up for S32G2 EVB and S32G274A RDB2 boards. The boot modes of above boards are controlled by the boot configuration DIP switches and jumpers on the board. The following sections list basic boot setup configurations only.

The space between 0x0 and 0x28_0000 is occupied by some or all of the following components: IVT, QSPI Parameters, DCD, HSE_FW, SYS_IMG, Application Boot Code Header, TF-A FIP image. The actual layout is determined at boot time and can be obtained from the arm-trusted-firmware.

Example output:

IVT:	Offset: 0x1000	Size: 0x100
AppBootCode Header:	Offset: 0x1200	Size: 0x40
Application:	Offset: 0x1240	Size: 0x43200



For SD/eMMC the partitioned space begins at 0x40_0000. For more information see the support in meta-alb-bsp/classes/image_types_fsl_sdcard.bbclass. For QSPI, the region after 0x29_0000 is organized as follows:

Kernel	[0x029_0000 : 0x115_ffff]
DTB	[0x116_0000 : 0x118_ffff]
Rootfs	[0x119_0000 : 0x309_ffff]
PFE Firmware	[0x30a_0000 :]

For more information see the support in meta-alb-bsp/classes/image_types_fsl_flashimage.bbclass and include/configs/s32-cc.h

4.1 Writing images

4.1.1 Setup an SD/MMC card

Information found here describes the steps to prepare an SD/MMC card to boot up for S32G2 EVB and S32G274A RDB2 boards. The supported SD classes for SD are 4 and 10.

4.1.1.1 Requirements

An SD/MMC card reader is required. It will be used to transfer the boot loader and kernel images to initialize the partition table and copy the root file system. To simplify the instructions, it is assumed that a 4GB SD/MMC card is used.

Any Linux distribution can be used for the following procedure.

The Linux kernel running on the Linux host will assign a device node to the SD/MMC card reader. The kernel might decide the device node name or udev rules might be used. In the following instructions, it is assumed that udev is not used.

To identify the device node assigned to the SD/MMC card, enter the command:

```
cat /proc/partitions
major minor  #blocks    name
   8     0   85647168    sda
   8     1   82628608    sda1
   8     2          1    sda2
   8     5    3015680    sda5
```

```
11    0      58258    sr0
8     16   7707648    sdb
8     17   7703552    sdb1
```



The `lsblk` command may also be useful for finding the device node.

In this example it is assumed that the device assigned is `/dev/sdb`. Set `DEVSD` accordingly, e.g.

```
export DEVSD=/dev/sdb
```



Make sure the device node is correct for the SD/MMC card! Otherwise, it may damage your operating system or data on your computer!

There are two ways to prepare a bootable SD Card:

1. To use Comprehensive Yocto Image
   After successfully building Yocto, look for build result in
   <builddirectory>/tmp/deploy/images/<board_name>

   The .sdcard format creates an image with all necessary partitions and loads the bootloader, kernel and rootfs to this image.

   Ensure that any partitions on the card are properly unmounted before writing the card image, or you may end up with a corrupted card image. Also ensure to properly `sync` the filesystem before ejecting the card to ensure all data has been written.

   You can also low-level copy the data on this file to the SD card device using `dd` as in the following command example:

   ```
   sudo dd if=./fsl-image-base-s32g274aevb.sdcard of=${DEVSD} bs=1M && sync
   ```

   If you want to verify the SD card content against the original image, remove the SD card from the slot, insert the SD card once more and use cmp command in the following way:

   ```
   sudo cmp ./fsl-image-base-s32g274aevb.sdcard ${DEVSD}
   ```

   Yocto supports the generation of both `uImage` and `Image` kernel binary formats, but only the `Image` format works with the current configuration of U-Boot. It is not expected that uImage support will be restored in the future.

2. To manually copy BSP binaries into the SD Card, please follow the below chapters.

4.1.1.2 Creation of the partitions on the SD card

Our SD Card will be split into two main partitions:

- One for the kernel and board device tree blob;

- Another for the root file system that will run on the board.

An additional, unpartitioned/unformatted space will also be created at the beginning of the SD card to hold the U-Boot and TF-A bootloaders.

To create the partitions, enter the following command:

```
sudo fdisk ${DEVSD}
```

Then type the following parameters to the fdisk console (each followed by <ENTER>):

```
d [repeat this until no partition is reported by the 'p' command]
n [create a new partition]
p [create a primary partition]
1 [the first partition]
8192 [create a partition that starts at offset 8192; this leaves 4MB of
      unpartitioned space for the bootloader]
+255M [size of the actual partition = 255 MB]

[It may be that fdisk detects a previous filesystem signature and asks for confirmation of erase:
"Partition #1 contains a vfat signature.
Do you want to remove the signature? [Y]es/[N]o:".    Answer Y in this case.]

p [list the parameters of the previously created partition, look at the number of the End sector: 530431]
n [create a new partition]
p [create a primary partition]
2 [the second partition]
530432 [Start sector of the new partition; partition will be adjacent to the previous one]
<enter>[using the default value will create a partition that uses the remaining space on the card]

t [set partition type]
1 [partition #1]
c [FAT32]

t [set partition type]
2 [partition #2]
83 [Linux]

w [ this writes the partition table to the medium and fdisk exits]
```



The fdisk version used for the above commands was 2.31.1 from Ubuntu package util-linux.

These newly created partitions need to be formatted to be usable. Firstly, remove and reinsert SD card into its slot, then run the following commands to format the partitions:

```
sudo mkfs.vfat -n boot ${DEVSD}1
sudo mkfs.ext4 -L rootfs ${DEVSD}2
```

4.1.1.3 Copying the BSP binaries onto the SD Card



Before proceeding, ensure you have downloaded the SD card binary from flexnet account.

If you remove the SD card from its slot after it has been formatted and you reinsert it, you should see two partitions in your file explorer program (in this example it is assumed that your user is 'public'):

- boot (in */media/public/boot*)
- rootfs (in */media/public/rootfs*)

If the partitions are not mounted automatically, use the following steps to mount them. If you do not know the device node assigned to the SD, see the "Requirements" section.

```
export SD_MOUNT_POINT=/media/public
sudo mount ${DEVSD}1 ${SD_MOUNT_POINT}/boot
sudo mount ${DEVSD}2 ${SD_MOUNT_POINT}/rootfs
```

Run the following commands to copy the necessary binaries onto SD Card:

```
export SD_MOUNT_POINT=/media/public
cd <builddirectory>/tmp/deploy/images/<board_name>
sudo dd if=fip.s32 of=${DEVSD} conv=notrunc,fsync seek=512 skip=512 oflag=seek_bytes iflag=skip_bytes
sudo cp Image ${SD_MOUNT_POINT}/boot/
sudo cp <dtb_file> ${SD_MOUNT_POINT}/boot/<dtb_file>
sudo tar xf <rootfs> -C ${SD_MOUNT_POINT}/rootfs
sync
```

Unmount the file systems and eject cleanly the SD card.



If targeting the S32G2 EVB board, copy *s32g2xx-evb.dtb* to *${SD_MOUNT_POINT}/s32g2xx-evb.dtb*.
For manual component building, the component names and paths should be updated accordingly to the corresponding build path. For example, for Linux use dtb file *arch/arm64/boot/dts/freescale/s32g2xx-evb.dtb* and image file *arch/arm64/boot/Image*

4.1.2 Setup QSPI memory

4.1.2.1 Generating the .flashimage file for booting from QSPI

The simplest method of generating a flash bootable image from yocto is with

```
bitbake fsl-image-flash
```

which will produce in *build_*/tmp/deploy/images/<machine>* a file named *fsl-image-flash-<machine>.flashimage*, an equivalent in terms of packages with fsl-image-base, but ready to be written to QSPI memory (see chapter "Booting from QSPI" about burning a file into flash).

By default, `fsl-image-flash` contains the root file system packages defined in `fsl-image-base`. To use root file system packages from a different, custom image, the following changes should be done: in file meta-alb-bsp/conf/machine/include/s32cc-board.inc, update variable `FLASHIMAGE_ROOTFS` to the desired custom image. For example:

```
FLASHIMAGE_ROOTFS = "<custom image name>"
```

All that is left is to write the .flashimage file to the beginning of QSPI memory. More details can be found in chapter "Booting from QSPI".



- The custom image should be small enough to fit into the allocated memory map. If the size of the image is too high, an error is thrown when building *fsl-image-flash*.
- By default, *fsl-image-flash* will boot using a RAM-based filesystem, without a backing store. Thus, all altered or generated files will return to their original state after a board reset.



With respect to ERR051257, which is applicable for S32G2XXA Rev 2.0, U-Boot should be compiled with CONFIG_ERR051257_WORKAROUND enabled if the errata applies. By default, this config option is enabled only for S32G2XXA RDB and should be disabled if S32G2XXA Rev 2.1 is used.

4.1.3 Setup eMMC

Information found here describes the steps to prepare the eMMC memory to be used for Linux BSP

Build U-Boot

```
export CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu-
make -j9
```

Method 1: Write using Flash Tool

Connect the board to your workstation using an USB cable.

Load Flash Tool on chip

```
./bin/S32FlashTool -t ./targets/S32G2xxx.bin -a ./flash/EMMC.bin -i uart -p /dev/ttyUSB0
```

Write U-Boot image

```
./bin/S32FlashTool -t ./targets/S32G2xxx.bin  -fwrite -addr 0 -f ./fip.s32 -i uart -p /dev/ttyUSB0
```

Method 2: Write from QSPI/SD

Load the desired image into DDR (from QSPI/SD or any other sources). It can be fip.s32, containing TF-A and U-Boot images, or SD card image. The below examples will load fip.s32 binary or an eMMC image from FAT32 partition and write it to eMMC.

TF-A and U-Boot update

```
=> setenv image fip.s32
=> run loadimage
```

```
=> setexpr loadaddr ${loadaddr} + 0x1000
```

Jump over the MBR during TF-A and U-Boot update

SD card image update

```
=> setenv image fsl-image-base-s32g274aevb.sdcard
=> setenv loadaddr 0x86000000
=> run loadimage
```

Calculate number of eMMC blocks needed for the loaded image.

```
=> setexpr n_blocks ${filesize} / 0x200
=> setexpr n_blocks ${n_blocks} - 7
```

```
=> setexpr n_blocks ${filesize} / 0x200
=> setexpr n_blocks ${n_blocks} + 1
```

Make eMMC available by adapting board jumpers / switches
On S32G2 EVB board this can be done by placing J50 on position 2-3
On S32G274A RDB2 board this can be done by setting SW3 to OFF

Write the image from DDR to eMMC starting with IVT.

```
=> mmc rescan
=> mmc write ${loadaddr} 8 ${n_blocks}
```

Write the image from DDR to eMMC starting with block 0x0.

```
=> mmc rescan
=> mmc write ${loadaddr} 0 ${n_blocks}
```

Power off the board and adapt switches configuration for eMMC boot. For more information on this please refer to section “Booting from eMMC”

4.2 Board set-up and boot

After boot set-up is complete using one of the methods presented in previous section, follow these steps to set-up the target board:

- Power off the Target board system if the power is already on.
- Connect the Target board to the host machine via the serial port with an RS-232 cable
- Connect the Target board to the network via an Ethernet port on the board, if needed.
- Start the serial console tool (such as Minicom or TeraTerm) on the host system. Details about selecting a specific baud rate can be found in Changing UART baud rate.
- Verify that all switches and jumpers are setup correctly as described in section “Boards”.
- Power on the board.
- The sections below describe steps to boot Linux with corresponding deployment method.

4.2.1 Booting from SD

The procedure for running Linux is the following:

1. Prepare the SD card setup using one of the methods presented in section “Setup a SD/MMC card”.
2. Insert the SD Card into SD Card socket.
3. Verify that all switches and jumpers are setup correctly for booting from SD as described in section “Boards”.
4. Reset the board.

5. The U-Boot should start to print on the console a count down, and then it will start the Linux kernel. If any key is pressed during the U-Boot countdown, boot process will stop at U-Boot prompt. Run the following commands to boot Linux:

   - Load the image:

     ```
     => run loadimage
     ```

   - Load the dtb:

     ```
     => run loadfdt
     ```

   - Set the bootargs in order to use the rootfs from SD:

     ```
     => run mmcargs
     ```

   - Start the booting process:

     ```
     => ${boot_mtd} ${loadaddr} - ${fdt_addr}
     ```

6. After that you should be prompted with a Linux console.

How to reset U-Boot environment

When reusing an SD card with a new version of U-Boot, if `saveenv` was previously run in the U-Boot console, the new default environment might not be visible. To clear environment variables previously saved with U-Boot, the following steps should be performed:

1. Print one, several or all variables of the U-Boot environment

   ```
   printenv <variable>
   printenv <variable_1> <variable_2> ...
   printenv
   ```

2. Reset the environment to the default values of the build

   ```
   env default -fa
   ```

3. Save the currently loaded environment to persistent storage (SD/HyperFlash)

   ```
   saveenv
   ```

4.2.2 Booting from eMMC

The procedure for running Linux is the following:

1. Prepare the eMMC setup using one of the methods presented in section “eMMC setup”..
2. Verify that all switches and jumpers are setup correctly for booting from eMMC as described in section “Boards”.
3. Reset the board.
4. The U-Boot should start to print on the console a count down, and then it will start the Linux kernel.
5. After that you should be prompted with a Linux console.



Sometimes characters are lost when copy-paste. To fix that, a transmit delay of 1 ms/char should be set in the terminal client.

4.2.3 Booting from TFTP

To load Image and dtb files using TFTP it required to setup a TFTP server and to do the TFTP client settings in U-Boot. A prerequisite for TFTP is to configure Ethernet (see chapter ‘S32G274A Ethernet Support’) to have IP connectivity from U-Boot.

TFTP server

During testing phase, the TFTP booting was validated using an Ubuntu-20.04 LTS workstation which had installed the tftp-hpa and tftpd-hpa services:

```
apt-get install tftp-hpa tftpd-hpa
```

The instructions for the configuration of TFTP server are outside the scope of this document. For more details please consult the tutorials which are available on the Internet.

U-Boot setup

- Power-on the board and stop at U-Boot prompt
- Setup the interface IP and TFTP server IP:

```
=> set ipaddr <yourIPv4>
=> set serverip <serverIPv4>
```

- Test the connectivity with server:

```
=> ping <serverIPv4>
```



If the antivirus or firewall settings are too restrictive, it’s likely to drop the U-Boot ARP packets considering them an ARP injection attack.

- Download Image and dtb files

```
=> tftp ${loadaddr} Image
=> tftp ${fdt_addr} <dtb_file>
```



The *Image* and <*dtb_file*> files should be available in the root directory of TFTP server.

- Set the bootargs in order to use the rootfs from SD.

```
=> run mmcargs
```

- Start the booting process

```
=> booti ${loadaddr} - ${fdt_addr}
```

4.2.4 Booting from NFS (root filesystem)

After U-Boot has been loaded (either from QSPI or from SD), the boot process can be continued with files obtained via Ethernet, but the U-Boot configuration must be changed in advance to prepare for this boot flow.



The Ubuntu images on all supported machine does not currently support NFS booting.



This boot flow is currently validated on:

- S32G2 EVB

The default U-Boot configuration for the S32G2 EVB board contains some predefined commands which can be used to continue the boot process using Ethernet. The default configuration will attempt to access the Linux kernel, dtb and root filesystem from the SD card, so it is necessary to change the configuration to prevent the default boot flow. The following steps should be performed:

- Configure another system to be both a TFTP and NFS server which can serve files for S32G2 EVB. The TFTP server software will serve the Image and the dtb files, while the NFS server software will serve the S32G2 EVB root filesystem built with Yocto.
  - make sure the NFS share is exported from */tftpboot/rfs* on the server and it can be mounted, accessed and modified from another linux system
- Stop the U-Boot counting down in order to have access to console
- Change the `image`, `fdt_file` variables to match the files on server
- If necessary, change the `ipaddr`, `serverip` and/or `netmask` configurations to match the used set up.
- After all configurations are correct, from the U-Boot prompt run `run nfsboot` to continue the boot process.

4.2.5 Booting from QSPI

There are at least two ways of writing images into the QSPI:

- Using **S32FlashTool** from **S32 Design Studio**
- Through U-Boot or Linux by loading all the binaries over an Ethernet connection or from SD

The following steps will only cover the second option. Please refer to the **S32 Design Studio** documentation for more details about **S32FlashTool** programmer.

Perform the following steps in the U-Boot or Linux console after you have prepared all the binaries involved in a Linux boot (FIP image, Linux Kernel image, DTB and filesystem) on SD card or on a TFTP server.



MX25UW51245G supports full flash erase command which is faster than erasing the flash memory sector by sector. One could use the following command to erase the entire memory before writing the images instead of erasing individual blocks, by using the QSPI partition which maps to the entire flash memory:
In U-Boot

```
=> run flashbootargs
=> sf probe 6:0
=> sf erase 0 ${flashsize}
```

In Linux

```
root@s32g274aevb:~# flash_erase /dev/mtd0 0 0
```

For the following steps, in case of using Linux, it is assumed that all binaries are in the current working directory. Furthermore, under '/opt/bsp_utils/' there is the 's32cc_qspi_write.sh' bash script available which can be used to write files in the QSPI Flash Memory.

1. Optional: Inspect QSPI Flash Memory partitions
   In U-Boot

```
=> mtd list
SF: Detected mx25uw51245g with page size 256 Bytes, erase size 64 KiB, total 64 MiB
List of MTD devices:
* nor0
  - device: mx25uw51245g@0
  - parent: spi@40134000
  - driver: spi_flash_std
  - type: NOR flash
  - block size: 0x10000 bytes
  - min I/O: 0x1 bytes
  - 0x000000000000-0x000004000000 : "nor0"
          - 0x000000000000-0x000004000000 : "Flash-Image"
          - 0x000000000000-0x000000280000 : "FIP"
          - 0x000000280000-0x000000290000 : "U-Boot-Env"
          - 0x000000290000-0x000001160000 : "Kernel"
          - 0x000001160000-0x000001190000 : "DTB"
          - 0x000001190000-0x0000030a0000 : "Rootfs"
          - 0x0000030a0000-0x000004000000 : "PFE-Firmware"
```

   In Linux

```
root@s32g274aevb:~# cat /proc/mtd
dev:    size   erasesize  name
mtd0: 04000000 00001000 "Flash-Image"
mtd1: 00280000 00001000 "FIP"
mtd2: 00010000 00001000 "U-Boot-Env"
mtd3: 00ed0000 00001000 "Kernel"
mtd4: 00030000 00001000 "DTB"
mtd5: 01f10000 00001000 "Rootfs"
mtd6: 00f60000 00001000 "PFE-Firmware"
```

2. Write complete Flash Image generated by Yocto

   When writing the entire fsl-image-flash image in U-Boot, we change the RAM load address in order to not overlap with the reserved memory ranges.
   Also, the U-Boot steps below assume that all binaries are stored on SD, on the first partition (FAT). The `loadimage` command has to be adapted if the images are placed on a different partition or it can be replaced with `loadtftpimage` in case the TFTP transfer is preferred.

   In U-Boot

```
=> run flashbootargs
=> sf probe 6:0
=> setenv image fsl-image-flash-<machine>.flashimage
=> setenv loadaddr 0x86000000
=> run loadimage
=> sf erase 0 ${flashsize}
=> mtd write Flash-Image ${loadaddr}
```

   In Linux

```
root@s32g274aevb:~# cd /opt/bsp_utils/
root@s32g274aevb:~# ./s32cc_qspi_write.sh Flash-Image \
fsl-image-flash-<machine>.flashimage
```

3. Power off the board and set the jumpers for QSPI boot. These settings are covered by "Boards" section.



Instead of following steps 2 and 3, another option is to individually write each image as exemplified below. Each image such as: FIP, Linux Kernel, DTB, Ramdisk is stored in QSPI using a MTD partition and the following steps will update the partitions content one by one.

4. Update FIP partition containing TF-A and U-Boot images

   In U-Boot

```
=> setenv image fip.s32-qspi
=> run loadimage
=> mtd erase FIP
=> mtd write FIP ${loadaddr}
```

   In Linux

```
root@s32g274aevb:~# cd /opt/bsp_utils/
root@s32g274aevb:~# ./s32cc_qspi_write.sh \
FIP fip.s32-qspi
```

5. Update partition containing Linux Kernel image

In U-Boot

```
=> setenv image Image
=> run loadimage
=> mtd erase Kernel
=> mtd write Kernel ${loadaddr}
```

In Linux

```
root@s32g274aevb:~# cd /opt/bsp_utils/
root@s32g274aevb:~# ./s32cc_qspi_write.sh \
Kernel Image
```

6. Update DTB partition containing Linux device tree

In U-Boot

```
=> setenv image ${fdt_file}
=> run loadimage
=> mtd erase DTB
=> mtd write DTB ${loadaddr}
```

In Linux

```
root@s32g274aevb:~# cd /opt/bsp_utils/
root@s32g274aevb:~# ./s32cc_qspi_write.sh \
DTB <fdt_file>
```

7. Update Rootfs partition containing Linux filesystem

In U-Boot

```
=> setenv image \
fsl-image-base-<machine>.cpio.gz.u-boot
=> run loadimage
=> mtd erase Rootfs
=> mtd write Rootfs ${loadaddr}
```

In Linux

```
root@s32g274aevb:~# cd /opt/bsp_utils/
root@s32g274aevb:~# ./s32cc_qspi_write.sh \
Rootfs \
fsl-image-base-<machine>.rootfs.cpio.gz.u-boot
```

8. Update PFE-Firmware partition containing PFE FW

In U-Boot

```
=> setenv image s32g_pfe_class.fw
=> run loadimage
=> mtd erase PFE-Firmware
=> mtd write PFE-Firmware ${loadaddr}
```

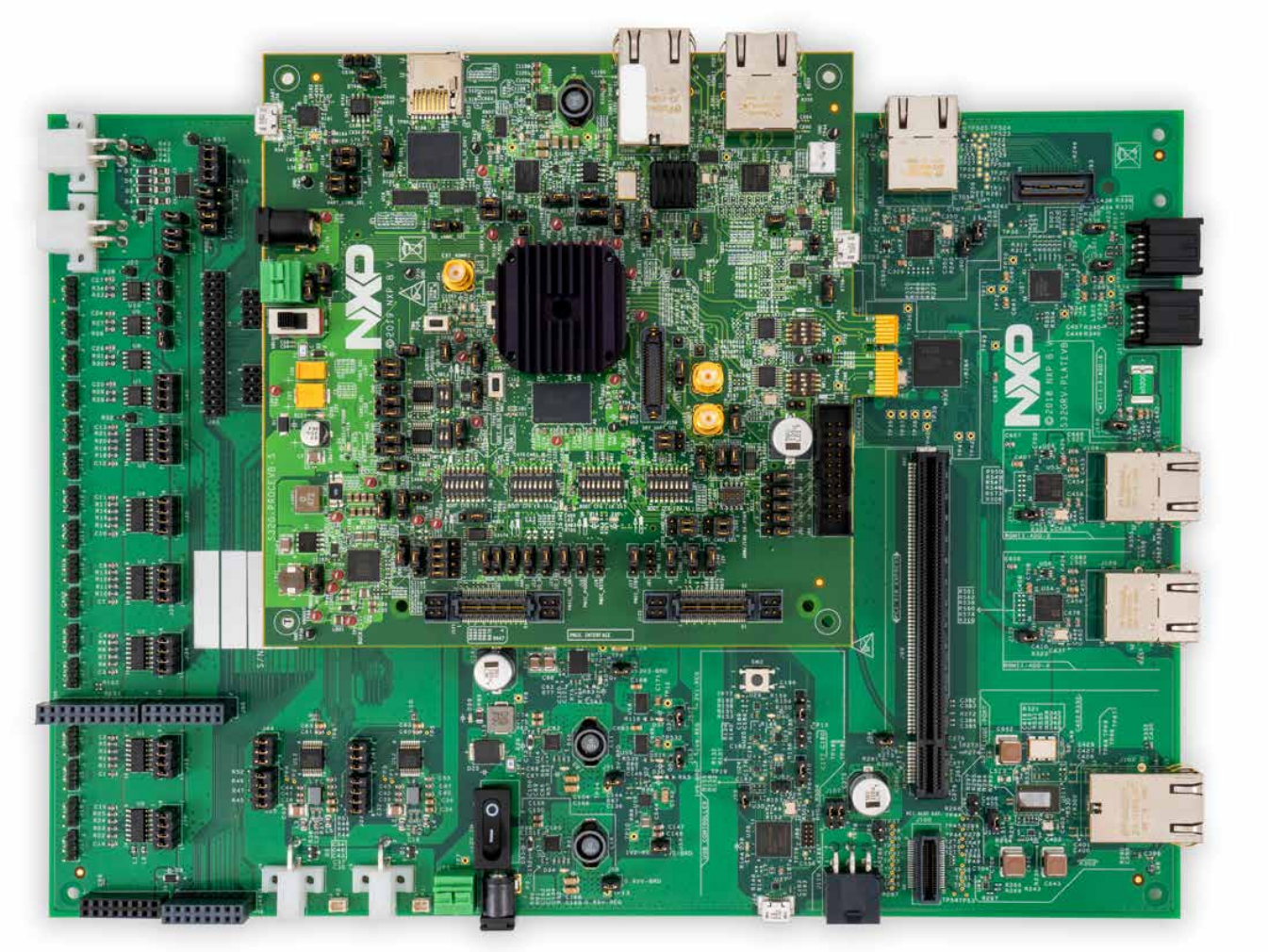
In Linux

```
root@s32g274aevb:~# cd /opt/bsp_utils/
root@s32g274aevb:~# ./s32cc_qspi_write.sh \
PFE-Firmware s32g_pfe_class.fw
```

9. Power off the board and set the jumpers for QSPI boot. These settings are covered by “Boards” section.

4.3 Boards

4.3.1 S32G2 EVB



This chapter describes the steps to configure the boot jumpers and switch configuration on the board S32G2 EVB SCH-32170 REV B1.

With respect to power source selection, in case S32G-PROCEVB-S is used without S32G-PLATEVB-S the default position of J96 (2-3) is enough. However, shall the user use S32G-PROCEVB-S in conjunction with S32G-PLATEVB-S, J96 needs to be set to position 1-2. The effect of this jumper position change is that the power supply of the S32G-PROCEVB-S is now taken from S32G-PLATEVB-S (instead from the daughter card, as in the default position).

The current pinmuxing configuration for UART using LF0 requires that jumpers J124 on the S32G-PROCEVB-S board be in the 1-3, 2-4 position, which is their default position but is different from BSP22.5.

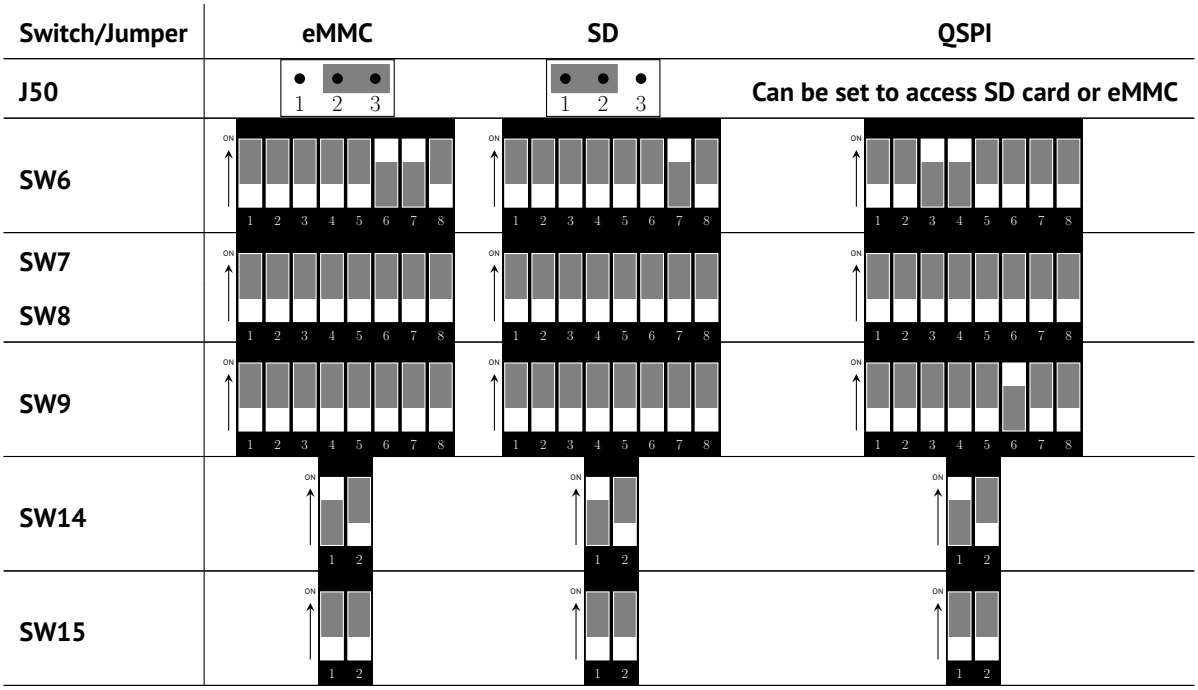
Jumpers J159 should be in the 1-3, 2-4 position (default configuration).

The S32G2 EVB board supports booting from parallel RCON, as well as serial RCON. In the parallel RCON boot

scenario, switches SW6 - SW9, which are connected to RCON's 32 general-purpose I/O pins (RCON[0] to RCON[31]), select the boot configuration. When booting from serial RCON, the boot configuration is read from an external serial EEPROM that communicates with the chip via I^2C. Selecting between serial/parallel RCON boot modes is done by toggling RCON[8] i.e. RCON[8] = 0b when booting from parallel RCON, whereas RCON[8] = 1b when booting from serial RCON. Once the RCON boot mode is selected, one should choose the boot medium (SD, eMMC or QSPI) by setting up RCON[0:7].

4.3.1.1 Boot from parallel RCON

In this mode switches SW6 - SW9 select the values of RCON[0:31], each switch representing 1 byte in the RCON[0:31], i.e. SW6 represents RCON[0:7], SW7 represents RCON[8:15] and so on. SW6 selects the boot medium. These are the settings of the jumpers and switches which select SD, eMMC or QSPI booting:



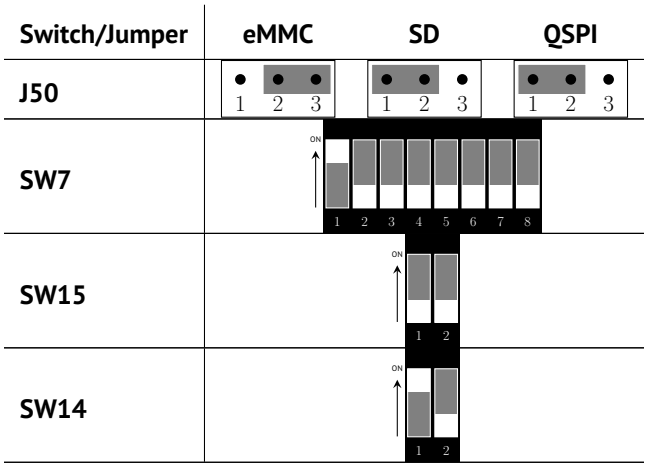
The rest of the switches and jumpers are set to default position.

4.3.1.2 Boot from serial RCON

In this mode, the first 4 bytes residing in EEPROM represent the boot configuration. The first byte represents RCON[0:7], the second byte represents RCON[8:15] and so on. The first byte selects the boot medium. The EEPROM memory should have the following layout, according to the desired boot medium (SD, eMMC or QSPI):

EEPROM Offset	eMMC	SD	QSPI
0h	0x60	0x40	0x0
1h	0x1		
2h	0x7		
3h	0x0		

Jumpers and switches should have the following values:



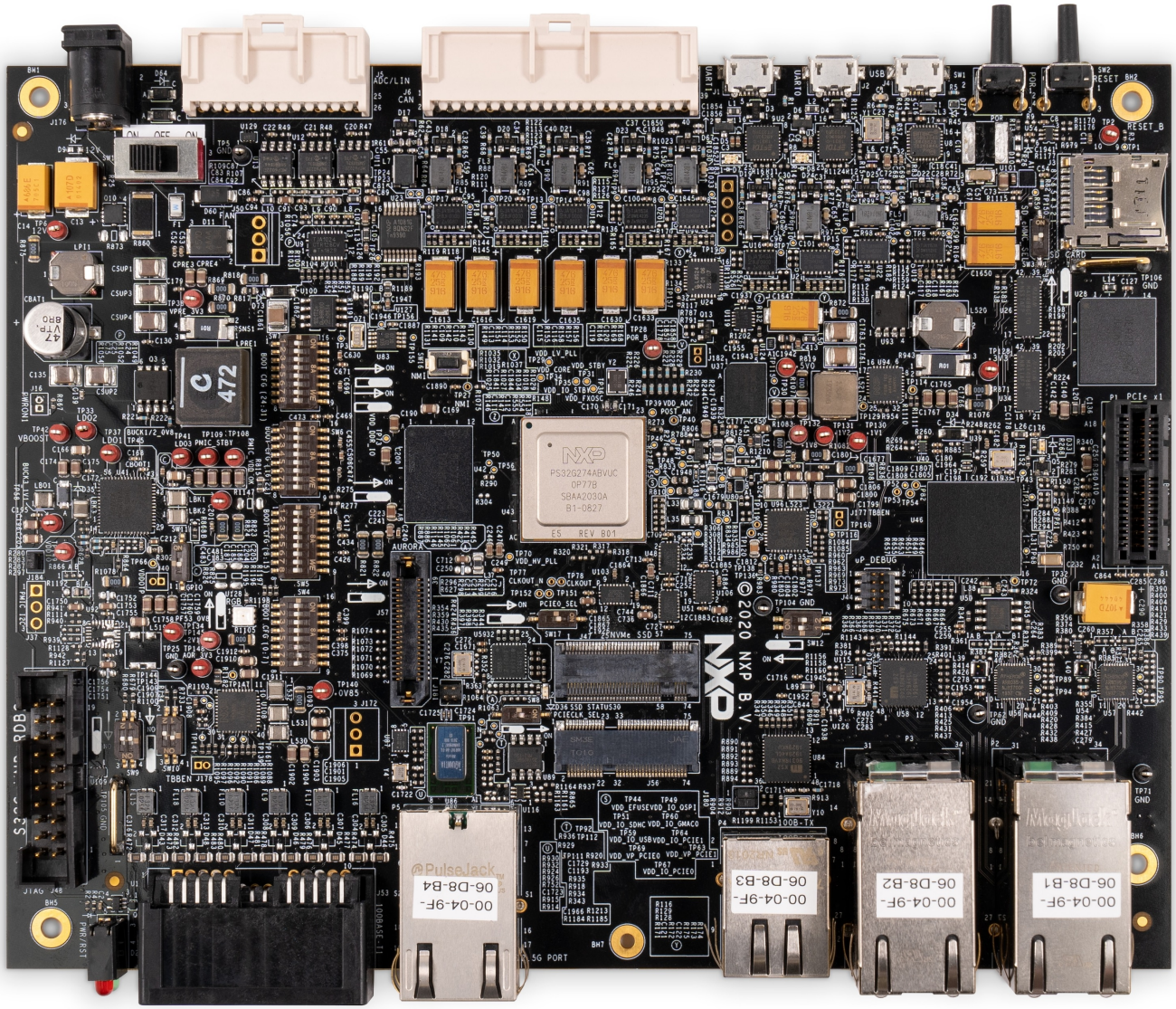
The values of SW6, SW8 and SW9 are irrelevant. The rest of the switches and jumpers are set to default position.

One possible way of populating the first 4 bytes of the I^{2}C-connected EEPROM with an SD boot configuration, for example, is by issuing the following commands in U-Boot:

```
=> i2c dev 0
=> i2c mw 0x50 0.1 0x40
=> i2c mw 0x50 1.1 0x01
=> i2c mw 0x50 2.1 0x07
=> i2c mw 0x50 3.1 0x00
```

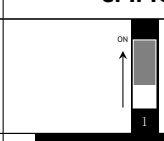
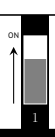
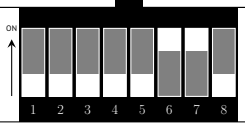
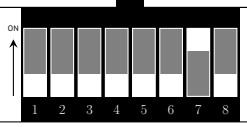
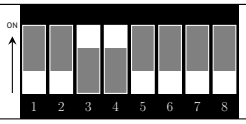
Power off the board, make sure SW7 is set to 0x1 and then power on the board. The newly written boot configuration is fetched from EEPROM no matter what the values of switches SW6, SW8 and SW9 are. Please note that the second command in the above sequence sets the boot medium according to the first table in the paragraph "Boot from serial RCON".

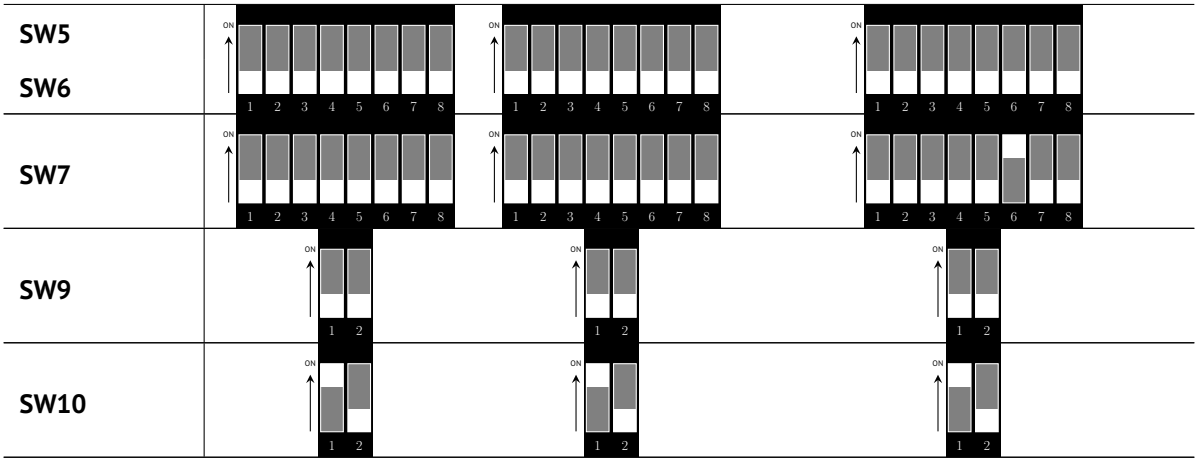
4.3.2 S32G274A RDB2



This chapter describes the switch configuration on the board S32G274A RDB2 SCH-47440 REV C2.

These are the settings of the jumpers and DIP switches which select SD, eMMC or QSPI booting:

Switch/Jumper	eMMC	SD	QSPI
SW3			Can be set to access SD card or eMMC
SW4			



The rest of the switches and jumpers are set to default position.

5 SMP Boot

The S32G2 BootROM can start either the M7_0 or the A53_0 core. This document assumes the boot core is A53. The A53_0 core is thus considered the "primary" (or "master") core (or CPU), while cores 1, 2 and 3 are considered "secondary" (or "slave") cores (or CPUs).

Secondary cores with Linux SMP boot on Armv8 are managed through PSCI calls [1].

An Arm Trusted Firmware (TF-A) [2] implementation is responsible for implementing PSCI functions related to CPU management, in particular SMP boot.

5.1 TF-A SMP Boot

Secondary cores are managed by the Trusted Firmware. At cold boot, the secondary cores are in reset. During kernel boot, Linux uses `PSCI_CPU_ON` calls to enable individual cores. PSCI calls trap at Exception Level 3 (EL3) and are executed by the Trusted Firmware's Secure Monitor component.



U-Boot does not touch the secondary cores. It is the TF-A Secure Monitor that hands them over to Linux - see section "TF-A SMP Boot".

5.2 Linux Kernel SMP

In the Linux device-tree source file, the `cpus` parent node contains children for each `cpu<n>`. The bindings for the `cpu` nodes are specified in the kernel documentation: Documentation/arm64/booting.rst. The device-tree included in the Linux BSP sources specifies the `psci` as `enable-method` by default.

The following is an excerpt of the Linux device-tree `cpus` node:

```
cpus {
        #address-cells = <2>;
        #size-cells = <0>;

        cpu0: cpu@0 {
                device_type = "cpu";
                compatible = "arm,cortex-a53";
                reg = <0x0 0x0>;
                enable-method = "psci";
        };
        [...]
};
```

[1] More details about PSCI can be found in Arm documentation

[2] Please see chapter 25 for more details about Arm Trusted Firmware

6 Lockstep Operation

Lockstep mode is an operation mode of the chip that runs the same set of operations on two cores or clusters. The chip supports lockstep operation for both Cortex-M7 and Cortex-A53 cores. In this document, lockstep operation refers to Cortex-A53 lockstep operation – as Cortex-A53 is the core where Linux BSP runs.

The chip supports two modes of operation for Cortex-A53 clusters.

- Decoupled mode (or performance mode) allows the two Cortex-A53 clusters to operate in an independent manner. All the four Cortex-A53 are available for the software applications.
- Lockstep mode: the two Cortex-A53 clusters operate in delayed lockstep configuration, that is:
  - core0 of cluster0 is in lockstep with core0 of cluster1
  - core1 of cluster0 is in lockstep with core1 of cluster1

  Only cluster0 is available in the system for software applications. Cluster1 is not visible but works as a checker or redundant module.

The default mode of operation for the Cortex-A53 clusters is the decoupled mode. The operation mode (decoupled or lockstep mode) can be changed only when A53 reset domain is in the reset state. This means that the operation mode cannot be set from software running on Cortex-A53. This can only be done from software running on Cortex-M7 while A53 reset domain is in the reset state.

In the Linux BSP default booting flow, the boot target is Cortex-A53, as set in the Image Vector Table (IVT) Boot configuration word. In this case, Cortex-A53 core0 in cluster0 is the first enabled core that can further start any other core. This flow enables the default mode of operation, that is, decoupled mode.

Lockstep mode of operation can be enabled by changing the default booting flow. In this case, the IVT Boot configuration word should be changed to set the Cortex-M7 as the boot target. The software running on Cortex-M7 enables the Cortex-A53 lockstep mode by setting the `A53_GPR06[CA53_LOCKSTEP_EN]` register bit while the Cortex-A53 is still in reset state. After lockstep is enabled, the Cortex-A53 core is also enabled and it starts in lockstep mode.

Linux BSP software running on Cortex-A53 can operate in lockstep mode with no supplementary configuration or software changes. However, the Cortex-M7 software and IVT Boot configuration that enable lockstep mode need to be updated. To demonstrate lockstep operation mode, Linux BSP provides a booting flow example with Cortex-M7 set as boot target. The M7 booting flow can be enabled as described in section Building Images with M7 as Boot Target

6.1 Enablement flow

Once Cortex-A53 core0 (core0 from cluster0) has started, the software (TF-A or U-Boot) running on Cortex-A53 core0 dynamically handles both decoupled and lockstep modes of operation. The mode of operation is detected at run-time such that there is no configuration for Cortex-A53 software to enable or disable the lockstep mode. The same binary images (for TF-A, U-Boot and Linux) handle lockstep or decoupled modes. The Cortex-A53 software performs the following actions specific to lockstep mode:

- Reset AArch64 Generic Timer registers. The complete list of Generic Timer registers is specified in the SoC's Reference Manual. Reset means that, for each of the mentioned registers, the value 0x0 is written to the register before performing any read access to it. The software running on a Cortex-A53 core can only reset its core registers. For example, software running on Cortex-A53 core0 can only reset core0 registers, but not core1 registers. Therefore, the reset operation is performed executing the same procedure from each of the available Cortex-A53 cores. The purpose of the reset is to guarantee the same registers values for the lockstep cores. In

lockstep mode only cluster0 is visible for the Cortex-A53 software such that the software performs the reset only for core0 and core1 of the cluster0. It is the SoC's lockstep mechanisms that automatically performs the reset for the redundant cores. Therefore, a software reset to core0 cluster0 registers means that hardware performs the same write operation to the core0 cluster1 registers. Similarly, a software reset to core1 cluster0 registers triggers the hardware to perform the reset for core1 cluster1 registers.

- Reset general purpose registers. The same procedure described for AArch64 Generic Timer but applied to the generic purpose registers: X0, X1, …., X30. For general purpose registers, the reset value may be different than 0x0 as needed in the assembly code. However, the goal is to always write a value before performing a read access to these registers.
- For lockstep mode only cluster0 is visible to Cortex-A53 software, therefore cluster1 should not be accessed. The following actions are performed:
  - NCore is configured only for cluster0. Refer to the SoC Reference Manual for more information about NCore.
  - Secondary cores are enabled as described in section SMP Boot but only core1 from cluster0 is available as secondary core.

6.2 Errors handling

In lockstep mode, the equivalent operation of the redundant resources (GIC, cluster) is supervised by dedicated comparators, part of the Redundancy Control and Checker Unit (RCCU). Any operation deviations between the active and the redundant resource causes the Fault Collection and Control Unit (FCCU) to be notified.

The Linux FCCU driver allows updating the list of Non-Critical Faults (NCF) handled in FCCU and selecting one of the supported NCF actions using device tree. The NCF lockstep errors and their actions can be configured as part of the FCCU driver configuration in device tree. One example for device tree configuration for lockstep error handling in FCC is the following:

```
fccu: fccu@4030C000 {
        [...]
        /* A53 NCF fault list */
        nxp,ncf_fault_list = <0 10 35 36 37 38>;
        nxp,ncf_actions = <S32_FCCU_REACTION_ALARM
            S32_FCCU_REACTION_ALARM
            S32_FCCU_REACTION_NONE
            S32_FCCU_REACTION_NONE
            S32_FCCU_REACTION_NONE
            S32_FCCU_REACTION_NONE>;
        status = "disabled";
};
```

In this example, lockstep NCF handling is enabled by adding 0 (A53 lock-step RCCUs combination of all redundant signals from A53 RCCUs) and 10 (A53 RCCU lock-step error A53 clusters) to FCCU's NCF list. The NCF's action can be selected from the following options:

- No action on detecting the NCF. The status bits are cleared at driver initialization
- Alarm: an alarm is generated and handled in the FCCU's driver interrupt handler. The alarm information is printed in the kernel's log (dmesg).
- Functional reset is triggered when a NCF error is detected
- Non Maskable Interrupt is triggered when a NCF error is detected
- EOUT signal is asserted when a NCF error is detected

7 Ethernet Information

7.1 Prerequisites

PFE Linux driver relies on PFE firmware, which is a separate NXP product available through https://www.nxp.com/.

The compatible version number can be found in PFE Linux driver Release Notes. The required firmware has to be downloaded from NXP Software Center.

For more information, and to find out how to obtain PFE Firmware, please contact NXP marketing department.

7.2 Ethernet Hardware Support on S32G2

7.2.1 Ethernet support on S32G2 SoC

The SoC contains two Ethernet controllers:

1. **GMAC**
2. **PFE** - an Ethernet accelerator with three EMACs (PFE_EMAC_0, PFE_EMAC_1 and PFE_EMAC_2; abbreviated: PFE0, PFE1, PFE2).

The controllers can be programmed to use MII/RMII/RGMII and SGMII interface modes supporting 10/100/1000/2500 Mbit/s speeds.



The SoC contains three RGMII blocks which can be connected to the GMAC/PFE EMAC ports in a limited number of combinations.

The SGMII block shares the SerDes subsystem with PCIe. S32G2 contains two SerDes blocks with two lanes each. By default, SerDes_0 is used by PCIe and SerDes_1 by SGMII.



The 10Mbps MII/RMII mode only works with PHYs that support octet-only alignment.

7.2.2 SerDes configuration for SGMII in U-Boot

To use SGMII interface it is necessary to configure SerDes. S32G supports two SerDes instances and each instance has multiple configurations available. As the SerDes configured with SGMII can be also shared with PCIe additional care is necessary with configuration. The configuration itself is performed via TF-A device tree and U-Boot `hwconfig` environment variable.

To enable SerDes, the `serdes` nodes in the TF-A device tree have to be enabled (one per SerDes) in fdts/s32cc.dtsi.

```
serdes0: serdes@40480000 {
        #phy-cells = <3>;
        compatible = "nxp,s32cc-serdes";
        reg = <0x00 0x40480000 0x0 0x04000  /* serdes registers */
              0x00 0x40400000 0x0 0x80000>; /* dbi registers */
        reg-names = "ss", "dbi";
        #address-cells = <3>;
        #size-cells = <2>;
        device_type = "pci-generic";
        device_id = <0>;
        num-lanes = <2>; /* supports max 2 lanes */
```

```
        clocks = <&clks S32CC_SCMI_CLK_SERDES_REF>;
};
```

The `hwconfig` can be displayed via `printenv hwconfig`.
The format for this variable is (all on one row):

```
=> setenv hwconfig "serdes0:<ss_settings_0>[;pcie0:<pci_settings_0>][;xpcs0_j:<xpcs_settings_0_j>];
                    serdes1:<ss_settings_1>[;pcie1:<pci_settings_1>][;xpcs1_j:<xpcs_settings_1_j>]"
```

Where:

- `<ss_settings_i>` is :

  `mode=<ss_mode_i>,clock=<clock_i>[,skip=<value_i>][,fmhz=<fmhz_i>]`

  `i` is the index of the SerDes module (0 or 1), and the valid values for SerDes options `mode`, `clock`, `fmhz` are defined below:

mode	clock	fmhz	description
'pcie'	'ext' (Gen3) or 'int' (Gen2)	(unset)	Mode 0, Serdes 0 or 1
'pcie&xpcs0' (1G)	'ext' (Gen3) or 'int' (Gen2)	'100'	Mode 1, Serdes 1 only (PFE0)
'pcie&xpcs1' (1G)	'ext' (Gen3) or 'int' (Gen2)	'100'	Mode 2, Serdes 0 or 1
'xpcs0&xpcs1' (1G)	'ext' or 'int'	'100' or '125'	Mode 3, SerDes 0 or 1
'xpcs0' (2.5G)	'ext' or 'int'	'125'	Mode 4, SerDes 1 only
'disabled'	N/A	N/A	All modules connected to that SerDes (PCIe, XPCS interfaces) are disabled

Table 7: `serdesi` options values

- `<xpcs_settings_i_j>` is :

  `speed=<10M|100M|1G|2G5>[,an=<0|1>]`

  `i` is the index of the SerDes module (0 or 1), `j` is the index of the XPCS interface (0 or 1).
  `an` represents the auto-negotiation on the Ethernet interfaces (GMAC or PFE). `an=0` translates in adding a `fixed-link` sub-node to the device tree node corresponding to the Ethernet interface connected to the XPCS being configured. If `an` is missing, it is assumed to be '1' (as for the case where the PFE_EMAC interface is output directly to an AQR PHY without involving the SJA1110).



XPCS0 is GMAC0 on SerDes0 and PFE0 on SerDes1.

XPCS1 is PFE2 on SerDes0 and PFE1 on SerDes1.

On EVB boards, SerDes0 is only routed to the PCIe slot (so GMAC0 and PFE2 can not be set as SGMII).

Options `xpcsi_j` are required only if the string `xpcsj` can be found in the value of the option `mode` for the corresponding SerDes*i* module.

Please refer to the "PCIe in U-Boot" for more detailed information on SerDes configuration.

7.2.2.1 Example configurations

SerDes modules can operate in several modes. Below are some configurations showing some of these modes. For the modes enabling multiple XPCS interfaces, the active one can be selected through variable `ethact`.



The boards have to support the configuration to test it.

Board Name	Configuration	Command

S32G274A RDB2	SerDes0 (Mode 0): PCIe Gen3 X2
	SerDes1 (Mode 2 1G):
	PCIe Gen3 X1 on lane0,
	PFE1 1G on lane1
	Switches: SW17 all ON, SW8 OFF
	Note: SJA1110 with 1G FW
	PFE1 out on port P2 (A or B)
		setenv hwconfig "serdes0:mode=pcie,clock=ext;
		pcie0:mode=rc;
		serdes1:mode=pcie&xpcs1,clock=ext,fmhz=100;
		pcie1:mode=rc;xpcs1_1:speed=1G"

S32G2 EVB	SerDes0 (Mode 0): PCIe Gen3 X2
	SerDes1 (Mode 3 1G):
	PFE0 1G on lane0,
	PFE1 1G on lane1
	Switches:
	EVB: SW12 OFF,ON,*,* SW13 ON,OFF,*,*
		setenv hwconfig "serdes0:mode=pcie,clock=ext;
		pcie0:mode=rc;
		serdes1:mode=xpcs0&xpcs1,clock=ext,fmhz=125;
		xpcs1_0:speed=1G;xpcs1_1:speed=1G"

S32G274A RDB2	SerDes0 (Mode 0): PCIe Gen3 X2 SerDes1 (Mode 4 2.5G): PFE0 2.5G on lane0	setenv hwconfig "serdes0:mode=pcie,clock=ext; pcie0:mode=rc serdes1:mode=xpcs0,clock=ext,fmhz=125;xpcs1_0:speed=2G5"

Table 8: SerDes configuration examples



For specific configuration you can check arch/arm/mach-s32/include/s32-cc/serdes_hwconfig.h in U-Boot.

For changes required in order to have the modes in the above table working in Linux, please see section Linux Ethernet Support, NOTE-9.

Starting with U-Boot 2022.04, the `pfeng_mode` configuration variable is no longer supported. The PFE EMAC link mode configuration in U-Boot is driven by the variable `hwconfig` and the bootloader device tree, as specified by the PFE device tree bindings. (see PFE Linux driver UM for details).

Please refer to the `S32G2 Reference Manual` for more detailed information on SerDes configuration.

7.2.3 SerDes configuration for SGMII in Linux

Linux SerDes driver reconfigures SerDes based on device tree. The device tree is automatically updated from U-Boot to correspond to the `hwconfig` variable. The SerDes driver is necessary for allowing suspend/resume of the SerDes.

Example of default node:

```
serdes0: serdes@40480000 {
        #phy-cells = <3>;
        compatible = "nxp,s32cc-serdes";
        clocks = <&clks S32CC_SCMI_CLK_SERDES_AXI>,
                 <&clks S32CC_SCMI_CLK_SERDES_AUX>,
```

```
                <&clks S32CC_SCMI_CLK_SERDES_APB>,
                <&clks S32CC_SCMI_CLK_SERDES_REF>;
        clock-names = "axi", "aux", "apb", "ref";
        resets = <&reset S32CC_SCMI_RST_SERDES0>,
                 <&reset S32CC_SCMI_RST_PCIE0>;
        reset-names = "serdes", "pcie";
        fsl,sys-mode = <XPCSX2_MODE>;
        reg = <0x0 0x40480000 0x0 0x108>,
              <0x0 0x40483008 0x0 0x10>,
              <0x0 0x40482000 0x0 0x800>,
              <0x0 0x40482800 0x0 0x800>;
        reg-names = "ss_pcie", "pcie_phy", "xpcs0", "xpcs1";
};
```

Following values are updated based on `hwconfig` variable in U-Boot:

- SerDes mode (fsl,sys-mode allowed options are PCIE_GEN3X2_MODE, PCIE_XPCS0_MODE, PCIE_XPCS1_MODE, XPCSX2_MODE)
- Clock external/internal frequency (clocks, clock-names)

By default internal reference clock is used. To use external clock, add its phandle to 'clocks' and add "ext" to corresponding index in 'clock-names'.

The SerDes driver allows multiple configuration with SGMII and supports SGMII auto-negotiation. When SGMII AN is enabled, PHY provides speed/link status over SGMII link to MAC. The driver supports only the MAC role in SGMII AN process.

Configure the link management properties for the 'ethernet' device tree node, depending on the PHY SGMII AN support and MDIO connection, according to the following table:

PHY SGMII AN	MDIO to PHY	device tree properties	Information
disabled	connected	phy-handle = <...>;	SGMII link is statically configured based on PHY auto-negotiation result(clause 28).
enabled	connected	phy-handle = <...>; managed = "in-band-status";	SGMII link is configured based on SGMII AN result. Some statuses from the PHY are also used.
enabled	not connected	managed = "in-band-status";	SGMII link is configured based on SGMII AN result.

disabled	not connected	fixed-link {
		speed = ...;
		}
			SGMII link is statically configured based on fixed-link properties.



With incorrect configuration the SGMII link between PHY and SoC SerDes may not work.

7.2.4 Ethernet support on the S32G2 EVB board(s)

As a prerequisite for the EVB board revision 'B': erratum E-S32G_EVB-0001 must be applied.
The evaluation board contains two parts:

1. `S32G-PROCEVB-S` with SoC and two Ethernet sockets:
   - 2500Base-T with AQR107 PHY connected on SGMII to the PFE_EMAC_0;
   - 1000Base-T with KSZ9031 PHY connected on RGMII to the GMAC.
2. `S32GRV-VNP-PLATEEVB` with Ethernet switch SJA1105Q and the following sockets:

- 1000Base-T with KSZ9031 PHY connected to the RGMII PFE_EMAC_1/GMAC, but disabled as the S32G-VNP-PROC-S RGMII is used by default;
- 2x1000Base-T with KSZ9031 PHYs connected to the SJA1105Q switch;
- 2x100BASE-T1 with one TJA1102 automotive PHY connected to the SJA1105Q switch;
- 2500Base-T with AQR107/113 PHY connected on SGMII to the PFE_EMAC_1.

For more detailed information, please refer to the “S32G-VNP-EVB – User Guide” .

7.2.5 Ethernet support on the S32G274A RDB2 board

The evaluation board integrates all necessary components in one chassis, including Automotive TSN/AVB SJA1110A SGMII switch. The sockets are:

- 1x1000Base-T with AQR107 PHY connected to PFE_EMAC_1 (Disabled by default);
- 2x1000Base-T with AR8035 PHYs connected to the SJA1110A switch;
- 1000Base-T with AR8035 PHY connected to the RGMII PFE_EMAC_2;
- 1000Base-T with KSZ9031 PHY connected to GMAC;
- 100Base-TX directly connected to the SJA1110 switch;
- 6x100Base-T1 directly connected to the SJA1110 switch;

For more detailed information, please refer to the `S32G-VNP-RDB2/3 Reference Manual`.

7.3 Ethernet Software support on S32G2

7.3.1 U-Boot Ethernet support

On the S32G2 EVB and S32G274A RDB2 targets we have both controllers enabled by default. It can be visible on U-Boot start-up or later by the `dm` command:

- U-Boot 2020.04 (or older):

```
=> dm uclass
uclass 0: root
- * root_driver @ 9fb2b050, seq 0, (req -1)

uclass 11: simple_bus
uclass 21: eth
- * eth_eqos @ 9fb2b640, seq 0, (req -1)
- * eth_pfeng @ 9fb2b7e0, seq 1, (req -1)
```

- U-Boot 2022.04 onward:

```
=> dm uclass
uclass 36: ethernet
0   * ethernet@4033c000 @ ffc036b0, seq 0
1   * pfe0 @ ffc08430, seq 1
2   * pfe1 @ ffc08570, seq 2
3   * pfe2 @ ffc086b0, seq 3
```

In U-Boot, when multiple Ethernet ports are available, the active port is selected via the `ethact` environment variable. To change the active Ethernet interface use: `setenv ethact <eth-device-name>`, where the Ethernet device name can be extracted with the `dm` command.
E.g.:

```
=> setenv ethact pfe0
```

7.3.1.1 GMAC U-Boot driver

The *GMAC U-Boot driver* configuration is driven by the GMAC Ethernet and MDIO AT-F device tree nodes located in fdts/s32cc.dtsi, i.e.:

```
gmac0: ethernet@4033c000 {
        compatible = "nxp,s32cc-dwmac";
        reg = <0x0 0x4033c000 0x0 0x2000>, /* gmac IP */
              <0x0 0x4007c004 0x0 0x4>;    /* S32 CTRL_STS reg */
        interrupt-parent = <&gic>;
        interrupts = <GIC_SPI 57 IRQ_TYPE_LEVEL_HIGH>;
        interrupt-names = "macirq";
        tx-fifo-depth = <20480>;
        rx-fifo-depth = <20480>;
        status = "disabled";
        phy-names = "gmac_xpcs";
        phys = <&serdes0 PHY_TYPE_XPCS 0 0>;
        dma-coherent;
        clocks = <&clks S32CC_SCMI_CLK_GMAC0_AXI>,
                 <&clks S32CC_SCMI_CLK_GMAC0_AXI>,
                 <&clks S32CC_SCMI_CLK_GMAC0_TX_SGMII>,
                 <&clks S32CC_SCMI_CLK_GMAC0_TX_RGMII>,
                 <&clks S32CC_SCMI_CLK_GMAC0_TX_RMII>,
                 <&clks S32CC_SCMI_CLK_GMAC0_TX_MII>,
                 <&clks S32CC_SCMI_CLK_GMAC0_RX_SGMII>,
                 <&clks S32CC_SCMI_CLK_GMAC0_RX_RGMII>,
                 <&clks S32CC_SCMI_CLK_GMAC0_RX_RMII>,
                 <&clks S32CC_SCMI_CLK_GMAC0_RX_MII>,
                 <&clks S32CC_SCMI_CLK_GMAC0_TS>;
        clock-names = "stmmaceth", "pclk",
                      "tx_sgmii", "tx_rgmii",
                      "tx_rmii", "tx_mii",
                      "rx_sgmii", "rx_rgmii",
                      "rx_rmii", "rx_mii",
                      "ptp_ref";
        gmac0_mdio: mdio0 {
                compatible = "snps,dwmac-mdio";
                #address-cells = <1>;
                #size-cells = <0>;
        };
};
```

and in fdts/s32gxxxa-evb.dtsi, i.e.:

```
&gmac0 {
        status = "okay";
        phy-mode = "rgmii";
        phy-handle = <&mdio_c_phy4>;
};

&gmac0_mdio {
        #address-cells = <1>;
        #size-cells = <0>;
        /* ARQ107 on S32RGV-VNP-PLAT */
        mdio_c_phy1: ethernet-phy@1 {
                compatible = "ethernet-phy-ieee802.3-c45";
                reg = <1>;
```

```
        };
        /* KSZ9031RNX on S32G-VNP-PROC */
        mdio_c_phy4: ethernet-phy@4 {
                reg = <4>;
                max-speed = <1000>;
        };
        /* KSZ9031RNX on S32RGV-VNP-PLAT */
        mdio_c_phy5: ethernet-phy@5 {
                status = "disabled"; /* blocked by USB by default */
                reg = <5>;
                max-speed = <1000>;
        };
};
```

The driver is supporting PHY link autoconfiguration over the MDIO bus.

7.3.1.2 PFE U-Boot 2020.04 (or older) driver

The *PFE U-Boot 2020.04 driver* configuration is managed by the following the U-Boot environment variables and commands:

- `pfeng_mode <state>,<port0>,<port1>,<port2>`, where
  - `<state>` - Enable or disable the pfeng driver,
  - `<portN>` - The PFE_EMAC interface type `rgmii`, `sgmii` or `none`;

  U-Boot does not check the configuration validity as it is mostly board specific. The `sgmii` port type requires also corresponding `hwconfig` variable setup. The variable is read on U-Boot startup, in driver binding phase only. Any changes require U-Boot restart.
- `pfengemac` - environment variable that specifies the active PFE port number;

  The active PFE port number corresponds to the PFE_EMAC interface ID that is currently used for traffic (ping, tftpboot). This variable is checked by the PFE driver before every network operation so it is perfectly fine to change the PFE_EMAC port at runtime, with the following command: `setenv pfengemac <emac-number>`.
- `pfeaddr`, `pfe1addr` and `pfe2addr` - the Ethernet MAC address environment variables corresponding to each PFE_EMAC port;
- `xpcs` - command that configures the SerDes XPCS module directly;

  The usage is the following:

```
xpcs list                                      - List all registered XPCS modules
xpcs <instance_id> transit <1000M|2500M>       - Change serdes mode
xpcs <instance_id> ss <10M|100M|1000M|2500M> - Change speed and serdes mode when required
xpcs <instance_id> an <enable|disable>         - Auto-negotiation control
xpcs <instance_id> an_auto <enable|disable>  - Auto-negotiation control with automatic speed change
xpcs <instance_id> lo <enable|disable>         - PMA loopback enable/disable
xpcs <instance_id> dump                        - Dump XPCS indirect registers
```

  `instance_id` is obtained using command `xpcs list`. Only XPCS modules which are probed without issues (e.g. clocking mismatch or corresponding lane occupied by PCIe) will be displayed.
- `pfeng` - command that manages and queries the PFE controller's state;

  The usage is the following:

```
pfeng info                                  - important hw info
pfeng [disable|enable]                      - disable/enable full PFE/EMACs subsystem
pfeng stop                                  - stop the driver but do not disable PFE/EMACs
pfeng emacs [<intf0>,<intf1>,<intf2>]       - read or set EMAC0-2 interface mode
pfeng emacs reapply-clocks                  - reapply clock setting for all EMACs
pfeng reg <offset>                          - read register
```

The entire PFE subsystem can be disabled by the `pfeng disable` command.

Limitation: The U-Boot PFE driver does not support speed autoconfiguration. The link speed expected by the driver is 1000 Mbit/s or optionally 2500 Mbit/s for PFE_EMAC_0.

S32G2 EVB default port mapping in U-Boot

SoC port	**interface**	**board port**	**note**
PFE_EMAC_0	SGMII(1G/2.5G)	J102	NOTE-1, NOTE-1B for SGMII config
PFE_EMAC_1	SGMII(1G) on PLAT board	J102 on PLAT board	–
PFE_EMAC_2	RGMII on the SJA1105Q switch on PLAT board	J108, J109, J111, J112	NOTE-2 for SJA1105 firmware load
GMAC	RGMII	J108	NOTE-3 for RGMII
hwconfig	'serdes0:...;serdes1:mode=xpcs0,clock=ext,fmhz=125;xpcs1_0:speed=2G5'		

- NOTE-1: The PFE_EMAC_0 can use the SGMII interface and connect to the AQR107/113 PHY on the PROCEVB board. Aquantia PHY firmware is required on external flash connected to the PHY for this setup to work. If S32G EVB board is populated with sticker SCH-32170 REV B2 the Aquantia PHY firmware is populated.
- NOTE-1B: The PFE_EMAC_0 interface speed depends on PFE_EMAC_1 setting. In case of using SGMII also on PFE_EMAC_1, the speed can be 1G max.
- NOTE-2: The speed for the SJA1105 automotive switch needs to be set after every reboot:

```
=> sja speed -,-,-,1G,1G 5:0
```

- NOTE-3: GMAC can share RGMII port with PFE_EMAC_1. It is possible to instantly switch RGMII to the GMAC by explicitly disabling and enabling the GMAC driver, which also reapplies pin/clocks settings:

```
=> s32ccgmac disable
=> s32ccgmac enable
=> setenv ethact eth_eqos
=> ping
```

To assign RGMII back to PFE, the U-Boot helper command for reapplying pins/clocks can be used:

```
=> pfeng emacs reapply-clocks
```

S32G274A RDB2 default port mapping in U-Boot

SoC port	**interface**	**board port**	**note**
PFE_EMAC_0	SGMII(2.5G) to switch SJA1110A	P2, P4	–
PFE_EMAC_1	none	P5	NOTE-5
PFE_EMAC_2	RGMII	P3 top	–
GMAC	RGMII	P3 bottom	–
hwconfig	'serdes0:...;serdes1:mode=xpcs0,clock=ext,fmhz=125;xpcs1_0:speed=2G5'		

- NOTE-5: The connection using SGMII to AQR107 is currently not supported.

U-Boot MDIO support:

Ethernet controller	MDIO bus controller	MDIO PHY support
GMAC	clause 22 / clause 45	yes
PFE	clause 22 / clause 45	no (U-Boot 2020.04 or older)

7.3.1.3 U-Boot 2022.04 PFE driver

Starting with *U-Boot 2022.04*, the `pfeng_mode` configuration variable is no longer supported. The PFE EMAC link mode configuration in U-Boot is driven by the bootloader device tree instead, as specified by the PFE device tree bindings. (see the PFE Linux driver UM for details)

However, the `pfeng` command can still be used to display PFE driver info and hardware statistics:

```
pfeng - NXP S32G PFE accelerator status

Usage:
pfeng
       - Print various H/W statistics
```

On *U-Boot 2022.04*, the PFE base driver also provides a DM MDIO driver that creates MDIO device instances for every `"nxp,s32g-pfe-mdio"` compatible device tree node. E.g.:

```
=> dm tree
[...]
 misc          5  [ + ]   pfeng                |   |-- pfeng-base
 mdio          0  [ + ]   pfeng_mdio           |   |   |-- pfeng-mdio-2
 ethernet      1  [ + ]   pfeng_netif          |   |   |-- pfe0
 ethernet      2  [ + ]   pfeng_netif          |   |   |-- pfe1
 ethernet      3  [ + ]   pfeng_netif          |   |   `-- pfe2
```



The PFE U-Boot driver requires the PFE firmware to be located in the SD card boot partition under the name *s32g_pfe_class.fw*.

7.3.1.4 U-Boot 2022.04 PFE Slave driver

The U-Boot PFE Slave driver is based on the Master–Slave concept for running multiple PFE driver instances on one SoC. The PFE Master driver owns and controls the PFE block. It allows sharing the PFE provided connectivity using a dedicated host interface (HIF) to more PFE Slave drivers at the same time. The Slave driver cannot use its dedicated HIF until the Master is ready and functional. For this reason, a synchronization mechanism is built into the PFE drivers.

The U-Boot PFE Slave driver performs only the necessary PFE block configurations. It is limited to initialization of its dedicated HIF, to make basic communication with the PFE Master driver possible. The rest is up to the PFE Master driver. It must enable the desired interfaces for the PFE Slave driver and configure PFE for a required packet distribution over PFE.

For more information about the PFE Master configuration, see the `PFE Driver User Manual` and `PFE MCAL Driver User Manual`.

Check the PFE Health Monitoring User Manual for elimination of faulty scenarios on PFE HW and SW.

Health Monitor is not supported in PFE U-Boot Slave driver and can be used only on Master side.

7.3.1.4.1 PFE Slave driver configuration

The U-Boot PFE Slave configuration is driven by TF-A device tree node pfe_slave located in fdts/s32g-pfe-slave.dtsi.

For more details, see PFE device tree bindings in the PFE Driver User Manual.

7.3.1.4.2 Build PFE Slave driver

See first Setting up and building U-Boot bootloader chapter for more details.

The example of the PFE Slave driver DT configuration exists for the RDB3 board only (fdts/s32g399a-rdb3-pfems.dts).

In order to build the U-Boot PFE Slave driver follow steps:

```
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- s32g399ardb3_defconfig
```

In menuconfig, change the following:

```
CONFIG_NXP_PFENG=y
# CONFIG_NXP_PFENG_STANDALONE is not set
CONFIG_NXP_PFENG_SLAVE=y
CONFIG_NXP_PFENG_SLAVE_MANAGE_COHERENCY=y
```

For the coherency management setting, see prerequisites in the PFE Driver User Manual, chapter Slave coherency settings.

Run the U-Boot build

```
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu-
```

Build the Arm Trusted Firmware FIP image. See Building the Arm Trusted Firmware for more details how to build and deploy the TF-A.

```
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- \
ARCH=aarch64 PLAT=s32g399ardb3 DTB_FILE_NAME=s32g399a-rdb3-pfems.dtb BL33=<path-to-u-boot-nodtb.bin>
```

Finally deploy created fip.s32 image.

7.3.1.4.3 PFE Slave driver usage

```
=> pfeng help
pfeng - NXP S32G Slave PFE accelerator

Usage:
pfeng
        - Print various H/W statistics
pfeng stat
        - Print various H/W statistics
pfeng reset
        - Force PFE HIF channel reset
          HIF channel will be relinquished in the current u-boot session
```

The `pfeng` PFE Slave driver U-Boot command is available when the PFE Master driver is up and running, and when the PFE Slave driver is activated. The activation is triggered only before a communication over a PFE port is done. Otherwise, the `PFE IP is not ready` information is being displayed and no action is taken.

The `pfeng` command can be used to display PFE driver info, or to force a PFE HIF channel reset. The reset command is intended for validation purposes only.

7.3.2 Ethernet fixup procedure

When U-Boot is about to start the OS (e.g. Linux, QNX, etc.), it allows to update some properties in the provided device tree file. It is used to synchronize MAC addresses on all ports, to be in sync between U-Boot and Linux OS.

The second usage is for fixing PHY addresses, if particular hardware is able to provide revision information. This is valid for RDB2 and RDB3 boards, but not for EVB/PLAT boards.

The properties **for PFE and GMAC ethernet nodes** which are managed by fixup:

1. local-mac-address (both PFE and GMAC)

   The MAC address is set to the value defined in the following variables:

U-Boot environment variable	Ethernet controller	Corresponding ethernet node
eth1addr	PFE	pfe0
eth2addr	PFE	pfe1
eth3addr	PFE	pfe2
eth4addr	GMAC	eth0

2. phy-mode is set to SGMII for nodes corresponding to XPCS interfaces configured by the variable `hwconfig`.
3. fixed-link speed is set to nodes configured in SGMII mode connected to the SJA1110 switch
   The fixed link speed is updated based on the `hwconfig`, attribute `speed` for the appropriate XPCS interface.
4. Aquantia PHY address on S32G274A RDB2
   The Aquantia PHY may have different address on different S32G274A RDB2 revisions. This is handled automatically by the fixup via ADC measurement of resistor coding. Correct PHY address is stored also in U-Boot variable `pfe1_phy_addr`.



The NXP U-Boot has an incorporated mechanism to modify a loaded DTB before passing it for processing to the Linux OS. These DTB updates are driven by the `fdt_override` which, if not empty, is invoked during final stage of loading Linux OS.

The U-boot 2022.04 driver supports MAC address fixup via the standard `eth<X>addr` environment variables, where `<X>` is the PFE Ethernet device sequence number. Currently, the U-Boot 2022.04 PFE driver does not provide support for additional fixups, so the user is advised to opt for the `fdt_override` mechanism as an alternative.

7.3.3 Linux Ethernet support

By default, both drivers, GMAC and PFE, are enabled for all targets. The GMAC driver is called `dwmac-s32cc` and is kernel built-in. The PFE driver is called `pfeng` and is loaded on system boot.

The PFE Linux driver is an external module and is located in the git repository:
https://github.com/nxp-auto-linux/pfeng.
While the `PFE Driver User Manual` is available on Flexera.

The drivers are using device tree configurations, which are described in the following sources:

- GMAC:
  - arch/arm64/boot/dts/freescale/s32cc.dtsi
  - arch/arm64/boot/dts/freescale/s32gxxxa-evb.dtsi
- PFE:
  - arch/arm64/boot/dts/freescale/s32g-pfe.dtsi
  - arch/arm64/boot/dts/freescale/s32gxxxa-evb.dtsi
  - arch/arm64/boot/dts/freescale/s32gxxxa-rdb.dtsi

The general note: the default port mapping is the state which is valid for first start, when nothing is changed inside U-Boot environment. You can see the mapping in next two figures:

S32G2 EVB default port mapping in Linux

SoC port	interface	board port	Linux name	note
PFE_EMAC_0	SGMII(1G/2.5G)	J102	`pfe0`	NOTE-6, NOTE-6B for SGMII
PFE_EMAC_1	SGMII(1G) on PLAT board	J102 on PLAT board	`pfe1`	NOTE-7
PFE_EMAC_2	RGMII to the switch SJA1105Q on PLAT board	J108, J109, J111, J112	`pfe2`	NOTE-8
GMAC	RGMII	J108	`eth0`	–

- NOTE-6: PFE_EMAC_0 uses by default the SGMII interface and connects to the AQR107/113 PHY on the PROCEVB board. Aquantia PHY firmware is required on external flash connected to the PHY for this setup to work. For EVB3 boards, and for EVB boards with revision newer than SCH-32170 REV B2, the Aquantia PHY firmware is populated.

- NOTE-6B: The PFE_EMAC_0 interface speed depends on PFE_EMAC_1 setting. In case of using SGMII also on PFE_EMAC_1, the speed can be 1G max.
- NOTE-7: The PFE_EMAC_1 is using SGMII interface connected to the AQR107/113 on PLAT board. Unfortunately the PLAT board hw rev. CX1 and newer has changed the AQR PHY address from 0x1 to 0x3. This change cannot be covered by U-Boot ethernet fixup and requires adding the following script to the U-Boot variable `fdt_override`:

```
=> setenv fdt_override 'fdt addr ${fdt_addr}; fdt resize;
                        fdt set "/soc/ethernet/mdio0/ethernet-phy@1/" reg <0x3>;
                        ${fdt_override}'
=> saveenv; reset
```

- NOTE-8: The speed for the SJA1105 automotive switch needs to be set after every reboot in U-Boot:

```
=> sja speed -,-,-,1G,1G 5:0
```

S32G274A RDB2 default port mapping in Linux

SoC port	interface	board port	Linux name	note
PFE_EMAC_0	SGMII(2.5G/1G) to switch SJA1110A	P2, P4	`pfe0`	NOTE-8 for SGMII(1G)
PFE_EMAC_1	none/SGMII(1G)	P5	`pfe1`	–
PFE_EMAC_2	RGMII	P3 top	`pfe2`	–
GMAC	RGMII	P3 bottom	`eth0`	–

- NOTE-9: The device tree nodes for PFE_EMACs (0,1,2) and GMAC hold a configuration that matches:
  - the default configuration set on the board by dip-switches
  - the default `hwconfig` configuration set in U-Boot: Serdes0: Mode 0 (PCIe X2 Gen3); SerDes1: Mode 4 (PFE_EMAC_0 SGMII (2.5G) connected to SJA1110 port 4); PFE_EMAC_1 disabled; PFE_EMAC_2: RGMII.
  - SJA1110 firmware image for 2.5Gbps connection speed
  - GMAC0: RGMII
- NOTE-10: The PFE_EMAC_0 interface speed depends on PFE_EMAC_1 setting. In case of using SGMII also on PFE_EMAC_1, the speed can be 1G max. But setting speed to 1G requires compatible SJA1110 firmware.

Ethernet port mapping customization in Linux

U-Boot has implemented a device tree fixup procedure which sets automatically the phy mode to SGMII and speed as specified in the variable `hwconfig` for the desired XPCS interfaces. Also, it disables the XPCS interfaces configured by default in SGMII mode which are not mentioned in `hwconfig`. If other changes are desired, they can be made through the U-Boot variable `fdt_override`:

```
=> setenv fdt_override 'fdt addr ${fdt_addr}; fdt resize;
                        fdt set <node> <property> <value>;
                        fdt rm <node> <property>;
                        ${fdt_override}'
```

If any of the changes conflict with the fixups, then the fixups can be disabled for a particular XPCS interface by simply removing the corresponding alias from the device tree, using variable `fdt_override`. The aliases are named `ethernet0` for GMAC0 and `ethernet1,2,3` for PFE_EMAC_0,1,2:

```
=> setenv fdt_override 'fdt addr ${fdt_addr}; fdt resize;
                        fdt rm /aliases ethernetX;
                        ${fdt_override}'
```

where `X` is one of the values 0-3.

Linux MDIO support

Ethernet controller	MDIO bus controller	MDIO PHY support
GMAC	clause 22 / clause 45	yes
PFE	clause 22 / clause 45	yes

7.3.4 Build by Auto Linux BSP

To get the PFE standalone driver integrated into the NXP Auto Linux BSP, the following two lines must be added to the conf/local.conf file:

```
DISTRO_FEATURES:append = " pfe"
NXP_FIRMWARE_LOCAL_DIR = "/path/to/firmware/binaries/folder"
```

where `/path/to/firmware/binaries/folder` is the local path to the local s32g_pfe_class.fw fw file and optionally s32g_pfe_util.fw.

To build PFE multi instance driver and populate it, the following should be appended to DISTRO_FEATURES:

```
DISTRO_FEATURES:append = " pfe pfe-slave"
```

When `pfe` feature is added without `pfe-slave` feature, the populated driver will be standalone. If both features `pfe` and `pfe-slave` are present, PFE drivers are built as multi instance. Feature `pfe-slave` can be used without `pfe` feature, when the PFE master driver runs in different OS.

7.4 Setting up DHCP client in Linux

Prerequisites for using DHCP client:

- A DHCP server has to be available and correctly configured in the same network as the board.
  - Manual execution of DHCP client
    Just run the below command to obtain dynamically the IP on eth0:

    ```
    udhcpc -i eth0
    ```

7.5 SSH support in Linux

The SSH server is started by default at system startup.

7.6 Setting up SJA1105 switch

In U-Boot: The SJA1105 automotive switch is probed automatically on boot.
For custom setups where this does not happen, it can be probed manually from U-Boot prompt:

```
=> sja probe 5:0
```

However, the speed needs to be set manually after every reboot:

```
=> sja speed -,-,-,1G,1G 5:0
```

In Linux: There are two drivers for the SJA1105 switch. One driver is out-of-tree and is just a simple configuration loader. The second driver uses the DSA framework and is part of the Linux kernel.



For S32G2 EVB board(s), the DSA driver is probed by default.

7.6.1 SJA1105 Out-of-tree Linux driver

If used, the out-of-tree Linux kernel module is automatically inserted during boot time. Below is a kernel boot log example:

```
[    3.939877] sja1105pqrs spi1.0: Loading SJA1105P SPI driver
[    3.945225] sja1105pqrs spi1.0: Detected Device ID ae00030e ()
[    3.951026] sja1105pqrs spi1.0: Detected device id is invalid: 00000000
[    3.969679] sja1105pqrs spi1.0: 1 switch initialized successfully!
```

7.6.2 SJA1105 Linux DSA driver

The SJA1105 DSA driver is built into the Linux Kernel image. It presents useful switch ports as network interfaces and provides support for other features of the switch like VLAN and PTP.

Example of network interfaces when the SJA1105 DSA driver is loaded:

```
root@s32g274aevb:~# ip addr show
...
7: pfe2: <BROADCAST,MULTICAST> mtu 1504 qdisc noop state DOWN group default qlen 1000
    link/ether 00:01:be:be:ef:33 brd ff:ff:ff:ff:ff:ff
8: enet_p1@pfe2: <BROADCAST,MULTICAST,M-DOWN> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether 00:01:be:be:ef:33 brd ff:ff:ff:ff:ff:ff
9: enet_p3@pfe2: <BROADCAST,MULTICAST,M-DOWN> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether 00:01:be:be:ef:33 brd ff:ff:ff:ff:ff:ff
10: enet_p4@pfe2: <BROADCAST,MULTICAST,M-DOWN> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether 00:01:be:be:ef:33 brd ff:ff:ff:ff:ff:ff
```

In order to use the switch interfaces, the DSA master port (`pfe2` interface in case of the S32G2 EVB board(s)) must be up, otherwise the link of these interfaces cannot be set up.



For the S32G2 EVB board(s): In order to use the DSA master port as SGMII (`pfe2` in this case), DSA master mode must be disabled.

This can only be done by disabling the SJA1105 switch, i.e. by disabling the SPI node used for configuring it (`spi5`). To do this from U-Boot, please see section Linux Ethernet Support, NOTE-8B.
SJA1105 is not being used in this scenario, since PFE_EMAC_2 - `pfe2` in SGMII mode is routed directly to the AQR107 PHY on the PLATEVB board.

Example output with the DSA master down:

```
root@s32g274aevb:~# ip link set dev enet_p4 up
RTNETLINK answers: Network is down
```

Example output with the DSA master up:

```
root@s32g274aevb:~# ip link set dev pfe2 up
root@s32g274aevb:~# [  828.805756] 002: pfeng 46000000.pfe: HIF2 started
[  828.806018] 002: pfeng 46000000.pfe pfe2: configuring for fixed/rgmii link mode
[  828.806037] 002: pfeng 46000000.pfe pfe2: Set TX clock to 125000000Hz
[  828.806262] 002: pfeng 46000000.pfe pfe2: Set TX clock to 125000000Hz
[  828.806279] 002: pfeng 46000000.pfe pfe2: Link is Up - 1Gbps/Full - flow control off
[  828.806494] 002: IPv6: ADDRCONF(NETDEV_CHANGE): pfe2: link becomes ready
root@s32g274aevb:~# ip link set dev enet_p4 up
[  845.913867] 002: sja1105 spi5.0 enet_p4: configuring for phy/rgmii-id link mode
```

In the default use case, when there is no VLAN Filtering enabled, no bridge is set up and so on, the IP addresses should be assigned to the switch interface.

```
root@s32g274aevb:~# ip link set dev enet_p4 up
[   75.109811] 002: sja1105 spi5.0 enet_p4: configuring for phy/rgmii-id link mode
[   80.218319] 002: sja1105 spi5.0 enet_p4: Link is Up - 1Gbps/Full - flow control off
[   80.218358] 002: IPv6: ADDRCONF(NETDEV_CHANGE): enet_p4: link becomes ready
root@s32g274aevb:~# ip addr add 10.42.0.101/24 dev enet_p4
root@s32g274aevb:~# ping 10.42.0.1
PING 10.42.0.1 (10.42.0.1) 56(84) bytes of data.
64 bytes from 10.42.0.1: icmp_seq=1 ttl=64 time=0.643 ms
64 bytes from 10.42.0.1: icmp_seq=2 ttl=64 time=0.315 ms
^C
--- 10.42.0.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1027ms
rtt min/avg/max/mdev = 0.315/0.479/0.643/0.164 ms
```

If the driver of the DSA master is compiled as a module and that module is removed at runtime and inserted again, the DSA slave needs to be bound manually. Example of binding sja1105 on s32g274aevb:

```
[root@s32g274aevb ~]# echo spi5.0 > /sys/bus/spi/drivers/sja1105/bind
[ 1342.167204] sja1105 spi5.0: Probed switch chip: SJA1105Q
[ 1342.185861] sja1105 spi5.0: configuring for fixed/rgmii-id link mode
[ 1342.187121] sja1105 spi5.0: Link is Up - 1Gbps/Full - flow control off
[ 1342.247496] sja1105 spi5.0 enet_p1 (uninitialized): PHY [PFEng Ethernet MDIO.2:06]
        driver [NXP TJA1102 Port 0] (irq=POLL)
[ 1342.288843] sja1105 spi5.0 enet_p1: configuring for phy/mii link mode
[ 1342.311675] sja1105 spi5.0 enet_p2 (uninitialized): PHY [PFEng Ethernet MDIO.2:07]
        driver [NXP TJA1102 Port 1] (irq=POLL)
[ 1342.353331] sja1105 spi5.0 enet_p2: configuring for phy/mii link mode
[ 1342.426419] sja1105 spi5.0 enet_p3 (uninitialized): PHY [PFEng Ethernet MDIO.2:02]
        driver [Micrel KSZ9031 Gigabit PHY] (irq=POLL)
[ 1342.471707] sja1105 spi5.0 enet_p3: configuring for phy/rgmii-id link mode
[ 1342.538420] sja1105 spi5.0 enet_p4 (uninitialized): PHY [PFEng Ethernet MDIO.2:03]
        driver [Micrel KSZ9031 Gigabit PHY] (irq=POLL)
[ 1342.539755] DSA: tree 0 setup
[ 1342.583596] sja1105 spi5.0 enet_p4: configuring for phy/rgmii-id link mode
```

For more information see:

- https://www.kernel.org/doc/html/latest/networking/dsa/dsa.html
- https://www.kernel.org/doc/html/latest/networking/dsa/sja1105.html

7.7 Setting up SJA1110 switch

7.7.1 Building firmware image for SJA1110 switch

Instructions for building the firmware image for SJA1110 can be found in: `S32G-VNP-RDB2 Reference Design Ethernet Enablement Guide`, chapter 4. SJA1110 Port4 PHY Speed Configuration.

See the `References` section of this manual.

7.7.2 Setting up SJA1110 switch over SPI

The Linux kernel module is automatically inserted at boot time on S32G274A RDB2. In order for the driver to update the firmware of the switch and/or the uC, please provide the path to the binaries on the host machine in `conf/local.conf` before the yocto build.

```
SJA1110_SWITCH_FW = <path_to_sja_switch_binaries>
SJA1110_UC_FW = <path_to_sja_uc_binaries>
```

Where path_to_sja_switch_binaries is `.../sja1110_switch.bin` and path_to_sja_uc_binaries is `.../sja1110_uc.bin`.



The user can also simply copy the binaries in rootfs at the following path: `/lib/firmware`.

In order for the firmware to be updated at boot time, SW12 must be set on the 1-1 position.

7.7.3 Enabling SJA1110 DSA driver

The SJA1110 DSA driver node is present in the device tree of the RDB board, having port 2 and port 3 configured with a fixed link at 1Gbps. As PHY management for these ports and port speed adjustment can be done only from SJA1110 firmware, the SJA1110 DSA driver is disabled by default.

To enable the driver, the boolean variable `sja1110_dsa` must be set in the U-Boot environment as in the following example. If the variable is set to `yes` or `true`, the driver will be enabled, if the variable is missing or is set to something else, the driver will be disabled.

U-Boot `setenv` example:

```
setenv sja1110_dsa yes
```

In order to use the SJA1110 DSA driver, SW12 must be set on the 1-1 position. Otherwise, the driver probe will fail.

8 PTP Information

8.1 Precision Time Protocol - IEEE 1588

The Precision Time Protocol (PTP) is a high-precision time synchronization protocol used to synchronize clocks throughout a computer network. PFE and GMAC interfaces have support for hardware timestamping. To check the support for hardware timestamping, *ethtool* can be used. The *linuxptp* tools are installed by default in *fsl-image-auto*. Depending of the *ptp4l* configuration parameters, the interfaces can act as a master clock or as a slave clock.

Example for checking the timestamping support:

```
root@s32g274aevb:~# ethtool -T eth0
Time stamping parameters for eth0:
Capabilities:
        hardware-transmit
        software-transmit
        hardware-receive
        software-receive
        software-system-clock
        hardware-raw-clock
PTP Hardware Clock: 0
Hardware Transmit Timestamp Modes:
        off
        on
Hardware Receive Filter Modes:
        none
        all
        ptpv1-l4-event
        ptpv1-l4-sync
        ptpv1-l4-delay-req
        ptpv2-l4-event
        ptpv2-l4-sync
        ptpv2-l4-delay-req
        ptpv2-event
        ptpv2-sync
        ptpv2-delay-req
```

Example of synchronization between a foreign master and GMAC using hardware timestamping(`-H`) at layer 2(`-2`):

```
root@s32g274aevb:~# ptp4l -2 -H -i eth0 -m -s
ptp4l[136.327]: selected /dev/ptp0 as PTP clock
ptp4l[136.380]: port 1: INITIALIZING to LISTENING on INIT_COMPLETE
ptp4l[136.380]: port 0: INITIALIZING to LISTENING on INIT_COMPLETE
ptp4l[143.364]: selected local clock 7684ef.fffe.c74e40 as best master
ptp4l[170.418]: port 1: new foreign master 7824af.fffe.d9dce0-1
ptp4l[174.418]: selected best master clock 7824af.fffe.d9dce0
ptp4l[174.419]: port 1: LISTENING to UNCALIBRATED on RS_SLAVE
ptp4l[176.418]: master offset -1627543141783358 s0 freq      +0 path delay     39752
ptp4l[177.418]: master offset -1627543141839918 s1 freq  -56556 path delay     39752
ptp4l[178.418]: master offset         70 s2 freq  -56486 path delay     39752
ptp4l[178.418]: port 1: UNCALIBRATED to SLAVE on MASTER_CLOCK_SELECTED
ptp4l[179.418]: master offset        150 s2 freq  -56385 path delay     39752
ptp4l[180.418]: master offset      11740 s2 freq  -44750 path delay     28112
ptp4l[181.418]: master offset      11557 s2 freq  -41411 path delay     16650
ptp4l[182.418]: master offset      -3775 s2 freq  -53276 path delay     16862
...
ptp4l[204.420]: master offset       -239 s2 freq  -56406 path delay     16501
```

```
ptp4l[205.420]: master offset       -209 s2 freq  -56448 path delay     16501
ptp4l[206.420]: master offset        -63 s2 freq  -56364 path delay     16465
ptp4l[207.420]: master offset        -73 s2 freq  -56393 path delay     16435
ptp4l[208.420]: master offset         -8 s2 freq  -56350 path delay     16435
...
ptp4l[232.423]: master offset         79 s2 freq  -56273 path delay     16413
ptp4l[233.423]: master offset        -40 s2 freq  -56368 path delay     1645
```



On EVB boards, where PFE2 or PFE0 (depending by the pinmuxing) is connected to SJA1105, the switch network interfaces should be used instead of the PFE interface connected to the switch.



When using the TAPRIO scheduler, there might be situations where a packet will not fit inside the programmed interval, but it will be sent anyway. This happens because the controller does not take into account the IPG of the packet. When this happens the IPG will be carried onto the next set of gates and overruns will happen. To mitigate this, one can set a dummy scheduler entry at the end of the scheduler with interval of one IPG (96ns for 1Gbps link speed). More info can be found on the tc-taprio man page

```
root@s32g274aevb:~# tc qdisc add dev eth0 root taprio \
                num_tc 5 \
                map 0 1 2 3 4 \
                queues 1@0 1@1 1@2 1@3 1@4 \
                base-time 0 \
                sched-entry S 10 50000 \
                sched-entry S 0f 450000 \
                sched-entry S 1f 96 \
                flags 0x2
```

8.2 802.1Qav

IEEE 802.1Qav Forwarding and Queuing Enhancements for Time-Sensitive Streams is a standard used by time-sensitive networks. It makes use of the Credit-Based Shaper (CBS) to manage audio/video traffic over Ethernet networks. CBS allocates credits to different streams, controlling their transmission rates based on priority for efficient and timely delivery.

CBS can be used in the Linux Kernel by enabling the following options:

```
CONFIG_NET_SCH_MQPRIO=y
CONFIG_NET_SCH_CBS=y
```

Configuration of this feature is done with the tc user space utility program.



The GMAC controller used on the S32G2 platform does not support offloading Qav traffic on the first queue because of hardware legacy reasons. When configuring the Qav traffic in hardware, it is strongly advised to not use the first queue.

Multiple parameters are included in the CBS configuration, such as `idleslope`, `sendslope`, `hicredit`, `locredit`. Those parameters can either be calculated manually or by using the calc-cbs-parameters.py script.

The following example limits the bandwidth on queue 1 to 67 Mbps on a 1G link connection.

```
root@s32g274aevb:~# tc qdisc add dev eth0 root handle 1: mqprio \
                        hw 0 num_tc 5 \
                        map 1 1 2 3 4 \
                        queues 1@0 1@1 1@2 1@3 1@4 \
root@s32g274aevb:~# tc qdisc replace dev eth0 parent 1:2 cbs  \
        idleslope 67000 sendslope -933000 hicredit 2147483647 locredit -2147483648 offload 1
```



The Qav traffic works as expected on the first queue if no offloading is done.

Testing for this feature can be done with iperf3 utility program by specifying the `-udp` and `-bandwidth` parameters.

9 PCIe Support

9.1 PCIe in U-Boot

S32G2 SoC has two SerDes modules (0 and 1), each of them with one PCIe controller.
A SerDes can be shared between PCIe and PFE or GMAC.
U-Boot enables PCIe by default on SerDes0 and PFE0 at 2.5Gbps on SerDes1; the default configuration on PCIe0 is Root Complex (RC) mode, using the external clock.
Boot configuration for both SerDes modules is set using the environment variable `hwconfig`.
The format for this variable is (all on one row):

```
=> setenv hwconfig "serdes0:<ss_settings_0>[;pcie0:<pci_settings_0>][;xpcs0_j:<xpcs_settings_0_j>];
                    serdes1:<ss_settings_1>[;pcie1:<pci_settings_1>][;xpcs1_j:<xpcs_settings_1_j>]"
```

Where:

- `<ss_settings_i>` is :

  `mode=<ss_mode_i>,clock=<clock_i>[,skip=<value_i>][,fmhz=<fmhz_i>]`

  `i` is the index of the SerDes module (0 or 1), and the valid values for SerDes options `mode`, `clock`, `fmhz` are defined below:

mode	clock	fmhz	description
'pcie'	'ext' (Gen3) or 'int' (Gen2)	(unset)	Mode 0, Serdes 0 or 1
'pcie&xpcs0' (1G)	'ext' (Gen3) or 'int' (Gen2)	'100'	Mode 1, Serdes 1 only (PFE0)
'pcie&xpcs1' (1G)	'ext' (Gen3) or 'int' (Gen2)	'100'	Mode 2, Serdes 0 or 1
'xpcs0&xpcs1' (1G)	'ext' or 'int'	'100' or '125'	Mode 3, SerDes 0 or 1
'xpcs0' (2.5G)	'ext' or 'int'	'125'	Mode 4, SerDes 1 only
'disabled'	N/A	N/A	All modules connected to that SerDes (PCIe, XPCS interfaces) are disabled

Table 9: `serdesi` options values

SerDes option	values	description
clock	'ext' 'int'	External clock (PCIE*i*_CLK_N/P is the reference clock for the PCIe PHY PLL). Can be used for all PCIe modes (Gen1 to Gen3). Internal clock (SERDES_REF_CLK is the reference clock for the PCIe PHY PLL). Can be used only for Gen1 or Gen2 PCIe modes.
fmhz	'100' or '125'	Clock frequency used for SGMII. Not required for PCIe only modes. For 'clock=ext' it should match the rate set for the board by jumper or switch.
skip	'boot' or 'kernel' or 'all'	If set, skips configuration of the corresponding SerDes and PCIe controller in U-Boot ('boot'), Linux ('kernel') or both ('all'). When enabled, settings from variable 'hwconfig' are ignored for that particular SerDes and the default configuration from device tree is applied.

Table 10: `serdesi` options descriptions

The `skip` is optional, it allows skipping SerDes and PCIe controller configuration in U-Boot, thus allowing full configuration to be performed from Linux directly. Some PCIe devices (e.g. PCIe switches) cannot be configured

twice, as the PHY does not link and remains in POLL_ACTIVE state. `skip` is enabled if `<value_i>` is "1".



SerDes modes with one lane PCIe, one lane SGMII, i.e. `pcie&xpcs0` or `pcie&xpcs1` are also called `combo modes`. When using combo modes, the only frequency supported (`fmhz`) is 100 (MHz).

- `<pci_settings_i>` is :

  `mode=<pci_mode_i>[,phy_mode=<phy_i>]`

  `i` is the index of the SerDes module (0 or 1), the valid values for `mode` are or, and `phy_mode` is optional and is described in "PCIE PHY CRSS, SRNS and SRIS support"



`pcie`*i* is required only if the mode for the corresponding SerDes*i* is one of `pcie`, `pcie&xpcs0` or `pcie&xpcs1`.

- `<xpcs_settings_i_j>` is :

  `speed=<10M|100M|1G|2G5>`

  `i` is the index of the SerDes module (0 or 1), `j` is the index of the XPCS interface (0 or 1).



XPCS0 is GMAC0 on SerDes0 and PFE0 on SerDes1.
XPCS1 is PFE2 on SerDes0 and PFE1 on SerDes1.
GMAC0 can't be used as SGMII for now.
Options `xpcsi_j` are required only if the string `xpcsj` can be found in the value of the option `mode` for the corresponding SerDes*i* module.

In case of SGMII only Mode 3 (1G) `serdesi` option `mode` must be set to `xpcs0&xpcs1` and both `xpcsi_0` and `xpcsi_1` should be configured for SerDes*i*. In case of SGMII only Mode 4 (2.5G), `serdesi` option `mode` can be set to `xpcs0` and only xpcs*i*_0 is required (as lane1 is unused).

The default value of `hwconfig` for S32G2 EVB and S32G274A RDB2 platforms is (all on one row):

```
=> setenv hwconfig "serdes0:mode=pcie,clock=ext;pcie0:mode=rc;
                    serdes1:mode=xpcs0&xpcs1,clock=ext,fmhz=125;xpcs1_0:speed=2G5;xpcs1_1:speed=2G5"
```

To override the default settings, the U-Boot environment variable `hwconfig` must be changed at runtime. If the variable is not set, neither PCIe nor GMAC nor PFE is enabled on any of the two SerDes modules.



If an SD-card is reused on different SoC versions, the user must make sure that both the U-Boot binary and the `hwconfig` variable are properly set/reset, since writing a new U-Boot will not change a pre-existing `hwconfig` value. For more details on the steps to set/reset the U-Boot environment, please refer to the section "How to reset U-Boot environment" in this manual.

9.1.1 PCIE PHY CRSS, SRNS and SRIS support

CRSS (Common Reference Clock, Spread Spectrum Clock) and SRIS (Split Reference, Independent Spread Spectrum Clock) can be enabled using the environment variable 'hwconfig'. The token which needs to be used is 'phy_mode', and can only be used in PCIe-only mode with external clock.



With respect to clock support, SRNS phy mode is an exception. It can also be used with internal clock. In this case the limitation is that it cannot be used at speeds higher than GEN2.

With this, the format of 'hwconfig' is (all on one row):

```
setenv hwconfig "serdes0:mode=pcie,clock=ext;pcie0:mode=<rc|ep>,phy_mode=<mode>;
                 serdes1:mode=pcie,clock=ext;pcie1:mode=<rc|ep>,phy_mode=<mode>"
```

where `<mode>` has one of the following valid values:

crns	crss	srns	sris



CRSS or SRIS PCIe PHY mode cannot be used with internal clock.

Spread Spectrum clocking can be enabled on S32G2 EVB platform, on PCIe0, by using the SW12 switch. Please note that Spread Spectrum clock should be provided externally, otherwise the non-Spread Spectrum clock will be used.



By default, if <phy_mode> is not specified in the 'hwconfig' environment variable, PCIe PHY mode is set to CRNS (Common Reference Clock, No Spread Spectrum Clock).
When using CRSS or SRIS PCIe PHY mode, no SGMII mode can be used on the same SerDes subsystem. Therefore, an example of 'hwconfig' environment variable modified in order to enable CRSS on PCIe0 and SRIS on PCIe1, would be:

```
setenv hwconfig "serdes0:mode=pcie,clock=ext;pcie0:mode=rc,phy_mode=crss;
                 serdes1:mode=pcie,clock=ext;pcie1:mode=rc,phy_mode=sris"
```

If CRSS, SRNS or SRIS PCIe PHY mode is enabled in 'hwconfig' environment variable, there are no other changes required for Linux Kernel, in order for CRSS, SRNS or SRIS PCIe PHY mode to be enabled, as U-Boot will automatically adjust the device tree passed to the OS to reflect this choice.

9.2 PCIe in Linux

With respect to Linux kernel setup, PCIe support is enabled by default.
PCIe enablement and selection EP versus RC is done exclusively from U-Boot, following the steps in section "PCIe in U-Boot" to select the mode from U-Boot's `hwconfig` variable. In particular, that means that U-Boot must have PCIe support in order to enable it in the kernel.

9.2.1 Enable PCIe manually

In order to have PCIe enabled, the kernel *.config* file must contain the following CONFIGs:

```
CONFIG_PCI=y
CONFIG_PCI_S32CC=y
CONFIG_PCIE_DW_PLAT_HOST=y
CONFIG_PCIE_DW_PLAT_EP=y
CONFIG_PCI_ENDPOINT=y
```

Please note that based on arch/arm64/configs/s32cc_defconfig, above CONFIG_* options should be enabled by default.



Both RC and EP PCIe support must be enabled at build time in the kernel to allow any of the two modes to be selected at runtime from U-Boot.

9.2.2 Rescan PCIe Bus

In the case when the S32 board is configured as RootComplex and for some reason an EndPoint is not enumerated or is not enumerated correctly (e.g. slower initialization or another reason), we may need to rescan the corresponding PCIe bus.

For example, for PCIe0:

remove the bus pcie0:

```
echo 1 > /sys/bus/pci/devices/0000\:00\:00.0/remove
```

rescan the bus pcie0:

```
echo 1 > /sys/bus/pci/rescan
```

This will trigger a re-enumeration of the PCIe bus and re-initialization of its EndPoints.

9.2.3 PCIe MSIs

There are two PCIe MSI delivery mechanisms supported by S32G2 and configurable in the Linux BSP:

1. Writes to the GIC-500 configuration space, triggering a message-based GIC SPI. Linux BSP currently reserves a range of 16 physical interrupt numbers, starting with GIC interrupt 167. This is configurable through the Linux device-tree, should this range need to be modified. If you decide to use a different interrupt range, make sure to choose one that is not in use by active SoC peripherals.

   Refer to the S32G2 Reference Manual (the Interrupt Routing annex) for the list of SoC interrupts.

   Refer to Documentation/devicetree/bindings/pci/nxp,s32-pcie.yaml for the relevant device-tree bindings in Linux BSP.

   This is the default configuration. It is most useful for multi-queue PCIe Endpoints such as NVMe-s, advanced Ethernet cards, or for multiple Endpoints connected to the PCIe RootComplex.

2. Incoming MSIs can also be handled by iMSI-RX (Integrated MSI Receiver [AXI Bridge]), an integrated MSI reception module in the PCIe controller's AXI Bridge which detects and terminates inbound MSI requests received on the RX wire. Instead, a hard-wired interrupt documented as "DSP AXI MSI Interrupt" in the SoC RM (Interrupt Routing annex) is asserted.

   The hardware defaults enable this option at reset, but the PCIe Host device-tree node overrides it with the other option, presented above.

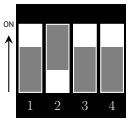
   The disadvantage of this approach is that only one hardware interrupt can be used, thus multiplexing multi-queue Endpoints onto a single local SPI and potentially producing a single-core performance bottleneck.

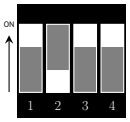
Changing the default MSI handling from GIC SPIs to hard-wired interrupts can be done fairly easily by looking up and commenting the *"msi-parent"* node property in arch/arm64/boot/dts/freescale/s32cc.dtsi for the PCIe node of interest and recompiling the device-tree:

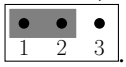
```
/* msi-parent = <&gic>; */
```

9.3 PCIe Clock Configuration

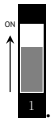
In order to use SerDes lanes as PCIe, the clock should be set to 100 MHz.

In case of S32G2 EVB, for PCIe0, set **SW12** 

in order to select 100 MHz. For PCIe1, set **SW13** 

along with **J165** 

.

In case of S32G274A RDB2, for PCIe1, set **SW8** 

.

10 HSE Security Support

The NXP Hardware Security Engine is a security subsystem aimed at running relevant security functions for applications with stringent confidentiality and authenticity requirements. This chapter describes the supported HSE features.

10.1 Prerequisites

The HSE Firmware is required to leverage the features described in the next sections. The latest supported firmware versions are the following:

- HSE_FW_S32G2_0_2_51_0
- HSE_FW_S32G2_1_2_51_0

HSE is a separate NXP product, available through Flexera. For more information, and to find out how to obtain HSE, please contact our marketing department.

Information on how to configure the HSE Firmware is available in the HSE Reference Manual, available on the NXP DocStore. More information regarding the HSE API is available in the HSE Service API Reference Manual, available on Flexera.

10.2 Building HSE Features with Yocto

The following HSE features can be automatically built with Yocto:

- PKCS11 Support
- HSE Linux Driver
- HSE Secure Boot
- HSE Verified Boot
- HSE Crypto Driver in OP-TEE

To enable PKCS11 Support & HSE Linux Driver, update the *conf/local.conf* file in the Yocto build directory with these lines:

```
DISTRO_FEATURES:append = " hse"
NXP_FIRMWARE_LOCAL_DIR = "/path/to/firmware/folder"
```

To enable SECBoot Support, update the *conf/local.conf* file in the Yocto build directory with these lines:

```
DISTRO_FEATURES:append = " secboot"
NXP_FIRMWARE_LOCAL_DIR = "/path/to/firmware/folder"
```



If Secure Boot Support is used, formatting the HSE key catalog without activating secure boot will result in a panic after a reboot.

Enabling HSE Secure Boot in the *conf/local.conf* file will automatically enable PKCS11 Support & HSE Linux Driver as well. A maximum of four RSA key pairs can be generated for the BL images. A secure boot activation script - *secboot_script.sh* - will be generated as well, containing the *hse-secboot* command specific to the chosen boot medium. The script will use the extracted public keys. The public keys and the script are stored in the rootfs, in */etc/keys/secboot* directory. If you want to use specific private keys and/or specific HSE key handles, you can set them in *conf/local.conf* file as follows:

```
RSA_PRIV_BL2 = "/path/to/bl2_private_key.pem"
RSA_PRIV_BL31 = "/path/to/bl31_private_key.pem"
RSA_PRIV_BL32 = "/path/to/bl32_private_key.pem" (used only if optee is present in DISTRO_FEATURES)
RSA_PRIV_BL33 = "/path/to/bl33_private_key.pem"
BL2_HANDLE = "0x010700"
BL31_HANDLE = "0x010500"
BL32_HANDLE = "0x010501"
BL33_HANDLE = "0x010502"
```

All BL image key variables must be valid paths to existing RSA private keys or paths where you want the TF-A to generate RSA private keys at compile time. The RSA public keys that are installed in the rootfs are extracted from these keys. The HSE key handles must be valid key handles corresponding to the HSE NVM key catalog format defined at examples/hse-secboot/keys-config.h. By default, all BL images use the BL2 image's private key and HSE key handle (0x010700 - NVM catalog, group 7, slot 0).

To enable VerifiedBoot Support, update the *conf/local.conf* file in the Yocto build directory with these lines:

```
DISTRO_FEATURES:append = " verifiedboot"
```

The resulted image will contain the public key used for signature verification appended to the TF-A DTB. For a SD image only the kernel and DTB signature are signed and checked. For a QSPI image the kernel, DTB and rootfs are signed and checked. The public and private keys used in verified boot can be found in TF-A yocto build folder under *image/etc/keys/verifiedboot* build directory.

To also enable the HSE Crypto Driver in OP-TEE, append "optee" to the DISTRO_FEATURES variable after enabling either PKCS11 Support & HSE Linux Driver or SECBoot Support:

```
DISTRO_FEATURES:append = " hse optee"
NXP_FIRMWARE_LOCAL_DIR = "/path/to/firmware/folder"
```

or

```
DISTRO_FEATURES:append = " secboot optee"
NXP_FIRMWARE_LOCAL_DIR = "/path/to/firmware/folder"
```

`NXP_FIRMWARE_LOCAL_DIR` must point to the the location of the folder containing one or more HSE Firmware packages. A HSE Firmware package is a folder with the name HSE_FW_S32G2_[`HSE_VERSION`]. In order to use a specific HSE Firmware package during build, set `HSE_VERSION` in *conf/local.conf*.

Important configuration variables for enabling HSE Features with Yocto:

- `HSE_VERSION`: the numerical version of the firmware, separated by underscores, as provided in the name of the firmware package. Set by default to the standard firmware version.
- `HSE_LIC`: specifies the name of the license provided in the HSE Firmware package and its checksum. Possible values: "license.rtf", "license.txt". Set by default to the standard version's license name.
- `HSE_LIC_MD5`: MD5 Checksum of the license.
- `HSE_SOC_REV`: Some HSE Firmware packages are shipped with two firmware binaries, each corresponding to a SOC revision. Not set by default

Example of custom configuration for the HSE Firmware package in Yocto

If one wants to use the premium version, 1.2.51.0, for a target board which has a rev2.1SOC and the license checksum is different from the one in the standard package, *conf/local.conf* must be updated with the following lines:

```
NXP_FIRMWARE_LOCAL_DIR = "/path/to/firmware/folder"
HSE_VERSION = "1_2_51_0"
HSE_SOC_REV = "rev2.1"
HSE_LIC_MD5 = "6720961e568628c6e229459cc3c5bf87"
```



Yocto is configured to use Firmware Version 0.2.51.0by default

This is the content of the folder pointed by `NXP_FIRMWARE_LOCAL_DIR` :

```
ls /path/to/firmware/folder
HSE_FW_S32G2XX_1_2_51_0
```

10.3 HSE Userspace Driver

The userspace HSE messaging module (`libhse.so.2.1`) provides direct low-level access to HSE, by using the Userspace I/O (UIO) driver which allows mapping shared memory between HSE and userspace for registers, service descriptors, internal memory and reserved memory. The Userspace I/O driver needs to be kept in sync with the userspace library, as some control structures reside in shared memory - to this end a version control check has been put in place.

The userspace library enables plenty of already-existing user space libraries and tools to be used in the same application and features multi-process and multi-thread support, allowing up to 15 different instances (with MU channel affinity) to independently send requests to HSE firmware at the same time. It allows the user application to leverage all HSE functionality: administrative services (such as PUBLISH_SYS_IMAGE); any crypto services supported by HSE firmware, regardless of being defined or not in the PKCS#11 specification or other standard interfaces. The userspace library allows the upper layers to use hidden or encrypted keys, referencing them by key handle.

Both the userspace library and Userspace I/O driver are portable, hence there is no dependency on firmware version.



There is a maximum of 16 channels per MU instance, with the first being reserved just for administrative services, leaving 15 usable for crypto operations.

10.3.1 Prerequisites

The Linux Kernel must be compiled with the `CONFIG_UIO_NXP_HSE` config enabled.

10.3.2 Configuration

The following Kconfig options are available:

- Messaging Unit Interface (CONFIG_UIO_NXP_HSE_MU):

  There are 4 Messaging Unit instances available for interfacing application processor subsystems with HSE and the user can configure which one is used for sending service requests and receiving replies. By default the chosen MU is MU0, which does not need to be activated by the HSE firmware beforehand. If another MU is chosen, the userspace can not request the activation of all MUs through UIO with the default HSE firmware.

- Driver internal memory size (CONFIG_UIO_NXP_HSE_INTL_MEM_SIZE)

  The user can configure the internal size of the UIO driver which is used for driver internal structures and objects/metadata. This represents the order of magnitude of page allocation, therefore the number of pages allocated will be equal to 2 to the power of this order. This order is ranged from 0 to 8 (default 4), the maximum internal memory size is 256 pages.

  A reserved DDR memory range is required, configurable via device tree: "hse_reserved" node in *s32cc.dtsi*.



The address and size of the reserved memory node can be modified, however, due to access limitations for some HSE services, the entire memory range should reside between 0x80000000 and 0xDFFFFFFF



HSE does not share a coherency domain with the A53 processor cores, so for them to exchange data with the firmware, the shared memory needs to be mapped as non-cacheable. Therefore, none of the standard libc routines can be used on this memory range, since they present the risk of causing an alignment fault more often than not. Libhse provides separate memory management routines that take physical memory alignment into account (this does not apply to the kernel crypto driver or OP-TEE drivers, where explicit cache management operations are performed)

10.4 PKCS11 Support

The HSE support for PKCS11 provides a userspace module (`libpkcs-hse.so`) that enables communication with HSE from userspace applications. The PKCS11 support is based on the PKCS11 v3.0 spec [3].

10.4.1 Prerequisites

The current implementation has been tested with the following:

- libp11 0.4.11 (2020-Oct-11)
- OpenSSL 3.0.5 (2022-Jul-5)
- OpenSC 0.21.0 (2020-Nov-24)

Testing can be done with either libp11 or OpenSC's *pkcs11-tool*, both of which depend on OpenSSL. Since the HSE PKCS11 module is compiled for `aarch64`, all of them need to be cross-compiled for `aarch64`, as well.

Take note that the paths provided in the following sections are simply examples, and can be changed to fit your build environment.

Before beginning cross-compilation, make sure a cross-compiler is installed, which fits the desired target system's architecture. The `CROSS_COMPILE` environment variable should already be set, if you have followed 3.2.2



Going forth, it is assumed `CROSS_COMPILE` is set.

[3] https://docs.oasis-open.org/pkcs11/pkcs11-base/v3.0/pkcs11-base-v3.0.html

10.4.2 Building OpenSSL 3.0.5 for aarch64

The OpenSSL source code can be download from the project's website:

www.openssl.org/source/openssl-3.0.5.tar.gz

Configure the build system for aarch64 cross-compilation:

```
./Configure linux-aarch64 --prefix=/<path>/<to>/openssl-aarch64 --openssldir=/<path>/<to>/openssl-aarch64
```

As before, `-prefix` indicates a directory in which to place the cross-compiled files. It is recommended to use the option, otherwise the files will be placed in the host's *$PATH*. The path provided is an example.

OpenSSL can now be built:

```
make
sudo make install
```

The compiled files can be found under */<path>/<to>/openssl-aarch64*. *lib/libcrypto.so.3* must be copied to the target's */usr/lib* directory. Run `ldconfig` afterwards, so that the library can be dynamically linked to.

10.4.3 Building libp11 0.4.11 for aarch64

The source code for libp11 can be download from the project's GitHub page:

https://github.com/OpenSC/libp11/releases

Configure the build system for aarch64 cross-compilation:

```
./configure CC=${CROSS_COMPILE}gcc --host=aarch64-none-linux-gnu --prefix=/<path>/<to>/libp11-aarch64 \
        OPENSSL_CFLAGS=-I/<path>/<to>/openssl-aarch64/include LDFLAGS=-L/<path>/<to>/openssl-aarch64/lib
```

`-host` specifies the target architecture, while `-prefix` indicates a directory in which to place the cross-compiled files. It is recommended to use the option, otherwise the files will be placed in the host's *$PATH*. The path provided for `-prefix` is an example.

libp11 can now be built:

```
make
sudo make install
```

The compiled files are under *<path>/<to>/libp11-aarch64*. *lib/libp11.so.3.5.0* must be copied to the target's */usr/lib* directory. Run `ldconfig` afterwards, so that the library can be dynamically linked to.

10.4.4 Building OpenSC's pkcs11-tool

The OpenSC source code can be downloaded from the project's GitHub page:

https://github.com/OpenSC/OpenSC/releases

Install the prerequisites beforehand:

```
sudo apt-get install pcscd libccid libpcsclite-dev libssl-dev libreadline-dev \
        autoconf automake build-essential docbook-xsl xsltproc libtool pkg-config
```

Configure the build system for aarch64 cross-compilation. OpenSSL must have been cross-compiled beforehand, as OpenSC needs to link against it:

```
./bootstrap
./configure --host=aarch64-linux --prefix=/<path>/<to>/opensc-aarch64-test --enable-openssl \
        CC=${CROSS_COMPILE}gcc \
        LDFLAGS="-g -Wl,-rpath-link,/<path>/<to>/openssl-aarch64/lib" \
        OPENSSL_LIBS="-lcrypto -L/<path>/<to>/openssl-aarch64/lib" \
        OPENSSL_CFLAGS=-I/<path>/<to>/openssl-aarch64/include
```

As before, `-prefix` indicates a directory in which to place the cross-compiled files. It is recommended to use the option, otherwise the files will be placed in the host's *$PATH*. The path provided is an example.

OpenSC can now be built:

```
make
sudo make install
```

The compiled files are under */<path>/<to>/opensc-aarch64*. *lib/libopensc.so.7.0.0* must be copied to the target's */usr/lib* directory. Run `ldconfig` afterwards, so that the library can be dynamically linked to.

bin/pkcs11-tool can be placed anywhere on the target (e.g. */home/<user>*).

10.4.5 Building the HSE PKCS11 module

Clone the module's source code:

```
git clone https://github.com/nxp-auto-linux/pkcs11-hse
cd pkcs11-hse/
git checkout release/bsp42.0
```

Since the module communicates with HSE, the HSE FW interface sources are needed and must be present in the host file system. As such, `HSE_FWDIR` should be set before running make (or provided when running make). Depending on firmware, the interface files have to correspond to premium or standard HSE FW.

The value used for `HSE_FWDIR` should point to the folder that contains the HSE FW *interface* folder, */<path>/<to>/HSE_FW_<platform>_<premium>_<vmajor>_<vminor>_<vpatch>*. The options represent:

1. <platform> - S32G2, S32G3, or S32R45
2. <premium> - 1 for premium FW, 0 for standard
3. <vmajor> - major version number
4. <vminor> - minor version number
5. <vpatch> - patch version number

In the case of standard S32G2 FW, `HSE_FWDIR` would be:

```
/<path>/<to>/HSE_FW_S32G2XX_0_2_51_0
```

To compile, run:

```
make HSE_FWDIR=/<path>/<to>/HSE_FW_<platform>_<premium>_<vmajor>_<vminor>_<vpatch>
```

This generates the `libpkcs-hse.so` and `libhse.so.2.1` modules. The `libhse.so.2.1` library should be copied to the target's rootfs in */usr/lib*. Afterwards, run `ldconfig` so that it can be dynamically linked to.

10.4.6 Building and running the libp11 PKCS11 HSE example

A usage example is provided in the repo under examples/. The sample application links against OpenSSL and libp11 to use functions provided by both. The demo application will:

- initialize libp11 with the `libpkcs-hse.so` module
- find the slot and token corresponding to HSE
- use openssl to generate two RSA key pairs
- store the key pairs in HSE
- enumerate the stored keys
- remove one of the key pairs from HSE

Since the application must be linked against both *libcrypto* and *libp11*, the directories in which the cross-compiled OpenSSL and libp11 are stored must be specified:

```
make OPENSSL_DIR=/<path>/<to>/openssl-aarch64 LIBP11_DIR=/<path>/<to>/libp11-aarch64
```

Copy the demo application, then run from the target's rootfs:

```
./pkcs-keyop /<path>/libpkcs-hse.so
```

The application outputs a message for each step described previously.

10.4.7 Running the pkcs11-tool PKCS11 HSE examples

Alternatively, instead of `pkcs-keyop`, `pkcs11-tool` can be used to load RSA keys (public/pair), EC keys (public) and AES keys. The `-id` switch corresponds to the key's slot (`00`), group (`06`) and catalog (`01`), in hexadecimal, from the HSE Key Catalog. Some examples:

```
./pkcs11-tool --module /<path>/libpkcs-hse.so --write-object /<path>/rsa_keypair.der \
        --type privkey --id 000601 --label "HSE-RSAPRIV-KEY"
./pkcs11-tool --module /<path>/libpkcs-hse.so --write-object /<path>/rsa_keypub.der \
        --type pubkey --id 000701 --label "HSE-RSAPUB-KEY"
./pkcs11-tool --module /<path>/libpkcs-hse.so --write-object /<path>/ec_keypub.der \
        --type pubkey --id 000401 --label "HSE-ECPUB-prime256v1-KEY"
./pkcs11-tool --module /<path>/libpkcs-hse.so --write-object /<path>/aes.key \
        --type secrkey --key-type AES:256 --id 000101 --label "HSE-AES-256-KEY"
```

The tool displays a message if the key import operation is successful.

10.4.8 Building the libhse examples

Two examples demonstrating the usage of the HSE Userspace Library (`libhse`) are provided: `hse-sysimg` and `hse-encrypt`. As they both depend on the HSE FW interface, *make* must be invoked in the *examples* folder while indicating the path to the HSE FW *interface* directory:

```
make HSE_FWDIR=<path>/<to>/HSE_FW_<platform>_<premium>_<vmajor>_<vminor>_<vpatch>
```

10.4.9 Building the HSE Secure Boot demo application

For the HSE Secure Boot support, a userspace demo application is provided as part of HSE PKCS#11 repo. The demo application requires the headers from a cross-compiled OpenSSL to read and interpret the key used for the authentication of the TF-A FIP, which can be built as described in section 10.4.2. The HSE FW interface is also required.

From the examples/ folder:

```
make OPENSSL_DIR=/<path>/<to>/openssl-aarch64 \
        HSE_FWDIR=/<path>/<to>/HSE_FW_<platform>_<premium>_<vmajor>_<vminor>_<vpatch>
```

The *hse-secboot* demo application must be copied to the target board's rootfs, as well as the OpenSSL cross-compiled library, as described in section 10.4.2.

The help for the demo can be brought up with:

```
./hse-secboot -h
```

10.5 HSE Secure Boot

Secure boot refers to a three-stage process of successive authentications starting from HSE and eventually reaching the Linux Kernel Image. This process requires a "Root of Trust" to begin with and which is known to be secure. In this case, the HSE acts as the Hardware Root of Trust .

Each image is authenticated by the preceding step. Thus, the secure boot flow is the following:

1. BootROM passes control over to HSE FW;
2. HSE FW authenticates TF-A's Boot Loader Stage 2 (BL2);
3. BL2 authenticates the other BL3x components: BL31 (Secure Monitor), BL32 (Trusted OS) and BL33 (U-Boot);
4. U-Boot authenticates the Kernel image.

10.6 Manually Building TF-A and Kernel for Authentication

Before beginning, make sure you have followed sections 3.2.1, 3.2.2 and 3.2.4, and obtained the U-Boot and TF-A source code repositories.

10.6.1 Configure TF-A's BL2 & BL3x Authentication

To authenticate the TF-A's BL2 image, HSE uses Advanced Secure Boot (ASB). The system configuration is saved as a **System Image** (SYS_IMG), which contains data related to imported keys, Secure Memory Region (SMR) configuration and Core Reset (CR) entry configuration. The SYS_IMG is signed with a device-specific key.

The keys used for ASB are stored in a **NVM key catalog**, which contains multiple slots for multiple key types and authentication schemes.

The **Secure Memory Region entry** is used to define which data needs to be authenticated from the boot medium, which key and authentication scheme is required, and where to place the data in memory after authentication.

The **Core Reset entry** is used to define which core should be enabled and what address to jump to after authentication, as well as what action to take in case authentication fails.

HSE will authenticate and load BL2 according to the SMR entry. After HSE passes control to BL2 as specified in the CR configuration, BL2 will perform some minimal platform initalizations and perform the authentication chain for each of the BL3x components.

Each BL3x image will be authenticated by BL2 at run-tme through TF-A's authentication framework according to S32's Chain of Trust (CoT). The CoT establishes a series of trust relationships between BL2 and the rest of the BL3x components. TF-A provides a reference implementation of ARM's Trusted Board Boot Requirements (TBBR) CoT. The ARM's TBBR document describes the set of features required for a secure boot process. For more information please consult: https://developer.arm.com/documentation/den0006/latest/.

The S32 Secure boot respects only a portion of TBBR's CoT. For example, the TBBR states that the RSA public keys used in the process of BL3x images authentication must be themselves authenticated as part of the component's CoT. The S32's CoT implementation does not include this step because it's already fulfilled by HSE: all of the keys used in the Secure Boot process are kept in the NVM key catalog. The catalog is authenticated by HSE during its boot. Hence, the authenticity of the RSA public keys is guaranteed.

10.6.1.1 Building U-Boot with support for HSE features

Begin by running make with the desired defconfig:

```
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- <board>_defconfig
```

You can also export the CROSS_COMPILE environment variable, so as not to repeat it for every build:

```
export CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu-
```

To be able to use HSE features, the following configuration options must be enabled or changed in menuconfig (`make menuconfig`):

```
CONFIG_NXP_HSE_SUPPORT=y
CONFIG_NXP_HSE_FW_FILE=<path>/<to>/<HSE>/<directory>/<hse_fw_file>.bin.pink
```

Afterwards, U-Boot can be compiled:

```
make
```

This generates the *u-boot-nodtb.bin* file, which is needed for the next step, compiling Arm Trusted Firmware.

10.6.1.2 Building TF-A with support for HSE features

Secure Boot requires up to 4 pairs of RSA Keys for signing and authenticating BL2, BL31, BL32 (optionally) and BL33 respectively. The private keys can be provided by the user, but, if they are not, TF-A will automatically generate them during build.

To manually generate an RSA private key to provide it to TF-A, one could use the openssl utilitary:

```
openssl genrsa -out <private_key>.pem 2048
```



Standard HSE FW supports key sizes up to 2048-bits, while Premium HSE FW supports up to 4096-bits, though it requires a different Key Catalog configuration.

TF-A requires only the private keys for signing the images and they are provided at build-time through four variables: `BL2_KEY`, `BL31_KEY`, `BL32_KEY` and `BL33_KEY`. If the user provides a path to a non-existing file, TF-A will automatically create that file and generate a private key in PEM format. If the user provides a path to an existing file, TF-A will check that the file contains a valid RSA private key in PEM format.

As stated before, the Secure Boot features make use of HSE's NVM Catalog to store the keys used during component authentication. In the catalog, each key is uniquely identified through an ID, called a **Key Handle**. All keys used in cryptographic operations performed with HSE Firmware are referenced by a key handle. Indeed, before referencing a key handle in a cryptographic operation, one must import the key material in a catalog slot corresponding to that key handle. For the secure boot feature, the keys are imported by the *hse-secboot* application.

At build time, TF-A embeds three key handles in the FIP, each corresponding to one of the BL3x components. These key handles will be extracted by BL2 at run-time and each will be used in the cryptographic operations performed for authenticating the BL3x images. The key handles are configurable at build-time through the use of three variables: `BL31_HSE_KEYHANDLE`, `BL32_HSE_KEYHANDLE`, `BL33_HSE_KEYHANDLE`.



By design, the HSE NVM Key Catalog has an upper limit of asymmetric keys that can be configured for the Standard HSE Firmware variant. This limit has been reached for the current NVM Catalog format of the Linux BSP. Hence, **if the keys/key handles are not specifically provided at build time, the same key that is used for signing BL2 will be used for all the other BL3x components by default. Please be aware that separate keys must be used for each BL image in a production environment!**

The default key handle used for secure boot is: 0x010700 - byte2 = 0x01 corresponds to NVM catalog, byte1 = 0x07 corresponds to group 7 of the catalog, byte0 = 0x0 corresponds to slot 0. The whole configuration of the catalog is defined in examples/hse-secboot/keys-config.h.

The TF-A Secure Boot external dependencies:

- The mbedTLS library version 3.4.1: https://github.com/Mbed-TLS/mbedtls/releases/tag/v3.4.1. The `MBEDTLS_DIR` build parameter must contain the path to the mbedtls directory. Follow the build instructions to compile the library before commencing with the TF-A build.
- The HSE Firmware Interface. Please consult the 10.1 section to obtain the correct firmware version. The path to the HSE Firmware directory must be specified in the `NXP_HSE_FWDIR` build parameter.

To build TF-A with support for HSE Secure Boot, the following make command needs to be run from the TF-A repo:

```
make ARCH=aarch64 PLAT=<board> BL33=/<path>/<to>/<u-boot-dir>/u-boot-nodtb.bin \
BL32=/<path>/<to>/tee-header_v2.bin> BL32_EXTRA1=/<path>/<to>/tee-pager_v2.bin SPD=opteed \
SECBOOT_SUPPORT=1 MBEDTLS_DIR=/<path>/<to>/mbedtls NXP_HSE_FWDIR=/<path>/<to>/HSE_FW_<version> \
BL2_KEY=/<path>/<to>/bl2_rsa2048_private.pem \
BL31_KEY=/<path>/<to>/bl31_rsa2048_private.pem \
BL32_KEY=/<path>/<to>/bl32_rsa2048_private.pem \
BL33_KEY=/<path>/<to>/bl33_rsa2048_private.pem \
BL31_HSE_KEYHANDLE=0x010700 \
BL32_HSE_KEYHANDLE=0x010700 \
BL33_HSE_KEYHANDLE=0x010700 \
```

Please note that `BL32`, `BL32_EXTRA1`, `SPD`, `BL32_KEY` and `BL32_HSE_KEYHANDLE` are optional build parameters and must be included in the build command only if one wishes to include OP-TEE in the Secure Boot enabled FIP.

The `SECBOOT_SUPPORT=1` option includes support for HSE firmware and reserves space in the FIP for the signature.

The public RSA keys can be extracted from their private counterparts using the following command:

```
openssl rsa -in <private_key>.pem -pubout -out <public_key>.pem
```

The RSA public keys are needed for BL authentication and are configured for this task in the next paragraphs.

10.6.1.3 SDCard Image Overview

```
IVT:                  Offset: 0x1000     Size: 0x100
DCD:                  Offset: 0x1200     Size: 0x2000    (optional)
HSE Reserved:         Offset: 0x1400
HSE SYS Image:        Offset: 0x52600    Size: 0xc000
AppBootCode Header:   Offset: 0x5e600    Size: 0x40
Application:          Offset: 0x5e640    Size: 0x150400

IVT Location:        SD/eMMC
Load address:        0x3407a2e0
Entry point:         0x34080000
```



Depending on the targeted board and U-Boot configuration, the output for component offsets, component sizes, IVT location, load address or entry point will differ.

The public RSA key must be copied to the boot partition of the SD Card, next to the Linux Kernel Image and DTB file. Flashing the signed fip.s32 to sd card can be done like this:

```
dd if=build/<board>/release/fip.s32 of=/dev/<sdcard_dev> seek=512 skip=512 \
        iflag=skip_bytes oflag=seek_bytes conv=notrunc,fsync
```

10.6.1.4 QSPI Flash Image Overview

```
IVT:                   Offset: 0x0            Size: 0x100
QSPI Parameters:       Offset: 0x200          Size: 0x200
DCD:                   Offset: 0x800          Size: 0x34
HSE Firmware:          Offset: 0xc38
HSE SYS Image:         Offset: 0x52a18        Size: 0xc000
```

```
AppBootCode Header:     Offset: 0x5ee18         Size: 0x40
Application:            Offset: 0x5ee58         Size: 0x40000

IVT Location:   QSPI
Load address:   0x3407e1a0
Entry point:    0x34085000
```



Depending on the targeted board and U-Boot configuration, the output for component offsets, component sizes, IVT location, load address or entry point will differ.

The public RSA key must be copied to the boot partition of the SD Card, next to the Linux Kernel Image and DTB file. Flashing the signed fip.s32 to QSPI flash can be done after copying the fip.s32 file to the rootfs of a SD Card image, by using the *s32cc_qspi_write.sh*:

```
./s32cc\_qspi\_write.sh FIP path/to/fip.s32
```

10.6.1.5 Enable TF-A FIP authentication

To enable the authentication of the TF-A FIP by HSE, the `hse-secboot` userspace binary is provided as part of the *pkcs11-hse* repository examples, as described in section 10.4.9. After building and copying the *hse-secboot* demo application to the target board's rootfs, you can use it to enable HSE Secure Boot for SD card, eMMC and QSPI flash.

The key input for any of the BL images represents the path to the RSA public key in PEM format and the HSE key catalog handle appended to the path, separated by ':'. There is an argument for each BL image key - BL2, BL31, BL32 and BL33, the BL32 image key is not required (OP-TEE-enabled TF-A FIP image). The key inputs are parsed before trying to extract the keys from the provided paths and importing them in the HSE key catalog. All the keys have a default HSE key catalog handle (0x010700 - NVM catalog, group 7, slot id 0). The key for BL2 image is required, being the first key to be imported. For every BL3X, if the default or the specified HSE key catalog handle matches a previous BL image key handle, the provided path will be ignored, meaning that the RSA public key used to verify that BL image is a previously imported key. To enable HSE Secure Boot, you can use one of the following commands:

```
./hse-secboot -b sd -s -d <device> --bl2_key bl2_rsa2048_public.pem:0x010700 \
                                   --b31_key -:0x010700 \
                                   --bl33_key -:0x010700
```

The options represent:

1. -b - specifies for which boot medium will format key store/will activate secure boot (if not present, SD card is the default boot medium)
2. -s - enable secboot
3. -d - specifies the <device> used as boot media (e.g. /dev/sdb or /dev/mtd0)
4. –bl2_key - full path to PEM format key file for BL2 and HSE key slot (required)
5. –bl31_key - full path to PEM format key file for BL31 and HSE key slot
6. –bl33_key - full path to PEM format key file for BL33 and HSE key slot

```
./hse-secboot -b mmc -s -d <device> --bl2_key bl2_rsa2048_public.pem:0x010700 \
                                   --b31_key -:0x010700 \
                                   --bl33_key -:0x010700
```

The options represent:

1. -b - specifies for which boot medium will format key store/will activate secure boot
2. -s - enable secboot
3. -d - specifies the <device> used as boot media (e.g. /dev/sdb or /dev/mtd0)
4. –bl2_key - full path to PEM format key file for BL2 and HSE key slot (required)
5. –bl31_key - full path to PEM format key file for BL31 and HSE key slot
6. –bl33_key - full path to PEM format key file for BL33 and HSE key slot

```
./hse-secboot -b qspi -s -d <device> --bl2_key bl2_rsa2048_public.pem:0x010700 \
                                    --b31_key -:0x010700 \
                                    --bl33_key -:0x010700
```

The options represent:

1. -b - specifies for which boot medium will format key store/will activate secure boot
2. -s - enable secboot
3. -d - specifies the <device> used as boot media (e.g. /dev/sdb or /dev/mtd0)
4. –bl2_key - full path to PEM format key file for BL2 and HSE key slot (required)
5. –bl31_key - full path to PEM format key file for BL31 and HSE key slot (path not specified, being a previously imported key)
6. –bl33_key - full path to PEM format key file for BL33 and HSE key slot (path not specified, being a previously imported key)

After the command is finished, simply reset the board to boot in secure mode. To verify if the board has booted in secure mode, check the BOOT_SEQ bit in the IVT, at offset 0x28. In the U-Boot command line, run:

```
=> mmc read ${loadaddr} 8 1
=> md.b ${loadaddr} 30

             0    2    4    6    8    A    C    E
800000000    d101 0060 0000 0000 0000 0000 0000 0000
800000010    0000 0000 0000 0000 0014 0000 0000 0000
800000020    00e6 0500 0000 0000 0900 0000 0000 0000
```

10.6.2 Configure Kernel Authentication

Kernel image authentication is provided by U-Boot, using the upstream verified boot method. This method uses an .its file, which defines a dtb/kernel configuration and a signing scheme. The following commands are given as if from a working directory which contains the Linux Kernel and U-Boot repository directories.

1. Create a directory in which to store the generated private key and certificate

```
mkdir kernel_keys
```

2. Generate the RSA2048 key pair

```
openssl genrsa -out kernel_keys/boot_key.key 2048
openssl req -batch -new -x509 -key kernel_keys/boot_key.key -out kernel_keys/boot_key.crt
```

Note that the names of the keys and the folder holding the keys are given as an example. If a different name is used for the keys or the folder, the private key and public key names must match. Moreover, the name of the private key must be used for the `key-name-hint` field in the .its file.

3. Create an .its file for Linux

The .its file will be used to create a FIT image, which will contain the kernel and dtb (optionally, the rootfs). An example .its file, for signing kernel and dtb configurations:

```
/dts-v1/;
/ {
        description = "kernel+dtb/fdt fit image";
        #address-cells = <1>;
        images {
                kernel-1 {
                        description = "kernel image";
                        data = /incbin/("<path>/<to>/Image");
                        type = "kernel";
                        arch = "arm64";
                        os = "linux";
                        compression = "none";
                        load = <load address>;
                        entry = <entry address>;
                        kernel-version = <1>;
                        hash-1 {
                                algo = "sha1";
                        };
                };
                fdt-1 {
                        description = "dtb blob";
                        data = /incbin/("<path>/<to>/<board>.dtb");
                        type = "flat_dt";
                        arch = "arm64";
                        compression = "none";
                        load = <load address>;
                        fdt-version = <1>;
                        hash-1 {
                                algo = "sha1";
                        };
                };
        };
        configurations {
                default = "conf-1";
                conf-1 {
                        kernel = "kernel-1";
                        fdt = "fdt-1";
                        signature-1 {
                                algo = "sha1,rsa2048";
                                key-name-hint = "<key_name>";
                                sign-images = "kernel", "fdt";
                        };
```

```
                };
        };
};
```



Make sure the `data` field for both `fdt-1` and `kernel-1` are updated when using a different configuration.

This .its file is provided as an example for defining a configuration (kernel and dtb combination), which is hashed and signed using the private key. Alternatively, the kernel image and dtb can be signed, by moving the signature block from the configuration block to both the kernel image and fdt blocks, replacing the hash blocks. For an example of this, see doc/uImage.FIT/sign-images.its.

The .its file should be placed in the Linux Kernel directory.

4. Build the mkimage tool

   From the U-Boot directory, run:

```
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- tools
```

5. Build Verified Boot components

   Before beginning, make sure you have followed sections 3.2.1 and 3.2.2.2. See section 10.6 for more details.

   If you haven't built U-Boot before (i.e. if you haven't completed section 10.6.1.1), start by running:

```
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- <board>_defconfig
```

   If you've already run the above command previously, you can skip it and continue by configuring the required options for Kernel authentication from `menuconfig`. If you want to authenticate both TF-A FIP and Linux, make sure you still have the `CONFIG_NXP_HSE_SUPPORT` and `CONFIG_NXP_HSE_FW_FILE` config options for FIP authentication enabled, from section 10.6.1.1.

   From the U-Boot directory, run:

```
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- menuconfig
```

   In menuconfig, make sure the following options are enabled:

```
CONFIG_OF_CONTROL=y
CONFIG_FIT_SIGNATURE=y
CONFIG_RSA=y
CONFIG_NXP_HSE_SUPPORT=y
CONFIG_NXP_HSE_FW_FILE=<path>/<to>/<HSE>/<directory>/<hse_fw_file>.bin.pink
```

   After saving the configuration, from the root of the U-Boot directory, run:

```
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu-
```

6. Build the Arm Trusted Firmware (DTB only)

   At this point, only the TF-A DTB must be built. Before TF-A appends the DTB to the FIP image, the DTB shall be appended with the FIT image verification (public) key (via the mkimage -K option - see further below).

```
make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- \
        ARCH=aarch64 PLAT=<plat> dtbs
```

7. Build the Linux Kernel

   This step is optional, as you can use the Kernel Image built in section 3.2.3.

   Otherwise, from the linux directory, run:

```
make ARCH=arm64 CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- \
        <soc_name>_defconfig
make ARCH=arm64 CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu-
```

   See section 3.2.3 for more information on building the Linux Kernel.

8. Sign the kernel image and pack it into a FIT image

   After the kernel has been compiled, from the U-Boot directory, run:

```
tools/mkimage -f ../linux/<its-filename>.its
              -K ../arm-trusted-firmware/build/<board>/release/fdts/<dtb-filename>.dtb
              -k ../kernel_keys
              -r <itb-filename>.itb
```

   The output of the mkimage command should display the parts of the FIT image: the kernel, dtb and configuration. The kernel and dtb have a hash value, while the configuration has a signed hash, if using the configuration signing method. Otherwise, the kernel and fdt have a signed hash.

   The options for mkimage specify:

```
-f    image tree source file describing the contents and structure
-K    public key destination; specify the dtb in which to write the
      public key/certificate, used for authentication
-k    private key source; specify directory where the keys are stored;
      private key is used for signing
-r    required; specify that keys are required, cannot boot if
      signature cannot be verified
```

   The resulting FIT image must be copied to the boot partition of the SD Card.

9. Build the Arm Trusted Firmware (using previously generated DTB)

   TF-A can be built normally, by following chapter 3.2.4, or with support for HSE Secure Boot, by following chapter 10.6.1.

   The Arm Trusted Firmware will be built to include the DTB in which the key was placed in the previous step. This will happen automatically, provided that no "clean" / "distclean" TF-A Makefile targets are run in-between.

10. Boot the board and stop at U-Boot command line

    Copy the FIT image from the SD Card to memory:

```
=> fatload mmc 0:1 <load-address> <fit-image-name>
```

    U-Boot must use the bootm command to boot, not booti. Trying to boot using booti displays the `Bad Linux Arm64 Image magic!` error, since the header for the FIT image is not recognized by booti.

```
=> run mmcargs
=> bootm <load-address>
```

    To check if the FIT image loaded on the board is correct, run:

```
iminfo
```

    The command checks the hashes of the image, dtb, and configuration, but does not check if the FIT image is properly signed.

10.7 HSE Linux Driver

This chapter contains general information about the HSE crypto driver, which provides support for offloading cryptographic operations to HSE's dedicated coprocessors through the kernel crypto API.

10.7.1 Prerequisites

The Linux Kernel must be compiled with the `CONFIG_CRYPTO_DEV_NXP_HSE` config enabled.

In order to use the HSE cryptographic offloading capabilities, HSE firmware must be installed and the key catalogs must be formatted before the kernel driver is probed. The AES-256 and HMAC groups from the RAM key store are used.

Following the steps described in section 10.4.9 in order to enable support for HSE firmware, *hse-secboot* must be used to format the HSE key catalogs:

```
./hse-secboot -f -d <device>
```

The options represent:

1. -f - format HSE key catalogs
2. -d - specifies boot media, e.g. /dev/mmcblk0

This can be used to format the key catalog without needing to boot in secure mode on the target device. Please verify that the RAM key group IDs and sizes are defined appropriately to your key catalog configuration:

- `CONFIG_CRYPTO_DEV_NXP_HSE_AES_KEY_GROUP_ID`
- `CONFIG_CRYPTO_DEV_NXP_HSE_AES_KEY_GROUP_SIZE`
- `CONFIG_CRYPTO_DEV_NXP_HSE_HMAC_KEY_GROUP_ID`
- `CONFIG_CRYPTO_DEV_NXP_HSE_HMAC_KEY_GROUP_SIZE`

A single Messaging Unit instance is used by the driver to communicate with HSE. The user can select a MU instance (out of a total of 4) in the configuration options (all MU instances have to be enabled beforehand in HSE firmware, except MU0) and must ensure exclusive access to it. In the crypto driver the MU is used in interrupt mode, so sharing channels of the same instance across multiple users is not possible.

10.7.2 Supported Algorithms

This driver currently supports the following crypto operations:

- Hashing: SHA1, SHA2
- Symmetric Key Ciphering: AES-CTR, AES-CBC, AES-ECB, AES-CFB, AES-OFB
- Message Authentication Codes: HMAC-SHA1, HMAC-SHA2
- Authenticated Encryption with Associated Data: AES-GCM
- True Random Number Generator: PTG.3 class

10.7.3 Configuration

The following Kconfig options are available:

- Messaging Unit (MU) Instance (CONFIG_CRYPTO_DEV_NXP_HSE_MU):

  There are 4 Messaging Unit instances available for interfacing application processor subsystems with HSE and the user can configure which one is used by the Linux driver for sending service requests and receiving replies. The MU instance indicated here shall be used in interrupt mode and therefore should be entirely reserved for the Linux crypto driver. Sharing an instance with another driver or application shall result in requests being dropped.

- Message Digest Support (CONFIG_CRYPTO_DEV_NXP_HSE_AHASH):

  Enables hash and hash-based MAC offloading to HSE.

- Symmetric Key Cipher Support (CONFIG_CRYPTO_DEV_NXP_HSE_SKCIPHER):

  Enables symmetric key cipher offloading to HSE.

- AuthEnc and AEAD Support (CONFIG_CRYPTO_DEV_NXP_HSE_AEAD):

  Enables authenticated encryption and AEAD offloading to HSE.

- Hardware RNG support (CONFIG_CRYPTO_DEV_NXP_HSE_RNG):

  Enables hardware true random number generation via HSE.

- Entropy cache maximum size (CONFIG_CRYPTO_DEV_NXP_HSE_RNG_CACHE):

  Total size of driver entropy cache. Used to improve request latency.

- RAM Key Catalog AES Group Configuration:

  AES 256-bit Key Group ID within RAM Key Catalog (CRYPTO_DEV_NXP_HSE_AES_KEY_GROUP_ID):

  This option specifies which key group is used by the driver for programming the AES 256-bit keys into HSE, depending on how the RAM catalog was initialized by the firmware.

  Number of key slots in the AES 256-bit Key Group (CRYPTO_DEV_NXP_HSE_AES_KEY_GROUP_SIZE):

  This option specifies the maximum number of keys that can be stored in the AES 256-bit key group.

- RAM Key Catalog HMAC Group Configuration:

  HMAC 1024-bit Key Group ID within RAM Key Catalog (CRYPTO_DEV_NXP_HSE_HMAC_KEY_GROUP_ID):

This option specifies which key group is used by the driver for programming HMAC 1024-bit keys into HSE, depending on how the RAM catalog was initialized by firmware.

Number of key slots in the HMAC 1024-bit Key Group (CRYPTO_DEV_NXP_HSE_HMAC_KEY_GROUP_SIZE):

This option specifies the maximum number of keys that can be stored in the HMAC 1024-bit key group.

- Debug Information for HSE Crypto Driver (CONFIG_CRYPTO_DEV_NXP_HSE_DEBUG):

  Enables printing driver debug messages to the kernel log.

10.8 HSE Crypto Driver in OP-TEE

The HSE Crypto Driver in OP-TEE provides cryptographic offloading capabilities for the OP-TEE Core and Trusted Applications (TAs). The design and configuration of the HSE OP-TEE Driver resembles the HSE Linux Driver. Its usage is oriented towards demonstrating how HSE's cryptographic and key management services could be useful for Trusted Applications.

10.8.1 Build Instructions

1. Build optee_os with HSE Crypto Driver Support by adding the **CFG_NXP_HSE=y** make parameter and setting up **CFG_NXP_HSE_FWDIR**:

```
make CROSS_COMPILE64=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- ARCH=arm \
        PLATFORM=s32 PLATFORM_FLAVOR=s32g2 CFG_NXP_HSE=y
        CFG_NXP_HSE_FWDIR=/<path>/<to>/HSE_FW_<platform>_<premium>_<vmajor>_<vminor>_<vpatch>
```

   Please refer to Building the optee_os component of OP-TEE section for more information on manually building the optee_os component.

2. Follow the instructions in the Configure TF-A's BL2 & BL3x Authentication section to build a FIP image containing the HSE Firmware binary. It is not mandatory to boot in Secure Boot mode in order to use the HSE Crypto Driver in OP-TEE, so the FIP signing and authentication operations can be skipped. Make sure U-Boot and TF-A are built with the specific HSE parameters as provided in the referenced section at the beginning of the paragraph. The optee_os component must be included in the FIP image by using the **BL32** and **BL32_EXTRA1** make parameters in the TF-A build command:

```
make ARCH=aarch64 PLAT=<board> BL33=<path>/<to>/<u-boot-dir>/u-boot-nodtb.bin
        BL32=<path>/<to>/tee-header_v2.bin \
        BL32_EXTRA1=<path>/<to>/tee-pager_v2.bin SPD=opteed \
        HSE_SUPPORT=1
```

10.8.2 Configure HSE OP-TEE Crypto Driver

The HSE Driver in OP-TEE is configured to use a number of pre-defined keygroups in the RAM/NVM Catalog. The whole Key Catalog is defined in examples/hse-secboot/keys-config.h. The keygroups defined in the OP-TEE's HSE Driver are just a reflection of the catalog in pkcs11-hse. If one changes the catalog, the keygroups in OP-TEE must be updated accordingly.

In the OP-TEE HSE Driver each keygroup has three distinct properties: catalog, ID and size. Each of these properties' configuration parameters adheres to the following format, the only variable being the key type of the keygroup:

- **CFG_NXP_HSE_<key_type>_KEYGROUP_CTLG**
  The value must be either 1 or 2, meaning the NVM or the RAM Catalog respectively.
- **CFG_NXP_HSE_<key_type>_KEYGROUP_ID**
  The value must be a valid index in the RAM/NVM Catalog.
- **CFG_NXP_HSE_<key_type>_KEYGROUP_SIZE**

`<key_type>` must be replaced with one of the following: AES, HMAC, SHARED_SECRET, ECCPAIR, ECCPUB, RSAPAIR, RSAPUB.

Example of setting up the AES keygroup configurations:

```
make ... CFG_NXP_HSE_AES_KEYGROUP_CTLG=2 \
         CFG_NXP_HSE_AES_KEYGROUP_ID=2 CFG_NXP_HSE_AES_KEYGROUP_SIZE=7
```



The keygroups used by the HSE Driver are defined in core/drivers/crypto/hse/crypto.mk. The example set above is a demonstration of modifying the config values without altering the source code.

The following build-time configuration variables are used to enable/disable the currently supported crypto drivers of HSE. If not stated otherwise, the configurations are enabled by default:

- **CFG_NXP_HSE_FWDIR**
  Must point to the folder that contains the HSE Firmware's *interface* folder. Example usage:

  ```
  make ... CFG_NXP_HSE_FWDIR=/<path>/<to>/HSE_FW_<platform>_<premium>_<vmajor>_<vminor>_<vpatch>
  ```

- **CFG_HSE_KP_PTA**
  Enables/Disables the TrustZone Key Provisioning Pseudo Trusted Application.
- **CFG_NXP_HSE_RNG_DRV**
- **CFG_NXP_HSE_HUK_DRV**
  Enables/Disables the Hardware Unique Key driver.
- **CFG_NXP_HSE_CIPHER_DRV**
- **CFG_NXP_HSE_RSA_DRV**
- **CFG_NXP_HSE_HASH_DRV**
- **CFG_NXP_HSE_MAC_DRV**
- **CFG_NXP_HSE_AUTHENC_DRV**
- **CFG_NXP_HSE_ECC_DRV**
  Disabled by default. Can be enabled using

  ```
  make ... CFG_NXP_HSE_ECC=y
  ```

  Enabling it will add support for ECDSA inside the crypto driver, but ECDH will stop working.

- **CFG_NXP_HSE_ECDH_DRV**
  Disabled by default. Needs to have support for ECC enabled as well, otherwise a compilation error will be generated. Can be enabled using

```
make ... CFG_NXP_HSE_ECC=y CFG_NXP_HSE_ECDH=y
```

  Due to limitations in the HSE, the output will not be in plain-text, it will be encrypted using AES-256-CTR with a zero-ed IV and key.

By default, the HSE OP-TEE Crypto Driver uses the third MU. The MU selection is done in the TF-A software component using the dedicated Device Tree fdts/s32cc-crypto.dtsi. The `status` property in the MU nodes determines which MU would be used by the HSE OP-TEE Crypto Driver, with `status = "okay"` value making the selection. Please note that only one MU is used by the driver. Hence, if multiple MU nodes have the property value set to `status = "okay"`, only the first node in the device tree containing this attribute value is registered by the driver.

Example of MU Device Tree Node:

```
mu2b@40212000 {
        compatible = "nxp,s32cc-hse";
        reg = <0x0 0x40212000 0x0 0x1000>,
              <0x0 0x22c02000 0x0 0x1000>;
        reg-names = "hse-regs",
                    "hse-desc";
        interrupts = <GIC_SPI 109 IRQ_TYPE_EDGE_RISING>, /* GIC 141 */
                     <GIC_SPI 110 IRQ_TYPE_EDGE_RISING>, /* GIC 142 */
                     <GIC_SPI 111 IRQ_TYPE_EDGE_RISING>; /* GIC 143 */
        interrupt-names = "hse-ack",
                          "hse-rx",
                          "hse-err";
        status = "okay";
};
```

10.8.3 Enable the OP-TEE Crypto Driver

Upon booting a freshly built FIP Image, the HSE OP-TEE Driver fails during its first initialization.

This happens because the Key Catalog is not yet formatted at that point. The only way to format the catalog is to build the `hse-secboot` example, as instructed in the paragraph 10.4.9, and run it in the Linux Userspace. Reset the board once the operation is complete:

```
hse-secboot -f -d <device>
reboot -n
```

Once the Key Catalog is formatted and the board resets, there should be no errors displayed by the HSE OP-TEE Driver during any subsequent initializations and the driver should function properly.

10.8.4 Features

This driver currently provides the following features:

- Symmetric Key Ciphering: AES-CBC, AES-ECB, AES-CTR
- Authenticated Ciphering: AES-GCM, AES-CCM
- Digests: SHA-1, SHA2-224, SHA2-256, SHA2-384, SHA2-512

- Message Authentication Code: HMAC-SHA-1, HMAC-SHA2-224, HMAC-SHA2-256, HMAC-SHA2-384, HMAC-SHA2-512, CMAC
- RSA encrypt/decrypt and sign/verify
- Elliptic curve cryptography sign/verify and shared-secret: ECDSA, ECDH
- True Random Number Generator: PTG.3 class
- Hardware Unique Key (HUK) derived from a HSE Shared Secret Key
- TrustZone Key Provisioning



The HSE OP-TEE Driver assumes that all the supported features are executed in the Life Cycle (LC) CUST_DEL, the default state of the SoC when it's shipped. In this state, the Host, i.e. the HSE OP-TEE Driver in this case, is granted SuperUser rights. Among others, one is granted the right to import keys into NVM Key Slots without encryption/authentication and the HSE OP-TEE Driver takes advantage of this in the RSA/ECC Crypto drivers: these drivers perform import/erase operations on NVM Key Slots. During vehicle intergration of the SoC, customers must consider advancing the LC of the chip to IN_FIELD. This particular state is associated with User rights which means that the Host can no longer import unencrypted/unauthenticated keys into NVM slots. The immediate consequence is that *the ECC/RSA operations can no longer be performed using the HSE OP-TEE Driver*. For more information about the LCs and the User/SuperUser rights, please consult the HSE Fimware Reference Manual.

11 DDR

11.1 Integration of DDR Tool generated code into TF-A

Below are the steps for integrating the code generated by the NXP DDR Tool into the BSP.
TF-A is compatible with the code generated using **S32 Design Studio 3.5.8** with the **Real-Time Drivers 4.4 Version 4.0.2 P03** extension package.

The default frequency used by S32G2 is 800 MHz. The clock frequency can be adjusted using the following defines within include/dt-bindings/clock/s32gen1-clock-freq.h

```
#define S32GEN1_DDR_PLL_VCO_FREQ (1600 * MHZ)
#define S32GEN1_DDR_FREQ (800 * MHZ)
```

Notice that the above values from TF-A are not the default configuration values in the DDR Tool. Please customize the `Clock Cycle Freq` and `DDR_CLK frequency` fields in the DDR Tool to match the above defines from TF-A.

After generating the DDR Code with the DDR Tool, it can be externally linked in TF-A by using the `CUSTOM_DDR_DRV` parameter for `make`. It must be set to a path (relative to the TF-A root folder) where all the DDR source and header files are, in the state they were generated by DDR Tool.



If the used external DDR Code version does not support decoupled DDR_GPRs setting, using it with the Shared Resource Manager feature enabled will lead to an abort if DDR_GPRs are protected from A53 access.

The first step is to prepare the code for the TF-A, which is done by calling the `import_ddr_drv` target with `make`. This step is not required for code generated with a DDR Tool version starting with the one included in the **Real-Time Drivers 4.4 Version 4.0.0** extension package. Then, the compilation is done as described in section Building the Arm Trusted Firmware, but also with the `CUSTOM_DDR_DRV` parameter set.

```
$ CUSTOM_DDR_FOLDER="$(realpath \
        --relative-to=/path/to/arm-trusted-firmware \
        /path/to/ddr/generated/folder)"
$ make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- \
        ARCH=aarch64 PLAT=<plat> BL33=<path-to-u-boot-nodtb.bin> \
        CUSTOM_DDR_DRV="$CUSTOM_DDR_FOLDER" import_ddr_drv
$ make CROSS_COMPILE=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- \
        ARCH=aarch64 PLAT=<plat> BL33=<path-to-u-boot-nodtb.bin> \
        CUSTOM_DDR_DRV="$CUSTOM_DDR_FOLDER"
```

11.2 Checking if Inline ECC Feature is Enabled

Checking if the Inline ECC feature is enabled can be done at runtime, by inspecting the *ECC_MODE* bits of the *ECCCFG0* DDR Controller register.

This can be done in U-Boot by running the following command:

```
=> md 0x403C0070 0x1
403c0070: 033f7f54                              T.?.
```

To confirm that ECC is enabled, the last hex digit of the read value should be 0x4 or 0x5. For other values, ECC is not enabled.

If the check needs to be done in Linux, it can be done using the *devmem2* tool:

```
root@s32g274aevb:~# devmem2 0x403c0070
/dev/mem opened.
Memory mapped at address 0x7f877a3000.
Read at address  0x403C0070 (0x7f877a3070): 0x033F7F54
```

The same value checking logic as for U-Boot can be applied here to confirm if ECC is enabled or not.

12 LLCE Support

The NXP Low Latency Communication Engine (LLCE) is an autonomous accelerator designed to offload communication tasks on the CAN, LINFlexD and FlexRay Protocols. It has hardware channels to abstract the communication that is fully supported through a firmware-defined host interface. A key advantage of this approach is that it delivers an optimized host interface to reduce the load on the host cores.

LLCE drivers rely on LLCE firmware, which is a separate NXP product available through https://www.nxp.com/. For more information, and to find out how to obtain LLCE Firmware, please contact NXP marketing department.



LLCE Logging capability is only available using the advanced version of the LLCE firmware.

The purpose of these drivers is to enable Linux users to access a set of features available in the LLCE firmware. The targeted areas are:

1. LLCE firmware loading
2. LLCE mailbox management
3. LLCE CAN interfaces management
4. LLCE CAN filters management
5. LLCE CAN Logging interfaces
6. LLCE LINFlexD
7. LLCE LPSPI

12.1 Yocto Integration

The Yocto build automates the building and deployment of the LLCE drivers. Their inclusion and loading into Linux kernel are controlled using a set of `DISTRO_FEATURES` that can be added to the `<builddirectory>/conf/local.conf` file. The set includes: `llce-can`, `llce-can-logger`, `llce-can-multihost`, `llce-linflex` and `llce-lpspi`. Each of them will be detailed in the subsequent chapters.

When working with LLCE, two other Yocto settings are relevant: `llce-fw-load` and `NXP_FIRMWARE_LOCAL_DIR`. The `llce-fw-load` must be part of the `DISTRO_FEATURES` and instructs the Linux drivers to load the necessary firmware, which should be stored in `NXP_FIRMWARE_LOCAL_DIR` folder. Therefore, `NXP_FIRMWARE_LOCAL_DIR` must contain the following binary files:

- dte.bin
- frpe.bin
- ppe_tx.bin
- ppe_rx.bin

These binaries are part of the LLCE firmware release and are located under `firmware/llce_bin/s32g2/bin/ghs/enablement` subfolder.

12.2 Manual compilation

The source code of the LLCE drivers is part of the Linux kernel repository and can be compiled and deployed manually. The drivers are conditionally included in the build by several Kconfig switches. The CAN relevant ones are placed under `menuconfig` menu:

```
→Networking support
  →CAN bus subsystem support
    →CAN Device Drivers
      →Platform CAN drivers with Netlink support
        →NXP LLCE CAN Support
```

After the compilation step, the locations of the LLCE drivers can be obtained using:

```
$ find -name "*llce*.ko"
./drivers/net/can/llce/llce_can.ko
./drivers/net/can/llce/llce_logger.ko
./drivers/mailbox/llce-mailbox.ko
./drivers/mfd/llce-core.ko
```

12.3 CAN

If the Yocto's `DISTRO_FEATURES` variable has the `llce-can` and `llce-fw-load` features added, the resulting image will have support for LLCE CAN. As such, the LLCE CAN interfaces will be accessible once reaching the Linux prompt. For example, these two lines appended to `<builddirectory>/conf/local.conf` will make it possible:

```
DISTRO_FEATURES:append = " llce-fw-load llce-can"
NXP_FIRMWARE_LOCAL_DIR = "/path/to/firmware/binaries/folder"
```

12.3.1 Usage example

1. Configure `llcecan0` and `llcecan14` interfaces

```
root@s32g274ardb2:~# ip link set up llcecan0 type can bitrate 500000 dbitrate 5000000 fd on
root@s32g274ardb2:~# ip link set up llcecan14 type can bitrate 500000 dbitrate 5000000 fd on
```

2. Capture the frames from all CAN interfaces.

```
root@s32g274ardb2:~# candump any,0:0,#FFFFFFFF >log &
```

3. Send a packet using `llcecan14`

```
root@s32g274ardb2:~# cansend llcecan14 15575555##3001122334455667788 99aabbcceedd112233
```

More information on how to send / receive packets through a SocketCAN interface in Linux can be found in section 13.0.1.

12.3.2 Timestamping

The LLCE CAN timestamp function enables the distribution of a joint timestamp base to all the CAN interfaces on the chip. It provides a temporal correlation of frames across all the CAN module instances and other events occurring on the platform depending on the timestamp source. The CAN bus can have multiple sources, and each can be selected based on the use case. The timestamp is enabled by default, and STM7 is chosen as the source. However, it can be changed by adapting the `idle-states` property of the `can_ts_ctrl` node found in the device tree.

All the available options for CAN timestamping are listed in include/dt-bindings/mux/nxp-s32g-can-ts.h.

12.3.3 Multi-Host

LLCE is an accelerator for CAN traffic whose resources can be partitioned between two host interfaces (HIFs). The split is made possible by the multi-host feature. Therefore, the controllers can be divided between multiple software instances. For example, software running on Cortex-M cores can have two controllers under its management, with the rest assigned to Linux running on Cortex-A cores. The assignment of the controllers to Linux is configured using nodes in the device tree. Consequently, all CAN controllers administered by Linux must be listed as enabled children of the `llce` device tree node. A disabled node will be considered handled by other software entities.

The multi-host feature is controlled by the `nxp,multi-hif` property. It can be enabled by updating the board's device tree with the multi-host property and the identification number used for the Linux host interface as follows:

```
&llce_mb {
    nxp,multi-hif;
    nxp,hif-id = <1>;
};
```

The identification number used for Linux to communicate with LLCE firmware in a multi-host scenario is set through another property, `nxp,hif-id`. The above snippet initializes it to 1, meaning that Linux will be advertised as HIF1.

Each HIF uses hardware FIFOs to send commands to LLCE firmware. To avoid race conditions, each HIF has a limited number of FIFOs that can be used in a multi-host scenario. Thus, some FIFOs might be shared when the number of assigned controllers to a HIF exceeds the number of FIFOs. These cases can result in reduced performance for controllers using shared FIFOs and will be signaled by the LLCE drivers using messages in the kernel log:

```
...
Interfaces that use shared TX/RX FIFOs:
       LLCE CAN 1 uses shared FIFOs: 7
       LLCE CAN 2 uses shared FIFOs: 6
       LLCE CAN 3 uses shared FIFOs: 5
       LLCE CAN 4 uses shared FIFOs: 4
       LLCE CAN 5 uses shared FIFOs: 3
       LLCE CAN 9 uses shared FIFOs: 7
       LLCE CAN 10 uses shared FIFOs: 6
       LLCE CAN 11 uses shared FIFOs: 5
       LLCE CAN 12 uses shared FIFOs: 4
       LLCE CAN 13 uses shared FIFOs: 3
...
```

`llce-can-multihost` is used to facilitate the integration with the software running on the M7 cores. The feature includes the addition of the `nxp,multi-hif` property to the device tree, drivers' compilation, deployment and probing. The firmware loading process is done by Linux drivers if the `llce-fw-load` option is used.

This is an example of how the `<builddirectory>/conf/local.conf` file should be adapted to enable the multi-host feature with firmware loading included:

```
DISTRO_FEATURES:append = " llce-fw-load llce-can-multihost"
NXP_FIRMWARE_LOCAL_DIR = "/path/to/firmware/binaries/folder"
```

To summarize, the process of integrating Linux into a LLCE multi-host scenario should include the following checkpoints:

1. Choose the appropriate HIF number for the Linux drivers from the device tree.
2. Adapt the device tree with the LLCE interfaces allocated to Linux HIF
3. Use the multi-host `DISTRO_FEATURES`.

12.3.3.1 Heterogenous Multi-Host CAN Demo

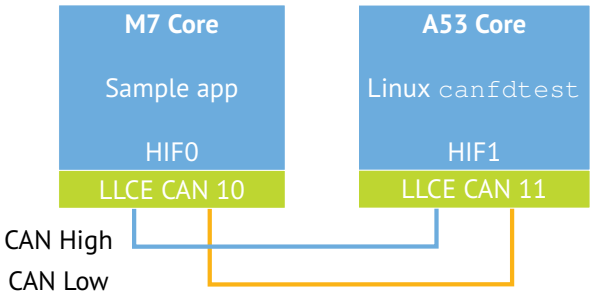


Figure 1: Heterogenous Multi-Host CAN Demo

Through the multi-hif capability, the LLCE accelerator allows the partitioning of the CAN controllers between two independent host interfaces. This is a demo where each HIF is controlled by a different type of core. HIF0 is initialized and controlled by the M7_0 core, while the HIF1 is managed by A53 cores which run Linux. To simplify things, one CAN controller is assigned to each HIF. As a result, HIF0 initializes CAN10, and HIF1 manages CAN11. Both controllers are connected through a physically conventional two-wire bus.

A sample application designed for multi-hif scenarios from LLCE FDK will be used as the software running on the M7_0 core. The application is named `can_mcal_demo_linux` and can be found in the `<LLCE_FDK>/sample_app_llce/llce_sample_app_af/sample_app_can_llce/can_mcal_demo_linux` folder. To configure and build the sample, please refer to the `README.txt` file in the same folder. The following steps assume the sample was configured, built, and resulted in `can_demo_linux.elf` file. Once started, it will load the LLCE firmware, configure HIF0 and wait for frames on the LLCE CAN 10 controller. For each received frame, a reply will be generated on the bus. This is done using the same heuristic as in `canfdtest`.

It is necessary to skip the firmware loading step in the Linux drivers since the M7_0 software will load it. Therefore, the following line must be added to `<builddirectory>/conf/local.conf` and the `llce-fw-load` should not be part of the enabled features:

```
DISTRO_FEATURES:append = " llce-can-multihost"
```

To complete the process, the resulting BSP image must be copied onto the SD card. Additionally, the M7 sample application should also be placed onto the FAT partition. Once this is done, the system should be booted until the U-Boot shell is reached. From this environment, the M7 sample application and the `canfdtest` must be loaded and started using the following steps:

1. Enable all LLCE clocks

```
=> scmi_clk gate llce 1
## Setting llce_can_pe clock ...
## Setting llce_sys clock ...
```

2. Init SRAM and load the M7 sample application in DDR

```
=> dcache off; mw.q 0x34000000 0x0 0x100000; dcache on
=> fatload mmc 0:1 ${loadaddr} /can_demo_linux.elf
```

3. Parse the sample ELF, copy program sections and start the M7 core using the address of the VTABLE symbol

```
=> bootm7 ${loadaddr} VTABLE
## Loading ELF from 0x80000000 ...
## Starting CM7_0 core using 0x34500400 ...
```

4. Instruct U-Boot not to reset SCMI config, preventing M7 hang.

```
=> setenv skip_scmi_reset_agent 1
```

5. Prepare a device tree fixup to remove all CAN instances not controlled by Linux and set the correct HIF ID before U-Boot hands over to Linux.

```
=> setenv llce_rm_m7can '
> for i in 0 1 2 3 4 5 6 7 8 9 10 12 13 14 15
> do
>     fdt rm /soc/llce@43a00000/llce_can${i}
> done'
=> setenv llce_en_mhif 'fdt set /soc/llce@43a00000/llce_mb@43a00000  nxp,multi-hif'
=> setenv llce_set_hif_id 'fdt set /soc/llce@43a00000/llce_mb@43a00000  nxp,hif-id <1>'
=> setenv fdt_fixups 'fdt addr ${fdt_addr};
> run llce_rm_m7can;
> run llce_en_mhif;
> run llce_set_hif_id'
```

6. Continue the boot process until the Linux shell is reached.

```
=> boot
```

7. Configure the `llcecan11` interface using 1 Mbit/s rate and 4 Mbit/s data rate, ensuring that these settings match those configured for the M7 endpoint.

```
root@s32g274ardb2:~# ip link set up llcecan11 type can bitrate 1000000 dbitrate 4000000 fd on
```

8. Start `canfdtest` on `llcecan11` in Linux shell

```
root@s32g274ardb2:~# canfdtest -f 1 -v -g llcecan11 -l 5000
interface = llcecan11, family = 29, type = 3, proto = 1
...................
Test messages sent and received: 5000
Exiting...
```

12.3.4 Statistics

LLCE CAN driver offers a set of statistics through `ip` and `ethtool` tools. Using `ip`, one can check the RX/TX counters and the number of CAN protocol errors.

```
root@s32g274ardb2:~# ip -details -statistics link show llcecan1
8: llcecan1: <NOARP,UP,LOWER_UP,ECHO> mtu 72 qdisc pfifo_fast state UP mode DEFAULT group
    default qlen 1000
    link/can  promiscuity 0 minmtu 0 maxmtu 0
    can <FD> state STOPPED restart-ms 1
          bitrate 500000 sample-point 0.875
          tq 12 prop-seg 69 phase-seg1 70 phase-seg2 20 sjw 1
          llce_can: tseg1 4..256 tseg2 4..128 sjw 1..128 brp 1..512 brp-inc 1
          dbitrate 5000000 dsample-point 0.750
          dtq 12 dprop-seg 5 dphase-seg1 6 dphase-seg2 4 dsjw 1
          llce_can: dtseg1 2..32 dtseg2 2..16 dsjw 1..16 dbrp 1..32 dbrp-inc 1
          clock 80000000
          re-started bus-errors arbit-lost error-warn error-pass bus-off
          0          2116       0          0          0          0
          numtxqueues 1 numrxqueues 1 gso_max_size 65536 gso_max_segs 65535
    RX: bytes  packets  errors  dropped overrun mcast
    0          0        0       0       0       0
    TX: bytes  packets  errors  dropped carrier collsns
    7936       125      2300    0       0       0
```

A broader set of statistics and errors is reported using `ethtool`:

```
root@s32g274ardb2:~# ethtool -S llcecan1
NIC statistics:
     general_rx: 2054
     general_tx: 2054
     can2can_in: 0
     can2can_out: 0
     can2eth: 0
     eth2can: 0
     txack_fifo_full: 0
     rxout_fifo_full: 0
     code_reserved_0: 0
     hw_fifo_full: 0
     sw_fifo_empty: 0
     sw_fifo_full: 0
     mb_notavailable: 0
     short_mb_notavailable: 0
     bcan_frz_exit: 0
     bcan_sync: 0
     bcan_frz_enter: 0
     bcan_lpm_exit: 0
     bcan_srt_enter: 0
     bcan_unknown_error: 0
     bcan_ackerr: 47013
     bcan_crcerr: 0
     bcan_bit0err: 2013
     bcan_bit1err: 5
     ...
```

12.3.4.1 Fault detection counters

Each core under the control of the LLCE firmware has a set of counters that are periodically updated to detect potential hang-ups or deadlocks in the LLCE subsystem. Information from these counters can be accessed through `debugfs` files, which can be used to create additional tools. Here is an example of how the counters can be inspected:

```
root@s32g274ardb2:~# cat /sys/kernel/debug/llce_core/core1_ts \
                        /sys/kernel/debug/llce_core/core2_ts \
                        /sys/kernel/debug/llce_core/core3_ts
0xe372e6ec
0xe373eb4e
0xe374218c
```

12.3.5 Filters and Routing

The Linux kernel has dedicated mechanisms for CAN frames routing and filtering. Typically, frames routing and filtering are performed through software and may require extra CPU cycles. On the other hand, the LLCE accelerator includes filters and destinations that can be used to offload CAN gateway tasks. In this way, the LLCE can route CAN frames according to the configured routing table without spending any host CPU cycles. This chapter describes how LLCE CAN filters and destinations are used from a Linux environment. It complements the LLCE firmware user's guide and does not detail all the used LLCE firmware interfaces.
Routing and filtering of CAN frames in LLCE are done in two steps:

1. A destination must be created
2. A filter that matches the CAN frames must be added to the routing table.

There are dedicated commands for both of the above-mentioned operations. Both of them are controlled using writes in debugfs files. `/sys/kernel/debug/llce_can_core` is the root folder containing `README` files for each step of this procedure and can be located using the following:

```
root@s32g274ardb2:~# find /sys/kernel/debug/llce_can_core/ -name README
```

The format and order of commands needed to create a destination, add a filter, or references to LLCE firmware structures and fields can be found in these `README` files.

12.3.5.1 CAN2CAN demo

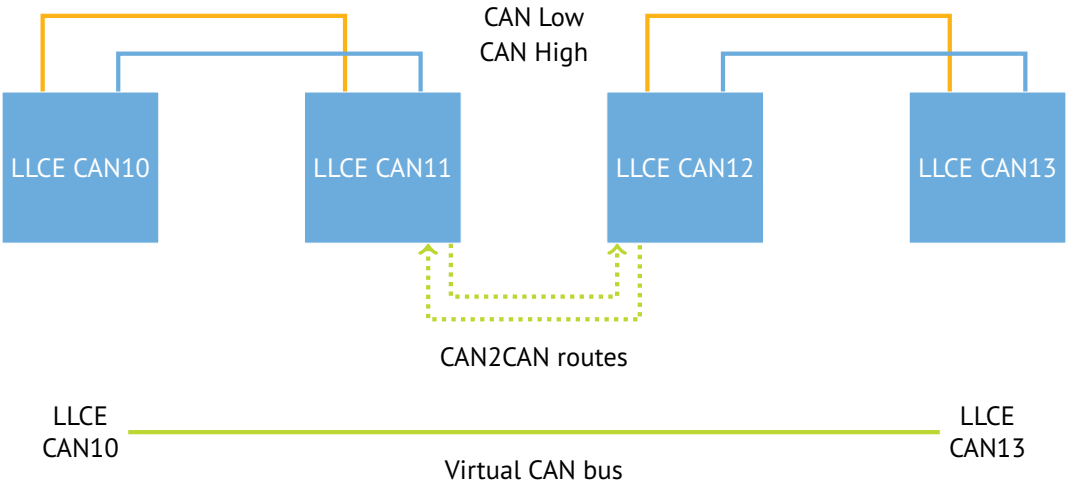


Figure 2: CAN2CAN demo

A CAN routing scenario can be observed in Figure 2. Four interfaces are involved in this use case organized in two pairs where the nodes of each pair are connected through a physically conventional two-wire bus. To establish communication between the LLCE CAN10 and LLCE CAN13 interfaces, we need to ensure that frames with ID `0x77` are available to LLCE CAN13 and frames with ID `0x78` are available to the LLCE CAN11 interface. Although these nodes are part of two different CAN buses, we can still achieve this goal by configuring the appropriate filters and destinations on both nodes. The desired setup can be easily verified using the following commands:

```
# LLCE CAN13 will reply with 0x78 to incoming frames with ID 0x77
canfdtest -v -i 0x77 -o 0x78 -x llcecan13 &
# LLCE CAN10 will start the generation of the frames with ID 0x77
canfdtest -v -i 0x77 -o 0x78 -g -x -l 10 llcecan10
```

To successfully transfer the required CAN frames, a specific set of CAN2CAN routes must be established between LLCE CAN11 and LLCE CAN12 interfaces. Here are the steps to be performed in Linux to have an equivalent of the above-described setup:

1. All four interfaces must be configured. This example uses a 500 kbit/s arbitration bitrate and a 5 Mbit/s data bitrate. The configuration of the interfaces will add LLCE CAN filters to direct all incoming frames to the HIF. Added filters can be inspected by listing the content of `/sys/kernel/debug/llce_can_core/can_filters/list`.

   ```
   root@s32g274ardb2:~# ip link set up llcecan10 type can bitrate 500000 dbitrate 5000000 fd on
   root@s32g274ardb2:~# ip link set up llcecan11 type can bitrate 500000 dbitrate 5000000 fd on
   root@s32g274ardb2:~# ip link set up llcecan12 type can bitrate 500000 dbitrate 5000000 fd on
   root@s32g274ardb2:~# ip link set up llcecan13 type can bitrate 500000 dbitrate 5000000 fd on
   ```

2. Disable the added host traffic filters applied to the LLCE CAN 11 and LLCE CAN 12 interfaces, as this demo will only focus on the routing capabilities.

   ```
   filters_list="/sys/kernel/debug/llce_can_core/can_filters/list"
   filters="$(awk '$1 == "11" || $1 == "12"  {print $7}' "$filters_list")"

   echo disabled > /sys/kernel/debug/llce_can_core/can_filters/set_status/status
   for filter_addr in $filters
   do
           echo $filter_addr > /sys/kernel/debug/llce_can_core/can_filters/set_status/addr
           echo 1 > /sys/kernel/debug/llce_can_core/can_filters/set_status/execute
   done
   ```

3. Create a destination to be used for LLCE CAN11→LLCE CAN12 route.

   ```
   # ID remapping is disabled
   echo 0x0 > /sys/kernel/debug/llce_can_core/can_destinations/add/can_id_remap
   # Unicast routing to LLCE CAN12
   echo 12 > /sys/kernel/debug/llce_can_core/can_destinations/add/ch_list
   # Enable routing for non-converting frames (See LLCE_CAN_ROUTING_NOCHANGE)
   echo 0x1 > /sys/kernel/debug/llce_can_core/can_destinations/add/routing_opts
   # Send the prepared destination to firmware
   echo 1 > /sys/kernel/debug/llce_can_core/can_destinations/add/execute
   ```

4. Add a new routing rule (CAN filter) to LLCE CAN11 for incoming frames with ID `0x77` to be delivered according to the previously created destination.

```
# Reference the newly created destination
cat /sys/kernel/debug/llce_can_core/can_destinations/add/execute > \
    /sys/kernel/debug/llce_can_core/can_filters/add/can_dest
# This field is ignored when using exact matching
echo 0x0 > /sys/kernel/debug/llce_can_core/can_filters/add/can_mask_id
# ID exact matching is used
echo exact > /sys/kernel/debug/llce_can_core/can_filters/add/entry
# The filter will be added to LLCE CAN11
echo 11 > /sys/kernel/debug/llce_can_core/can_filters/add/hw_ctrl
# No user defined tag is used
echo 0x0 > /sys/kernel/debug/llce_can_core/can_filters/add/tag
# 16 Maximum number of message buffers which can be used to store
# frames accepted by this filter at each specific point in time
echo 0x10 > /sys/kernel/debug/llce_can_core/can_filters/add/mb_count
# CAN FD frames will be used
echo long > /sys/kernel/debug/llce_can_core/can_filters/add/mb_type
# The ID of the frames to be routed
echo 0x77 > /sys/kernel/debug/llce_can_core/can_filters/add/msg_id
# Enable the filter after adding it
echo enabled > /sys/kernel/debug/llce_can_core/can_filters/add/status
# No authentication
echo disabled > /sys/kernel/debug/llce_can_core/can_filters/add/opt_auth
# No custom processing
echo disabled > /sys/kernel/debug/llce_can_core/can_filters/add/opt_custom
# The HIF is not supposed to receive the frames matching this filter
echo disabled > /sys/kernel/debug/llce_can_core/can_filters/add/opt_host
# No logging
echo disabled > /sys/kernel/debug/llce_can_core/can_filters/add/logging
# Send the prepared filter to firmware
echo 1 > /sys/kernel/debug/llce_can_core/can_filters/add/execute
```

5. Create a destination to be used for LLCE CAN12→LLCE CAN11 route. Benefit from the fact that all destination fields can be reused from the previous command and only update the channel field.

```
echo 11 > /sys/kernel/debug/llce_can_core/can_destinations/add/ch_list
echo 1 > /sys/kernel/debug/llce_can_core/can_destinations/add/execute
```

6. Add a new routing rule (CAN filter) to LLCE CAN12 for incoming frames with ID `0x78` to be delivered according to the previously created destination.

```
cat /sys/kernel/debug/llce_can_core/can_destinations/add/execute > \
    /sys/kernel/debug/llce_can_core/can_filters/add/can_dest
echo 12 > /sys/kernel/debug/llce_can_core/can_filters/add/hw_ctrl
echo 0x78 > /sys/kernel/debug/llce_can_core/can_filters/add/msg_id
echo 1 > /sys/kernel/debug/llce_can_core/can_filters/add/execute
```

7. Inspect the list of added destinations and filters

```
root@s32g274ardb2:~# cat /sys/kernel/debug/llce_can_core/can_filters/list
hw_ctrl    status can_mask_id     msg_id     tag mb_count  addr rx_intf entry mb_type | ... can_dest
     10  enabled  0x00000000 0x00000000 0x000a   0x0010   512       5  mask    long |
     11 disabled  0x00000000 0x00000000 0x000b   0x0010   513       4  mask    long |
     12 disabled  0x00000000 0x00000000 0x000c   0x0010   514       3  mask    long |
     13  enabled  0x00000000 0x00000000 0x000d   0x0010   515       2  mask    long |
     11  enabled  0x00000000 0x00000077 0x0000   0x0010     0       4 exact    long | ... 0
     12  enabled  0x00000000 0x00000078 0x0000   0x0010     1       3 exact    long | ... 1
```

```
root@s32g274ardb2:~# cat /sys/kernel/debug/llce_can_core/can_destinations/list
 id routing_opts can_id_remap ch_list
  0   0x00000001   0x00000000 12
  1   0x00000001   0x00000000 11
```

As it can be seen, new sets of filters and destinations have been added. It is also important to note the values from `addr` column for the new filters: 0 and 1. These are lower than the addresses of the filters added for host traffic (see 1 step). LLCE iterates through the list of enabled filters in ascending order of their addresses for each incoming frame. For this particular setup, it will check if a frame with ID `0x77` or `0x78` has to be routed before checking if it should be sent to the HIF. Therefore, the routed frames will never reach the HIF. As a result, the `candump` log will not contain frames with ID `0x77` reported on LLCE CAN11.

8. Finally, check that the communication between LLCE CAN11 and LLCE CAN13 interfaces works as expected.

```
root@s32g274ardb2:~# canfdtest -v -i 0x77 -o 0x78 -x llcecan13 &
root@s32g274ardb2:~# canfdtest -v -i 0x77 -o 0x78 -g -x -l 10 llcecan10
interface = llcecan10, family = 29, type = 3, proto = 1

Test messages sent and received: 10
Exiting...
```

12.3.6 Advanced parameters

The CAN interfaces are typically set up to distribute firmware resources in an equal manner by default. However, there may be instances where an uneven allocation of resources is required. In such cases, a range of parameters can be accessed through the device tree properties and `devlink` interface to achieve the desired outcome. Refer to the table below for the parameter name, default value and location.

Parameter	Description
`devlink` parameters	
`self-recv`	Enables self-reception mode **Default value:** false **LLCE FW Reference:** `LLCE_CAN_CONTROLLERCONFIG_SRX_EN_U32`
`host-rx-mb`	The number of transmission message buffer descriptors used for transmissions initialized by the host. **Default value:** 10 **LLCE FW Reference:** `Llce_Can_InitCmdType.u8TxMbCount`
Device tree parameters	
`nxp,max_regular_filters`	The maximum number of regular filters for an interface. **Default value:** 16 **LLCE FW Reference:** `Llce_Can_InitPlatformCmdType.au16MaxRegularFilterCount`
`nxp,max_adv_filters`	The maximum number of advanced filters for an interface. **Default value:** 16 **LLCE FW Reference:** `Llce_Can_InitPlatformCmdType.au16MaxAdvancedFilterCount`
`nxp,max_rx_mb`	The maximum number of RX message buffers for an interface. **Default value:** 100 **LLCE FW Reference:** `Llce_Can_InitPlatformCmdType.au16MaxIntMbCount`

nxp,max_tx_ack	The maximum number of reserved TX confirmation buffers for an interface. **Default value:** 16 **LLCE FW Reference:** Llce_Can_InitPlatformCmdType.au16MaxIntTxAckCount

12.3.6.1 devlink Parameters

All available device parameters can be listed using:

```
root@s32g274ardb2:~# devlink dev param
```

The value of a specific LLCE CAN parameter can be inspected using:

```
root@s32g274ardb2:~# devlink dev param show <device> name <param-name>
```

For example:

```
root@s32g274ardb2:~# devlink dev param show  platform/43ff8000.llce:llce_can0 name self-recv
platform/43ff8000.llce:llce_can0:
  name self-recv type driver-specific
    values:
      cmode runtime value false
```

The value of a CAN parameter can also be adjusted, but it will be applied during the next link-up operation. The following command should be used to accomplish this:

```
root@s32g274ardb2:~# devlink dev param set <device> \
        name <param-name> \
        value <value> \
        cmode runtime
```

For instance:

```
root@s32g274ardb2:~# devlink dev param set platform/43ff8000.llce:llce_can0 \
> name self-recv \
> value true \
> cmode runtime
```

12.3.6.2 Device Tree Parameters

Device tree parameters for LLCE CAN interfaces are passed to drivers through the properties mentioned above. The values of these properties can be adjusted by updating the board's device tree. This is an example of how the parameters of LLCE CAN0 should be updated:

```
&llce_can0 {
    nxp,max_regular_filters = <16>;
    nxp,max_adv_filters = <16>;
    nxp,max_rx_mb = <100>;
    nxp,max_tx_ack = <16>;
};
```

12.4 CAN Logger

The LLCE CAN Logger driver receives CAN frames and timestamps from LLCE hardware and logs them.

The driver is able to operate in two modes: along with Linux LLCE CAN driver or in scenarios where the LLCE CAN host management is part of an another software stack.

For the cases when the LLCE subsystem is managed by Linux, the following lines should be added in `<builddirectory>/conf/local.conf` file.

```
DISTRO_FEATURES:append = " llce-fw-load llce-can llce-can-logger"
NXP_FIRMWARE_LOCAL_DIR = "/path/to/firmware/binaries/folder"
```

For the cases when the LLCE subsystem is managed by other software stacks, the added line should be:

```
DISTRO_FEATURES:append = " llce-can-logger"
```

12.4.1 Using the Driver

Upon inserting the module, a set of virtual CAN (vcan) interfaces will be created. Each `llcelogger` interface will capture the traffic received on its associated hardware interface. For example, on `llcelogger12` will be logged the traffic received on `llcecan12`.



If the `llcelogger12` interface is down, the CAN frames are not read from the LLCE CAN Logger FIFO and the receipt of CAN frames on `llcecan12` will stall. To avoid the stalling of a LLCE CAN interface it is recommended to set up the corresponding LLCE CAN Logger interface.

```
root@s32g274ardb2:~# ip address show type can
2: can0: <NOARP,ECHO> mtu 16 qdisc noop state DOWN group default qlen 10
    link/can
3: can1: <NOARP,ECHO> mtu 16 qdisc noop state DOWN group default qlen 10
    link/can
6: llcelogger0: <NOARP> mtu 16 qdisc noop state DOWN group default qlen 10
    link/can
7: llcelogger1: <NOARP> mtu 16 qdisc noop state DOWN group default qlen 10
    link/can
8: llcelogger2: <NOARP> mtu 16 qdisc noop state DOWN group default qlen 10
    link/can
...
```

By default, all LLCE Logger interfaces are in down state and not marked as virtual CAN interfaces. This can be changed using:

```
root@s32g274ardb2:~# ip link set up llcelogger0 type vcan
```

The logs along with the hardware timestamp can be inspected using:

```
root@s32g274ardb2:~#  candump -t a -H llcelogger0,0:0,#FFFFFFFF
  (2535568793.000000)  llcelogger0  029   [8]  01 02 03 04 05 06 07 08)
  (2535605966.000000)  llcelogger0  029   [8]  02 03 04 05 06 07 08 09
  (2535643006.000000)  llcelogger0  029   [8]  03 04 05 06 07 08 09 0A
  (2535680046.000000)  llcelogger0  029   [8]  04 05 06 07 08 09 0A 0B
  (2535717086.000000)  llcelogger0  029   [8]  05 06 07 08 09 0A 0B 0C
  (2535754260.000000)  llcelogger0  029   [8]  06 07 08 09 0A 0B 0C 0D
```

The first column from above output contains timestamps in '(seconds.nanoseconds)' format. In case of the LLCE Logger, the `seconds` field represents the number of cycles reported by LLCE. In above example '(2535568793.000000)' corresponds to 0x9721B599 cycles.

12.5 LINFlexD

The LINFlexD LLCE firmware has four independent interfaces, one for every channel. Linux BSP can configure LINFlexD channels to be used in UART mode exclusively. One important limitation is that LLCE LINFlexD can be used as a serial console once the LLCE mailbox driver has been loaded, typically after reaching the Linux console. Additionally, the LLCE LINFlexD channels operate in interrupt mode and no DMA can be used. Taking this into account, in some scenarios, characters may be lost, "copy-paste" is an example.

In order to add support for LLCE LINFlexD, the following line should be added in `<builddirectory>/conf/local.conf` file:

```
DISTRO_FEATURES:append = " llce-fw-load llce-linflex"
NXP_FIRMWARE_LOCAL_DIR = "/path/to/firmware/binaries/folder"
```



If the LLCE subsystem is managed by other software stacks, `llce-fw-load` should be omitted.

In manual compilation scenario, `CONFIG_LLCE_CORE` and `CONFIG_NXP_LLCE_MBOX` should be enabled as modules.

For more details, please check Linux documentation:
Documentation/devicetree/bindings/serial/fsl,s32-linflexuart.yaml

12.6 LPSPI

In order to add support for LLCE LPSPI, the following line should be added in `<builddirectory>/conf/local.conf` file:

```
DISTRO_FEATURES:append = " llce-fw-load llce-lpspi"
NXP_FIRMWARE_LOCAL_DIR = "/path/to/firmware/binaries/folder"
```



If the LLCE subsystem is managed by other software stacks, `llce-fw-load` should be omitted.

In manual compilation scenario, `CONFIG_LLCE_CORE` and `CONFIG_NXP_LLCE_MBOX` should be enabled as modules.

Linux LPSPI driver supports full duplex transfers in both controller and target mode. By default, the LPSPI is configured to run in controller mode. For target mode the following code snipped:

```
#address-cells = <1>;
#size-cells = <0>;
spidev5: spidev@0 {
        compatible = "rohm,dh2228fv";
        spi-max-frequency = <4000000>;
        reg = <0>;
        fsl,spi-cs-sck-delay = <100>;
        fsl,spi-sck-cs-delay = <100>;
};
```

should be replaced with:

```
#address-cells = <0>;
#size-cells = <0>;
spi-slave;
slave {
        compatible = "rohm,dh2228fv";
};
```

`nxp,lpspi-slave-no-pin-swap` property is relevant in LPSPI target mode case. If **not** used, the driver configures `SOUT` to be used for input data and `SIN` to be used for output data. If the property is added to the device tree node, `SOUT` and `SIN` will not be interchanged.

For more details, please check the Linux documentation:
Documentation/devicetree/bindings/spi/spi-fsl-lpspi.yaml



`LPSPI_2` can be used on `S32G-PROCEVB3-S` if the following changes are done:

- modify jumper `J169` in the 3-5, 4-6 position
- disable `can2` node in device tree because of pinmuxing overlapping (`PB_12`)
- disable `pwm1` node in device tree because of pinmuxing overlapping (`PB_13`)
- enable `llce_spi2` node in device tree

13 CAN Information

This chapter describes the steps to configure CAN and lists basic examples for CAN message transmission.

With respect to using FlexCAN on S32G2 EVB together with S32GRV-PLATEVB, first make sure that you have the following jumpers on the required position: J166 on 1-3 and 2-4, J155 on 1-3 and 2-4, J169 on 1-3 and 2-4.
The mapping of the hardware pins of the CAN interface to the userspace interface is as follows:

- FlexCAN_0 has the hardware interface J79 on PROCEVB associated to `can0` in userspace.
- FlexCAN_2 has the hardware interface J19 on PLATEVB associated to `can1` in userspace.
- FlexCAN_3 has the hardware interface J20 on PLATEVB associated to `can2` in userspace.



For S32G274A RDB2 only `can0` and `can1` interfaces are available.

13.0.1 Example of sending a CAN message

- Set the CAN interfaces:
  - Set the bitrate of can0 and can1 interfaces to 500kbps:

```
ip link set can0 type can bitrate 500000
ip link set can1 type can bitrate 500000
```

  - Start the can interfaces like a standard net interface:

```
ip link set can0 up
ip link set can1 up
```

- Example of testing can1 transmission:

```
cansend can0 011#22335566778899
```

- Example of testing can0 reception:

```
candump can1
  can1  123   [8]  0A AA AA AA BB BB BB BB
  can1  123   [8]  0A AA AA AA BB BB BB BB
  can1  123   [8]  0A AA AA AA BB BB BB BB
  can1  123   [8]  0A AA AA AA BB BB BB BB
  can1  123   [8]  0A AA AA AA BB BB BB BB
  can1  123   [8]  0A AA AA AA BB BB BB BB
  can1  123   [8]  0A AA AA AA BB BB BB BB
```

14 SD/eMMC high speed modes support

14.1 SD/eMMC high speed modes support

High speed modes such as HS200/HS400/HS400ES for the eMMC or SDR50/DDR50 for the SD card are only supported if the memory runs at 1.8V. Before enabling this option, the board must be configured accordingly. On S32G-PROCEVB-S, setting `J110` and `J23` on 1-2, switches the eMMC memory to 1.8V. Afterwards, the following steps can be used to enable the modes listed above.



Do not use the steps below if the memory that the controller talks to (i.e. as defined by the position of the J50 jumper on S32G-PROCEVB-S) is not actually supplied at 1.8V, as it may lead to undefined behavior.
The HS400 mode has only been validated on the S32G-PROCEVB-S board.
SDR50/DDR50 modes might not work in some isolated cases due to particularities of the SD cards on the general market.

14.1.1 Enabling high speed modes in U-Boot

Go to the board specific device-tree in TF-A (e.g. fdts/s32g2xxa-evb.dts) and amend the usdhc node as follows:

```
&usdhc0 {
    /delete-property/ no-1-8-v;
};
```

If building with yocto, see chapter Building the Linux BSP using YOCTO for more information about patching various components.

14.1.2 Enabling high speed modes in Linux

Go to the board specific device-tree and amend the usdhc node as follows:

```
&usdhc0 {
    /delete-property/ no-1-8-v;
};
```

If building with yocto, see chapter Building the Linux BSP using YOCTO for more information about patching various components.

Alternatively, you can enable it at runtime, from the U-Boot environment. For example, executing the following command at the U-Boot prompt will enable HS400/ES in Linux. Executing the 'saveenv' command makes the changes persistent across multiple boots.

```
=> setenv fdt_override "run fdt_enable_hs400; ${fdt_override}"
=> saveenv
```



In both U-Boot and Linux, eMMC high speed modes can also be enabled using `EMMC_FEATURES` that can be added to `<builddirectory>/conf/local.conf` file:

```
EMMC_FEATURES:append = " high-speed"
```

14.2 eMMC HS400 Enhanced Strobe support

The HS400ES mode is selected automatically if HS400 is enabled and the connected memory supports HS400ES.



None of the supported platforms has an eMMC which supports HS400ES. This feature has been validated on a modified S32G-PROCEVB-S board with a different memory.

14.3 eMMC Performance enhancements

In certain conditions the eMMC performance throughput can be increased by enabling the CQE feature.

CQE should improve the read performance but its stability cannot be guaranteed. CQE enablement can be achieved using `EMMC_FEATURES` variable that can be added to `<builddirectory>/conf/local.conf` file:

```
EMMC_FEATURES:append = " cqe"
```

Alternatively, CQE support can also be enabled in the Linux Kernel by directly modifying the drivers/mmc/host/sdhci-esdhc-imx.c file, as follows:

```
static struct esdhc_soc_data usdhc_s32cc_data = {
        .flags = ESDHC_FLAG_USDHC | ESDHC_FLAG_MAN_TUNING
                        | ESDHC_FLAG_HAVE_CAP1 | ESDHC_FLAG_HS200
                        | ESDHC_FLAG_HS400 | ESDHC_FLAG_HS400_ES
+                       | ESDHC_FLAG_CQHCI
                        | ESDHC_FLAG_SKIP_ERR004536
                        | ESDHC_FLAG_BROKEN_SDR104
                        | ESDHC_FLAG_SKIP_CD_WAKE,
};
```



CQE feature is only available in the Linux Kernel.



CQE has a dedicated errata chapter, which is located at the following section.

14.4 'mmc_test' Kernel Driver Usage and Enablement

The Linux kernel contains drivers/mmc/core/mmc_test.c upstream development driver, which performs a series of reads and writes to a memory card in order to expose certain well-known bugs in host controllers. The tests are executed by writing to the "test" file in debugfs under each card.

'mmc_test' can be enabled within Linux BSP by usage of 'bsp-utils' DISTRO_FEATURE. This step requires the following line to be added in `<builddirectory>/conf/local.conf` file.

```
DISTRO_FEATURES:append = " bsp-utils"
```

Please note that in order to successfully use 'mmc_test' kernel module driver, QSPI Flash boot is required. Hence the flashimage (fsl-image-flash) should be used.

When booting from the QSPI Flash, under `/opt/bsp_utils/` there will be a `s32cc_mmc_test_tool.sh` bash script which can be used in order to ease the usage of 'mmc_test'.

Below is an example usage of the `s32cc_mmc_test_tool.sh` bash script.

```
root@s32g274aevb:/opt/bsp_utils# ./s32cc_mmc_test_tool.sh
[  103.866008] mmc_test mmc0:0001: Card claimed for testing.

Please select one of the following test cases for eMMC
0:      Run all tests
1:      Basic write (no data verification)
2:      Basic read (no data verification)
3:      Basic write (with data verification)
...
50:     Commands during non-blocking read - use Set Block Count (CMD23)
51:     Commands during non-blocking write - use Set Block Count (CMD23)

Enter test case number:
3
[   71.532821] mmc0: Starting tests of card mmc0:0001...
[   71.538492] mmc0: Test case 3. Basic write (with data verification)...
[   71.650341] mmc0: Result: OK

[   71.754077] mmc0: Tests completed.

Want to run another test? (y/n)
y
Enter test case number:
4
[   94.808784] mmc0: Starting tests of card mmc0:0001...
[   94.814462] mmc0: Test case 4. Basic read (with data verification)...
[   94.920812] mmc0: Result: OK

[   95.023915] mmc0: Tests completed.

Want to run another test? (y/n)
n
root@s32g274aevb:/opt/bsp_utils#
```



Whatever is on the SD card or eMMC memory will be overwritten by some of those tests.

15 Temperature Monitoring Unit

15.1 Reading Temperatures from the Thermal Monitoring Unit

The Thermal Monitoring Unit (TMU) allows reading the immediate SoC temperatures from three different sites.

The simplest way to access the temperatures is to use the `lm-sensors` package (available for the `fsl-image-auto` image).

For example:

```
root@s32g274aevb:~# sensors
a53_cores-virtual-0
Adapter: Virtual device
temp1:        +35.9 C  (crit = +100.0 C)

llce-virtual-0
Adapter: Virtual device
temp1:        +35.4 C

ddr_sram-virtual-0
Adapter: Virtual device
temp1:        +35.9 C
```

TMU supports thermal zone control based on Linux kernel thermal management interface which is described in more detail in:

- Documentation/devicetree/bindings/thermal/thermal-zones.yaml
- Documentation/devicetree/bindings/thermal/thermal-cooling-devices.yaml

The description of each site is added in the device tree under `thermal-zones` node. Given the fact that TMU does not support defining thresholds per site, interrupts can be used only for one thermal zone. For the others, polling should be used.

The `A53 cores` immediate temperature is monitored using interrupts. Two trip points are defined for it. The first one is triggered when the temperature reaches 80°C. When this threshold is hit, the Cortex-A53 cores frequency will be lowered. This capability is available only if Dynamic Frequency Scaling for the A53 cores feature is enabled. See Dynamic Frequency Scaling for more details. The default thermal governor is `step wise` which throttles the CPU one step at a time. The second trip point is critical and will trigger a system shutdown at 100°C. The other thermal zones are defined in the device tree, without any trip point definitions. In this case, the Linux kernel will not monitor them.

The temperature reading for the sites monitored using polling is not instantaneous and may require some time. Due to the hardware module limitations, the monitoring for those sites is off, so their temperature is not read by the TMU. When the temperature is requested, the Linux driver will change the TMU configuration to be able to read it. The time needed for a single temperature read of a polled site is determined based on `Central Module Configuration (TCMCFG)` register value [4]. The TMU interrupts are disabled during the temperature read operation of a polled site.



TMU has a dedicated errata chapter, which is located here.

[4] The delay is ≃8 ms for the current settings

16 SoC Level Time Source

16.1 General Information

This module provides a global timestamp which can be accessed from the other cores of the SoC, so that events on different CPUs of the SoC can be globally ordered.

For this purpose, we use a System Timer Module (STM), 32-bits long. This type of timer has the advantage that its registers can be accessed at SoC level, hence providing global access. The input clock signal for the timer has the frequency of ~133.33 MHz. It can also be obtained from */sys/kernel/debug/clk/stm_module/clk_rate*. In order to obtain rarer overflows, a clock divider of 256 was used. This results in a resolution of ~1.9 μs and an overflow period of ~137 min.

16.2 Kernel Configuration

In order to obtain the module for this driver you need to set CONFIG_NXP_GLOBAL_TIME_STM=m in `s32cc_defconfig`
You need to activate the STM2 in device tree. This can be done by simply adding the following lines to arch/arm64/boot/dts/freescale/s32cc.dtsi before compiling.

```
&stm2 {
        status = "okay";
};
```

Refer to subsection Patch the kernel to apply the above changes.
Generate the new Yocto image by rerunning `bitbake <imagename>` and deploy it.

16.3 Userspace Interface

The STM can be used after inserting */lib/modules/6.6.32-rt/kernel/drivers/clocksource/nxp_global_time.ko*.
This driver exposes two sysfs entries as userspace interface:

- */sys/devices/platform/soc/40124000.stm/value* - the current timestamp
- */sys/devices/platform/soc/40124000.stm/init* - start/stop timer command/status

16.4 Supported commands

```
Starting timer:         echo 1 > /sys/devices/platform/soc/40124000.stm/init
Reading timer value:    cat /sys/devices/platform/soc/40124000.stm/value
Stopping timer:         echo 0 > /sys/devices/platform/soc/40124000.stm/init
```

16.5 Accessing Time Source and Control from Outside Linux

In order to access the driver from outside Linux (say from real-time cores or the hardware accelerators) you need to read the corresponding counting register (CNT) and read/write the corresponding enable bit (TEN bit in CR register) of STM2 (System Timer Module 2), as described in the Reference Manual for S32G.

17 Switch the Linux System Timer from Generic Timer to PIT timer

The Linux kernel can switch the timers used for clocksource and clockevents as base. For a proper events timming/system timming, a timer should be assigned per core. S32G2 has only two PIT timers and two STM timers.

To enable the PIT/STM, you will have to follow the procedure:

- Select which core will use the PIT/STM timer.
  In arch/arm64/boot/dts/freescale/s32cc.dtsi can be found one entry for each PIT/STM timer:

```
cpus {
        #address-cells = <2>;
        #size-cells = <0>;

        pit0: pit@40188000{
                compatible = "fsl,s32cc-pit";
                reg = <0x0 0x40188000 0x0 0x3000>;
                interrupts= <0 53 4>;
                clocks = <&clks S32CC_SCMI_CLK_PIT_MODULE>;
                clock-names = "pit";
                cpu = <0>;
                status = "disabled";
        };
```

`cpu` field indicates which virtual cpu will use the timer module. Any value between 0 and NR_CPUS can be used.



The design of the STM module makes it unsuitable as clocksource so it is recommended not to be used for the master core.

- Enable PIT from menuconfig:
  Location:
  -> Platform selection
  -> Freescale S32 family
  -> PIT timer used as system timer
  -> PIT used as system timer
- Enable STM from menuconfig:
  Location:
  -> Platform selection
  -> Freescale S32 family
  -> STM timer used as system timer
  -> STM used as system timer

18 SPI Target Support

18.1 SPI Target Support in Linux

The DSPI controller can be configured to run in target mode if the `spi-slave` property is added in the SPI node as in the following example:

```
&spi1 {
        pinctrl-0 = <&pinctrl0_dspi1>;
        pinctrl-names = "default";
        #address-cells = <0>;
        #size-cells = <0>;
        spi-slave;
        status = "okay";

        slave {
                compatible = "rohm,dh2228fv";
        };
};
```

Use `rohm,dh2228fv` compatible for binding the spidev driver to the device. Information on spidev can be found in Documentation/spi/spidev.rst.

The target will run in DMA mode otherwise the driver is too slow to feed TX FIFO and underflow occurs frequently. DMA can work only with 8 and 16 bits per word. 32bits per word is not supported.

The target supports a maximum of 32767 words per transfer (this is the maximum that eDMA can transfer at once). For more than 32767 words, the driver splits the message into several DMA transfers and underrun can occur between them.

19 SAR-ADC Driver

19.1 Using the SAR-ADC in U-Boot

The Linux BSP U-Boot driver allows the user to operate the on-chip 12-bit analog-to-digital converter (SAR-ADC) in single and multi-channel conversion mode.

The SAR-ADC modules can be probed in U-Boot by running:

```
=> adc list
- adc@401f8000
- adc@402e8000
```



The above listed module names have the following correspondence:

adc@401f8000	saradc0
adc@402e8000	saradc1

1. One-shot single channel conversion mode:

```
=> adc single adc@401f8000 0
0
=> adc single adc@401f8000 1
2518
=> adc single adc@401f8000 3
4095
```

2. One-shot multi channel conversion mode:

```
=> adc scan adc@401f8000
[00]: 1
[01]: 2523
[02]: 1985
[03]: 4095
[04]: 2180
[05]: 2135
[06]: 2038
[07]: 2130
```

In order to obtain the voltage in millivolts, this value must be multiplied with the scale of value: 0.439453125. For example, for channel 1, the result is 2518 x 0.439453125 ~ 1106.54 mV.

19.2 Using the SAR-ADC in Linux

The Linux BSP Linux driver allows the user to operate the on-chip 12-bit analog-to-digital converter (SAR-ADC) through an IIO (Industrial Input/Output) driver.

Linux BSP supports three modes of operation for the SAR-ADC:

1. One-shot conversions in normal mode

   The files needed for reading converted values from the eight external ADC channels are located in the sysfs attributes directory of the IIO device. The directory path is `/sys/bus/iio/devices/iio:deviceX` where `X`

is the IIO index and has a value between 0 and 255. In case of S32CC platforms, most probably, there will be two IIO devices with index 0 and 1. If you see more directories, find the one corresponding to the ADC by reading the name file located inside:

```
$ cat /sys/bus/iio/devices/iio:device0/name
401f8000.adc
$ cat /sys/bus/iio/devices/iio:device1/name
402e8000.adc
```

Connect a DC power supply between the ADC ground and one of the first 8 pins. The voltage applied must have a value between 0 and 1.8V. Read the `in_voltageX_raw file`, where `X` is the channel number. For example:

```
$ cat /sys/bus/iio/devices/iio:device0/in_voltage0_raw
2738
```

In order to obtain the voltage in millivolts, this value must be multiplied with the scale in `in_voltage_scale`

```
cat /sys/bus/iio/devices/iio:device0/in_voltage_scale
0.439453125
```

In this case, the result is 2738 x 0.439453125 ~ 1203.22 mV.

2. Software triggered mode

   In software-triggered mode, the SAR_ADC acquires one sample from each enabled channel and pushes it into the IIO buffer whenever the software trigger fires.
   The Linux Kernel has several software triggers already implemented:

   - `IIO_SYSFS_TRIGGER` - fires when `1` is written to `/sys/bus/iio/devices/trigger0/trigger_now`
   - `IIO_INTERRUPT_TRIGGER` - fires when the assigned interrupt is triggered
   - `IIO_HRTIMER_TRIGGER` - fires when the hrtimer overflows
   - `IIO_TIGHTLOOP_TRIGGER` - fires continuously

   How to use the ADC with sysfs trigger:

   (a) Create a new sysfs trigger:

   ```
   $ echo 0 > /sys/bus/iio/devices/iio_sysfs_trigger/add_trigger
   ```

   (b) Bind device with the newly created trigger:

   ```
   $ cat /sys/bus/iio/devices/trigger0/name > \
           /sys/bus/iio/devices/iio:device0/trigger/current_trigger
   ```

   (c) Enable the channels in use (any channel combination can be enabled in software triggered mode):

   ```
   $ echo 1 > /sys/bus/iio/devices/iio:device0/scan_elements/in_voltage0_en
   $ echo 1 > /sys/bus/iio/devices/iio:device0/scan_elements/in_voltage3_en
   $ echo 1 > /sys/bus/iio/devices/iio:device0/scan_elements/in_voltage7_en
   ```

   (d) Set the buffer length, enable the buffer, fire the trigger and print the sample:

```
$ echo 100 > /sys/bus/iio/devices/iio:device0/buffer/length #optional
$ echo 1 > /sys/bus/iio/devices/iio:device0/buffer/enable
$ echo 1 > /sys/bus/iio/devices/trigger0/trigger_now
$ hexdump -e '"iio0 :" 1/2 "%04x " "\n"' /dev/iio:device0
```

3. Scan (continuous) conversions in normal mode

   In this mode, the SAR_ADC will sample all enabled channels in order, from the enabled channel with the lowest ID to the enabled channel with the highest ID. The SAR_ADC sample rate is 1 Msps. The sample rate of one channel is equal to `1/num_of_enabled_channels` Msps.

   To read the ADC samples at 1 Msps without affecting the system performance, the SAR_ADC Linux driver uses DMA. The DMA integration limits the available combinations of SAR_ADC channels to:

   - one channel at a time
   - 0 1
   - 2 3
   - 4 5
   - 6 7
   - 0 1 2 3
   - 4 5 6 7
   - 0 1 2 3 4 5 6 7

   The IIO buffer size should be greater than 4096 (DMA buffer size) to make sure that no samples are lost.
   To read SAR_ADC data continuously the user needs to set the buffer length, enable the used channels and the buffer.

   The following steps are a setup example for reading data from channels 0 and 1 in scan (continuous) mode, from SAR_ADC0 device:

   (a) Enable the channels in use:

   ```
   $ echo 1 > /sys/bus/iio/devices/iio:device0/scan_elements/in_voltage0_en
   $ echo 1 > /sys/bus/iio/devices/iio:device0/scan_elements/in_voltage1_en
   ```

   (b) The next step is to enable the buffer and set up a buffer length. It can be done manually, or using the `iio_generic_buffer` userspace application, located under kernel sources at tools/iio/iio_generic_buffer.c

      i. Enable buffer and read data manually:

      ```
      $ echo 4096 > /sys/bus/iio/devices/iio:device0/buffer/length
      $ echo 1 > /sys/bus/iio/devices/iio:device0/buffer/enable
      $ hexdump -e '"iio0 :" 8/2 "%04x " "\n"' /dev/iio:device0 | head -20
      ```

      ADC data can be read from `/dev/iio:device0`. In the above example, only the first 20 lines are printed to the console. (20 read operations from channels 0 and 1). Data read from `/dev/iio:device0` must be multiplied with the scale in `in_voltage_scale`, in order to obtain the voltage in millivolts.

      If you enable and read the adc data manually, you should disable buffer mode when you finish using the adc device, as follows:

```
$ echo 0 > /sys/bus/iio/devices/iio:device0/buffer/enable
```



The data capture can be re-enabled at any time later by writing value `1` to `/sys/bus/iio/devices/iio:device0/buffer/enable`

ii. Using the iio_generic_buffer application:
The `iio_generic_buffer` application can be compiled as follows:

```
$ make ARCH=arm64 -C <kernel-src-dir>/tools/iio
```

The `iio_generic_buffer` application performs all the ADC channel `enable` and `disable` actions for you.

```
root@s32g274aevb:~# ./iio_generic_buffer -N 0 -g -l 4096
iio device number being used is 0
iio trigger-less mode selected
0.439453 1799.560547
0.439453 1799.560547
0.439453 1799.560547
0.439453 1799.560547
0.439453 1799.560547
0.439453 1799.560547
0.878906 1799.560547
0.439453 1799.560547
0.439453 1799.560547
0.439453 1799.560547
0.439453 1799.560547
0.439453 1799.560547
0.000000 1799.560547
0.439453 1799.560547
```

In the above example, `-N` is the device number (0 -> SAR_ADC0 or 1 -> SAR_ADC1), `-g` is the trigger-less mode and `-l` is the buffer length.



If you want to use all SAR_ADC channels, please do not use '-a' parameter of the `iio_generic_buffer` application because it will also try to enable the timestamp soft channel, which is not the desired behaviour. Instead, you should enable all of SAR_ADC channels manually from `scan_elements/in_voltageX_en`.
By using `iio_generic_buffer` there is no need to multiply the output values with the scale in `in_voltage_scale`.

Check the Documentation/ABI/testing/sysfs-bus-iio file inside the Linux source code for a full reference of the standard IIO device attributes. Currently, the driver only supports buffer triggered and buffer software modes, by performing one-shot conversion when the trigger fires or continuous conversions in normal mode for one or multiple channels at a time.



On S32G274A RDB2 on each SAR_ADC channel there is a resistor divider circuit, which increases the S32G274A RDB2 voltage detection range (from 0-1.8V to 0-3.6V). Therefore, on S32G274A RDB2 platform, the SAR_ADC Linux driver outputs the voltage read after the resistor divider circuit, which is half of the voltage applied.
Channel 5 of ADC0 is not available for use on S32G274A RDB2 platform. This is due to channel 5 of ADC0 being used to determine which RDB board revision is in use.

20 Controlling GPIOs in Linux

20.1 About libgpiod

The libgpiod library allows users to write userspace applications that interact with GPIOs. The sysfs interface is now deprecated starting with kernel version 4.8. Therefore, new designs should use the libgpiod library. The library works by using the GPIO character device and it converts the API calls into ioctl calls [5] to the GPIO character device. The term GPIO chip has the same meaning as the GPIO character device. The library can be used to do the following:

- request input/output GPIOs
- set output/read input GPIO value
- enable interrupts for a GPIO
- configure GPIO bias i.e. pull-up, pull-down

Moreover, the Yocto libgpiod-tools package provides the following tools [6]:

gpiodetect	list available GPIOs
gpioinfo	list details of GPIO lines
gpioget	read the value for the requested GPIO lines
gpioset	set the value for the requested GPIO lines
gpiofind	find the name of the GPIO chip and the offset of GPIO line given the line name
gpiomon	wait for interrupts on the specified GPIO lines



This section refers to requesting GPIO pins in userspace. In order to request GPIO pins for use inside the kernel see: https://www.kernel.org/doc/html/latest/driver-api/gpio/board.html.

20.2 Using libgpiod-tools

20.2.1 gpiodetect

This tool is used to identify the GPIO chips available and the number of lines (i.e. GPIOs) they expose.

```
root@s32g274aevb:~# gpiodetect
gpiochip0 [...siul2-gpio0] (... lines)
```

20.2.2 gpioinfo

This tool is used to print out information about a GPIO chip's GPIO lines i.e. whether they are used or not, input or output, active-high or active low.

```
root@s32g274aevb:~# gpioinfo gpiochip0
gpiochip0 - ... lines:
        line   0:      "PA_00"       unused   input  active-high
        line   1:      "PA_01"       unused   input  active-high
        line   2:      "PA_02"       unused   input  active-high
        [...]
```

[5] Check out `man ioctl` for more details.
[6] More details can be found at https://git.kernel.org/pub/scm/libs/libgpiod/libgpiod.git.

20.2.3 gpioget

This tool is used to sample the values of the given GPIOs for a GPIO chip. It also allows the configuration of a pin bias and active-low operation. The following are some examples of its usage (Note that the GPIO pin is floating in the examples):

```
root@s32g274aevb:~# gpioget --bias=pull-up gpiochip0 27
1
root@s32g274aevb:~# gpioget --bias=pull-down gpiochip0 27
0
root@s32g274aevb:~# gpioget --active-low --bias=pull-down gpiochip0 27
1
```

20.2.4 gpioset

This tool is used to set the value of the given GPIOs for a GPIO chip. This command allows the user to configure pin properties such as: `--active-low` operation, `--bias` and `--drive`.

The user also has the option to specify via the `--mode` parameter what happens after setting the GPIOs' values. Note that after the program quits, the corresponding pad will revert to the previous settings.

A few examples:

```
# set it to 0V and wait for an Enter
root@s32g274aevb:~# gpioset --mode=wait --active-low 0 27=1
# set it to 0V and wait 3s
root@s32g274aevb:~# gpioset --mode=time  --sec=3 0 27=0
# set it to 0V and wait for SIGINT
root@s32g274aevb:~# gpioset --mode=signal --background 0 27=0
root@s32g274aevb:~# kill -INT $(pgrep gpioset)
```

20.2.5 gpiofind

This tool is used to return the GPIO chip's name and line offset given a GPIO name.

```
root@s32g274aevb:~# gpiofind PA_00
gpiochip0 0
```

20.2.6 gpiomon

This tool is used to wait for the specified number of interrupts (events) on the given GPIOs of a GPIO chip. It allows the user to wait for both fronts or only `--rising-edge`/`--falling-edge`. It can also configure the `--bias`.

A few examples:

```
# wait for 3 rising-edge interrupts
root@s32g274aevb:~# gpiomon --num-events=3 --rising-edge 0 27
# record 100 interrupts for GPIO27 and set its bias to pull-up
# print the seconds from timestamp and interrupt edge
root@s32g274aevb:~# gpiomon --bias=pull-up --num-events=100 --format="%s %e" 0 27
```

20.3 Devicetree Example of using a GPIO external interrupt (EIRQ)

On S32G2 SoC, an EIRQ can be mapped in some cases to more GPIOs. Therefore, the GPIO number is being used in order to know the exact EIRQ, since there isn't a case where a GPIO can have more EIRQs attached to it.

The following example illustrates the way the EIRQ associated to GPIO 22 is attached to a driver which is bounded to a device tree node having 'nxp,eirq' compatible string.

```
nxpeirqs: nxpeirqs-node {
        interrupt-parent = <&gpio>;
        interrupts = <22 IRQ_TYPE_EDGE_FALLING>;
        compatible = "nxp,eirq";
        status = "okay";
};
```

21 Clocking

21.1 Read clock frequency at runtime

To be able to read the clock frequency `CONFIG_DEBUG_FS` must be enabled in the kernel configuration. By default, it is enabled on the supported platforms.

The clock frequency can be obtained from */sys/kernel/debug/clk/a53/clk_rate* path. To read it, the following Linux command can be run:

```
cat /sys/kernel/debug/clk/a53/clk_rate
```

22 Setting up SIUL2 pads

The SIUL2 driver contains definitions for a restricted number of pads. If your driver needs additional PADS to be defined the following steps must be followed:

- in file drivers/pinctrl/freescale/pinctrl-s32g.c update:
  - `enum s32_pins` with entries that have the format `<PAD_NAME>= <MSCR_ID>`
  - the `s32_pinctrl_pads_siul2` array of `struct pinctrl_pin_desc`, for SIUL2_0 and SIUL2_1, with entries having the format:
    `S32_PINCTRL_PIN(<PAD_NAME_DEFINED_IN_s32_pins>)`
    Refer to the section "Register configuration" in the SoC Reference Manual for the mapping of MSCR/IMCR registers to SIUL2_0 and SIUL2_1.
- update the dts by adding a group for your driver: [7]

```
&pinctrl {
        status = "okay";

        <new_driver_pins> {
                <new_driver_grp0>: <new_driver_pins0> {
                        pinmux = <S32CC_PINMUX(<pin0>, <FUNC1>)>,
                                 <S32CC_PINMUX(<pin1>, <FUNC2>);
                        output-enable;
                        slew-rate = <S32CC_SLEW_150MHZ>;
                };

                <new_driver_grp1>: <new_driver_pins1> {
                        pinmux = <S32CC_PINMUX(<pin0>, <FUNC1>)>;
                        input-enable;
                        bias-pull-up;
                };
        };
}
```

- refer the group of pins in the dts entry for your driver

```
<new_driver> {
        pinctrl-names = "default";
        pinctrl-0 = <&<new_driver_pins>> [, ...];
        status = "okay";
};
```

[7] More details about the generic pinmux/pinconf syntax can be found at: Documentation/devicetree/bindings/pinctrl/pincfg-node.yaml and Documentation/devicetree/bindings/pinctrl/pinmux-node.yaml

23 Power Management

This chapter covers the power management features supported by BSP 42.0. Power state changes are typically requested by Linux (kernel or user-space) and are executed by the Secure Monitor component of the Arm Trusted Firmware on behalf of the non-secure world.

23.1 Platform Reset

The SoC can be software reset by typing

```
reboot
```

at the Linux command prompt.

23.2 CPU Hotplug

BSP 42.0 supports manually removing or adding individual CPUs via the command-line. The following command-line snippet unplugs, then plugs CPU1. The *nproc* command gives the number of online CPUs at a given moment:

```
root@s32g274aevb:~# nproc
4
root@s32g274aevb:~# echo 0 > /sys/devices/system/cpu/cpu1/online
[   14.405099] CPU1: shutdown
[   14.405115] psci: CPU1 killed (polled 0 ms)
root@s32g274aevb:~# nproc
3
root@s32g274aevb:~# echo 1 > /sys/devices/system/cpu/cpu1/online
[   64.068743] Detected VIPT I-cache on CPU1
[   64.068792] GICv3: CPU1: found redistributor 1 region 0:0x00000000508a0000
[   64.068871] CPU1: Booted secondary processor 0x0000000001 [0x410fd034]
root@s32g274aevb:~# nproc
4
```

23.3 Suspend to RAM

Suspend to RAM, or STR, is a complex power management feature supported by BSP 42.0. STR is a high-level concept, implemented by the Linux kernel and the TF-A, and underlaid by the S32G SoC's Standby state.

STR saves the software context and transitions the SoC to Standby state, which is an SoC low power state (not to be confounded with the VR5510 PMIC's Standby and Deep Sleep modes!). For details on the S32G hardware support for this feature, please refer to the Power Management chapters in the S32G SoC Reference Manual.

STR software puts LPDDR in self-refresh mode and all but the essential hardware blocks are powered off. A wakeup source must be configured prior to suspend. Suspend to RAM can be triggered from the Linux command-line as described below.

On executing the system suspend command on behalf of the Linux kernel, the TF-A Secure Monitor (see Arm Trusted Firmware) performs the necessary DDR and SoC configurations, then programs the chip for the Run→Standby transition. The SoC will remain in Standby until a wakeup source is triggered.

After a wakeup source is triggered, the SoC executes the Standby→Run transition and TF-A is loaded back into memory(as it does during a cold boot). During the BL2 setup stage, a check is made to find out the reset cause. In case it was a resume from Standby, the DDR and A53 clocks are enabled, the DDR is taken out of the self-refresh mode and execution continues into BL31, skipping a part of the initialization that is made during a cold boot. After that, execution will eventually resume into Linux.

While the TF-A saves and restores the core and DDR context, peripherals are generally expected to be dealt with in the Linux suspend/resume code. The kernel has a core framework for STR, and peripheral specifics are the responsibility of each Linux driver.

Because of the characteristics of the Standby→Run transition, the TF-A resume code is to a large extent common with the cold boot code. Note, however, that U-Boot is *not* involved in the Suspend/Resume sequence; only the TF-A is.



When booting from eMMC, the default speed mode settings (see the "Boards" subsection) might not permit the board to resume from Standby before the PMIC's watchdog is triggered, due to prolonged storage access times. In this scenario, the eMMC speed settings can be changed to a higher mode using the `SW6[4:7]` DIP switch for the S32G274A RDB2 board. For more information regarding the supported eMMC speed modes, refer to the S32G2 Reference Manual.

23.3.1 Wakeup Sources

The S32G2 SoC supports multiple wakeup sources. External sources can be GPIO pins, while the Real-Time Clock (RTC) can be configured as an internal source. The TF-A by default configures RTC and one external wakeup source. Actually configuring these sources to trigger a wakeup event is the responsibility of the caller of STR functionality (e.g. Linux).

23.3.2 Suspend to RAM with RTC as Wakeup Source

In order to use the RTC as a wakeup source, one needs to:

1. Preprogram the RTC module to trigger an interrupt at a later point in time
2. Put the platform into Standby

The simplest way of performing these steps is via the standard Linux *rtcwake* command, which configures one of the available RTC drivers (default */dev/rtc0*), then requests the kernel to execute a platform suspend. The Linux RTC driver will appropriately configure the RTC device, then will ask the TF-A to perform the transition to Standby.

The following code snippet showcases the use of *rtcwake* for STR:

```
s32g274ardb2 login: root
root@s32g274ardb2:~# echo;date;echo; rtcwake -d rtc0 -m mem -s 180; echo;date

Fri Nov 26 01:43:51 UTC 2021

rtcwake: assuming RTC uses UTC ...
rtcwake: wakeup from "mem" using rtc0 at Thu Jan  1 00:13:09 1970
[  460.260205] PM: suspend entry (deep)
[  460.260286] Filesystems sync: 0.000 seconds
[  460.260761] Freezing user space processes ... (elapsed 0.007 seconds) done.
[  460.268581] OOM killer disabled.
[  460.268585] Freezing remaining freezable tasks ... (elapsed 0.006 seconds) done.
[  460.275512] printk: Suspending console(s) (use no_console_suspend to debug)
[  460.294060] pci_bus 000b:01: busn_res: [bus 01-ff] is released
[  460.294268] pci_bus 000b:00: busn_res: [bus 00-ff] is released
[  460.310919] fsl_fccu 4030c000.fccu: FCCU status is 0 (normal)
[  460.312278] s32cc-dwmac 4033c000.ethernet: phy mode set to RGMII
[   21.016003] phy-s32cc-serdes 40480000.serdes: Using mode 0 for SerDes subsystem
[   21.017513] phy-s32cc-serdes 44180000.serdes: Using mode 3 for SerDes subsystem
[   21.051121] phy-s32cc-serdes 44180000.serdes: Unstable RX detected on XPCS1
[   21.051141] phy-s32cc-serdes 44180000.serdes: Unstable RX detected on XPCS0
[   21.252346] s32cc-pcie 40400000.pcie: Failed to stabilize PHY link
[   21.252353] s32cc-pcie 40400000.pcie: Configuring as RootComplex
[   22.206829] s32cc-pcie 40400000.pcie: Phy link never came up
[   22.206921] pci_bus 0001:00: PM: parent pci0001:00 should not be sleeping
```

```
[   22.206987] s32cc-pcie 40400000.pcie: PCI host bridge to bus 0001:00
[   22.206995] pci_bus 0001:00: root bus resource [bus 00-ff]
[   22.207002] pci_bus 0001:00: root bus resource [io  0x0000-0xffff]
[   22.207009] pci_bus 0001:00: root bus resource [mem 0x5800000000-0x5fffdfffff]
(bus address [0x00000000-0x7fffdfffff])
[   22.207058] pci 0001:00:00.0: [16c3:4300] type 01 class 0x060400
[   22.207080] pci 0001:00:00.0: reg 0x10: [mem 0x5800000000-0x58000fffff]
[   22.207098] pci 0001:00:00.0: reg 0x38: [mem 0x5800000000-0x580000ffff pref]
[   22.207192] pci 0001:00:00.0: supports D1
[   22.207197] pci 0001:00:00.0: PME# supported from D0 D1 D3hot D3cold
[   22.207328] pci 0001:00:00.0: PM: parent pci0001:00 should not be sleeping
[   22.212445] pci 0001:00:00.0: BAR 0: assigned [mem 0x5800000000-0x58000fffff]
[   22.212457] pci 0001:00:00.0: BAR 6: assigned [mem 0x5800100000-0x580010ffff pref]
[   22.212466] pci 0001:00:00.0: PCI bridge to [bus 01-ff]
[   22.228913] OOM killer enabled.
[   22.228916] Restarting tasks ...
[   22.229315] pcieport 0001:00:00.0: PME: Signaling with IRQ 94
[   22.230415] done.
[   22.278893] PM: suspend exit

Fri Nov 26 01:46:55 UTC 2021

root@s32g274ardb2:~#
```

The above command uses */dev/rtc0* , which is the S32G RTC device. That being the default, the *-d rtc0* argument might be omitted. It specifies the type of low-power transition (*mem* means "suspend to RAM") and the number of seconds before the wakeup interrupt (here, *180* seconds). The platform goes to sleep, then after the specified time it wakes on RTC input and resumes execution.

Note that the kernel timestamps printed during suspend and resume do not reflect the amount of time the platform has actually been sleeping. This is because the Arm Generic Timer, which is used as System Timer in Linux, is not running while the SoC is in Standby, and so it cannot count the time elapsed while suspended. The system date, however, correctly reflects the time slept.

23.3.3 Suspend to RAM with GPIO as Wakeup Source

In order to use an external GPIO pin as a wakeup source, one needs to make the following steps:

1. Enable a GPIO pin as an external wakeup source
   The TF-A must be configured via the device-tree to use a GPIO as an external wakeup source. Before building and deploying the TF-A binaries, the `nxp,irqs` property in the `wkpu` node of the fdts/s32cc.dtsi device tree should be configured as in the following example:

```
wkpu: wkpu@40090000 {
        compatible = "nxp,s32gen1-wkpu";
        reg = <0x0 0x40090000 0x0 0x10000>, /* WKPU */
                      <0x0 0x4007cb04 0x0 0x4>; /* S32G_STDBY_GPR */
        /*
        * Enable GPIO as wakeup source
        */
        nxp,irqs = <S32GEN1_WKPU_EXT_IRQ(16)
                      S32GEN1_WKPU_IRQ_RISING
                      S32GEN1_WKPU_PULL_DIS>;

        nxp,warm-boot = <S32GEN1_WKPU_LONG_BOOT>;

        status = "disabled";
};
```

In the above example the external wakeup interrupt 16 is set using the `S32GEN1_WKPU_EXT_IRQ()` macro. The WKPU unit supports up to 23 external wakeup interrupts numbered from 0 to 22. By default there is no external wakeup interrupt set.

2. Disable the RTC module

```
echo disabled > /sys/bus/platform/devices/40060000.rtc/power/wakeup}
```

This is important: if the RTC is set as a wakeup source (default behavior) and it is not configured, the Linux Kernel power management core will error out.

3. Perform the suspend to RAM transition

```
echo mem > /sys/power/state
```

This will leave the platform in Standby state indefinitely. In order to manually wake the platform up, one needs to apply 3V3 voltage to the configured GPIO pin. This specific process is outside the scope of this document.

23.4 Dynamic Frequency Scaling

BSP 42.0 supports Dynamic Frequency Scaling (DFS) for the A53 cores, meaning that the clock frequency of the CPUs can be adjusted on the fly in order to save energy.

CPU frequency scaling is achieved using the `cpufreq` subsystem [8] offered by the Linux Kernel. Requests to change the clock rate are forwarded through SCMI calls to the Secure Monitor component of TF-A, which handles the actual frequency change operation.

DFS enablement depends on the following configuration options which are activated by default:

```
CONFIG_CPU_FREQ=y
CONFIG_ARM_SCMI_CPUFREQ=y
```

The `cpufreq` subsystem provides a list of CPU governors, each governor implementing a specific frequency scaling algorithm.



The default scaling governor in Linux BSP is `performance`. This sets the frequency statically to the highest frequency supported by the CPU.

A scaling governor relevant for power management is called `schedutil`. It adjusts the clock rate according to the utilization metrics from the scheduler. Manually changing the frequency is only possible when using the `userspace` governor, which can be selected by enabling the *CONFIG_CPU_FREQ_GOV_USERSPACE* option [9].

From `menuconfig`, one can change the default governor and choose the scaling governors available on the system, under:

-> CPU Power Management

-> CPU Frequency scaling

Make sure that the *SCMI based CPUfreq driver* option (corresponding to *CONFIG_ARM_SCMI_CPUFREQ*) remains selected after doing this step. Option *CONFIG_ARM_SCMI_CPUFREQ* can be configured as built-in or module. If the module option was selected, the corresponding kernel module needs to be manually loaded at runtime.

[8] https://www.kernel.org/doc/html/latest/admin-guide/pm/cpufreq.html

[9] Refer to the official Linux Kernel documentation for a full description of the scaling governors



The DFS configuration can be done in two ways: through the sysfs filesystem or by using the `cpufrequtils` package installed on `fsl-image-auto`. Regardless of the approach, all CPUs are organized under the same cpufreq policy object. This implies that they share the same configuration interface, so any change made to one CPU will impact the others.

23.4.1 Via sysfs filesystem

Go to the sysfs directory associated with this feature:

```
root@s32g274aevb:~# cd /sys/devices/system/cpu/cpufreq/policy0
```

To see the current clock frequency of the A53 cores run:

```
root@s32g274aevb:/sys/devices/system/cpu/cpufreq/policy0# cat cpuinfo_cur_freq
1000000
```



Frequency values are expressed in kHz.

To list the available clock frequencies use:

```
root@s32g274aevb:/sys/devices/system/cpu/cpufreq/policy0# cat scaling_available_frequencies
48780 55555 62500 71428 86956 105263 133333 200000 333333 1000000
```

To see the current governor use:

```
root@s32g274aevb:/sys/devices/system/cpu/cpufreq/policy0# cat scaling_governor
performance
```

To get the list of the available governors in the system use:

```
root@s32g274aevb:/sys/devices/system/cpu/cpufreq/policy0# cat scaling_available_governors
schedutil performance
```

To set a new governor run the following command, where `<governor>` is one of the available governors from `scaling_available_governors` attribute:

```
root@s32g274aevb:/sys/devices/system/cpu/cpufreq/policy0# echo <governor> > scaling_governor
```

To manually set a specific frequency (only for the `userspace` governor) use the following command, where `<frequency>` is one of the available values from `scaling_available_frequencies` attribute:

```
root@s32g274aevb:/sys/devices/system/cpu/cpufreq/policy0# echo <frequency> > scaling_setspeed
```

23.4.2 Via cpufrequtils package

To see the current clock frequency, available frequency steps, current governor and available governors of the A53 cores run:

```
root@s32g274aevb:~# cpufreq-info
```

To set a new governor run the following command, where `<governor>` is one of the available governors from `cpufreq-info` command output:

```
root@s32g274aevb:~# cpufreq-set -g <governor>
```

To manually set a specific frequency (only for the `userspace` governor) run the following command, using one of the available values from `cpufreq-info` command output:

```
root@s32g274aevb:~# cpufreq-set -f 371Mhz
```

To manually set a minimum frequency that governor can use run the following command, using one of the available values from `cpufreq-info` command output:

```
root@s32g274aevb:~# cpufreq-set -d 200Mhz
```

To manually set a maximum frequency that governor can use run the following command, using one of the available values from `cpufreq-info` command output:

```
root@s32g274aevb:~# cpufreq-set -u 1.3Ghz
```



When using frequency values from `cpufreq-info` command as input to `cpufreq-set` command, the space between the last digit and the unit of measurement needs to be removed.

24 Virtualization

24.1 Linux Containers User Guide

Linux Containers is a lightweight virtualization technology that allows the creation of environments in Linux called "containers" in which Linux applications can be run in isolation from the rest of the system and with fine grained control over resources allocated to the container (e.g. CPU, memory, network).

LXC is a user space package that provides a set of commands to create and manage containers and uses existing Linux kernel features to accomplish the desired isolation and control.

Why are containers useful?

Below are a few examples of container use cases:

- Application partitioning – control CPU utilization between high priority and low priority applications, control what resources applications can access.
- Virtual private server – boot multiple instances of user space, each which effectively looks like a private instance of a server. This approach is commonly used in website infrastructure.
- Software upgrade – run Linux user space in a container, when it becomes necessary to upgrade applications in the system, create and test upgraded software in a new container. The old container can be stopped and the new container can be started as desired.
- Terminal servers – user accesses the system with a thin client, with containers on the server providing applications. Each user gets a private, sandboxed workspace.

There are two general usage models for containers:

- Application containers – Running a single application in a container. In this scenario, a single executable program is started in the container.
- System containers – Booting an instance of user space in a container. Booting multiple system containers allows multiple isolated instances of user space to run at the same time.

Containers are conceptually different than virtual machine technologies such as QEMU/KVM. Virtual machines emulate a hardware platform and are capable of booting an operating system kernel. A container is a mechanism to isolate Linux applications. In a system using containers there is only one Linux kernel running – the host Linux kernel.

24.1.1 Build, Installation and Configuration

24.1.1.1 Building with Yocto

LXC is a Linux user space package that can easily be added to a root filesystem using the Yocto build system. Update the DISTRO_FEATURES:append variable in the *conf/local.conf* file in the Yocto build environment as follows:

```
DISTRO_FEATURES:append = " lxc"
```

Then bitbake the image:

```
bitbake fsl-image-auto
```

24.1.1.2 Update the root filesystem so the system is ready to support containers

In order to use containers, mount the 'cgroup' pseudo-filesystem:

```
root@s32g274aevb:~# mount -t cgroup cgroups /sys/fs/cgroup
```

24.1.2 LXC How To's (Getting Started with a BusyBox System Container)

The following article describes steps to run a simple container example. All the commands below are issued on a target running Linux.

1. Confirm that your kernel environment is configured correctly using lxc-checkconfig

```
root@s32g274aevb:~# lxc-checkconfig
---Namespaces---
Namespaces: enabled
Utsname namespace: enabled
Ipc namespace: enabled
Pid namespace: enabled
User namespace: enabled
Network namespace: enabled
Multiple /dev/pts instances: enabled
---Control groups---
Cgroup: enabled
Cgroup namespace: required
Cgroup device: enabled
Cgroup sched: enabled
Cgroup cpu account: enabled
Cgroup memory controller: missing
Cgroup cpuset: enabled
---Misc---
Veth pair device: enabled
Macvlan: enabled
Vlan: enabled
File capabilities: enabled
```



Before booting a new kernel, you can check its configuration using:

```
CONFIG=/path/to/config/usr/bin/lxc-checkconfig
```

If the cgroup namespace option shows as required:

```
Cgroup namespace: required
```

The */cgroup* directory needs to be created and/or mounted.
See 24.1.1.2

2. Create a container

   Create a system container using lxc-create and specify the BusyBox template and *lxc-empty-netns.conf* config file. *lxc-empty-netns.conf* is a simple config file with no networking:

```
root@s32g274aevb:~# lxc-create -n foo -t busybox -f /usr/share/doc/lxc/examples/lxc-empty-netns.conf
/home/root
```

3. List containers that exist

```
root@s32g274aevb:~# lxc-ls -f
NAME STATE   AUTOSTART GROUPS IPV4 IPV6 UNPRIVILEGED
foo  STOPPED 0         -      -    -    false
```

4. Modify lxc.mount.auto parameter

   Specify which standard kernel file system should be automatically mounted. The file systems are:

proc:mixed (or proc)	mount */proc* as read-write, but remount */proc/sys* and */proc/sysrq-trigger* read-only for security / container isolation purposes
proc:rw	mount */proc* as read-write
sys:mixed	mount */sys* as read-only but with */sys/devices/virtual/net* writable
sys:ro	mount */sys* as read-only for security / container isolation purposes
sys:rw	mount */sys* as read-write



In order to have access rights to the file system, the configuration file of the container */var/lib/lxc/container_name/config* should have the lxc.mount.auto parameter **proc:rw**

```
root@s32g274aevb:~# cat /var/lib/lxc/foo/config
# Template used to create this container: /usr/share/lxc/templates/lxc-busybox
# Parameters passed to the template: examples/lxc-empty-netns.conf
# For additional config options, please look at lxc.container.conf(5)
lxc.rootfs = /var/lib/lxc/foo/rootfs
lxc.haltsignal = SIGUSR1
lxc.rebootsignal = SIGTERM
lxc.utsname = foo
lxc.tty = 1
lxc.pts = 1
lxc.cap.drop = sys_module mac_admin mac_override sys_time

# When using LXC with apparmor, uncomment the next line to run unconfined:
#lxc.aa_profile = unconfined
lxc.mount.entry = /lib lib none ro,bind 0 0
lxc.mount.entry = /usr/lib usr/lib none ro,bind 0 0
lxc.mount.entry = /sys/kernel/security sys/kernel/security none ro,bind,optional
lxc.mount.auto = proc:rw
```

5. How to start the container

   From a shell on the target, start the container.

```
root@s32g274aevb:~# lxc-start -n foo
```

If no error message is displayed, then the container was started successfully (as a daemon).

In order to actually use the container, connect to its console then login using user 'root' and an empty password:

```
root@s32g274aevb:~# lxc-console -n foo

Connected to tty 1
Type <Ctrl+a q> to exit the console, <Ctrl+a Ctrl+a> to enter Ctrl+a itself

foo login: root
~ #
```

Note that the shell is now running within the container. Normal Linux commands can be executed. To exit the container console, press <Ctrl+a q> as instructed.

Important notice: the container can be started in foreground, with argument '-F', in which case it will directly connect to one of its terminals:

```
root@s32g274aevb:~# lxc-start -n foo -F

init started: BusyBox v1.32.0 ()

Please press Enter to activate this console.
/ #
```

When prompted, press 'Enter'. However, this mode has a minor caveat: the terminal will be stuck in this container console until the container is halted (either from here, by running 'halt', or from another terminal by running lxc-stop).

6. List processes in the container
   From the container shell use the ps command to list processes:

```
~ # ps
  PID USER TIME COMMAND
    1 root 0:00 init
    6 root 0:00 /bin/syslogd
    8 root 0:00 /bin/getty -L tty1 115200 vt100
    9 root 0:00 /bin/sh
   10 root 0:00 ps
```

7. Stop the container (from a host shell)

```
root@s32g274aevb:~# lxc-stop -n foo
root@s32g274aevb:~# lxc-info -n foo
Name:       foo
State:      STOPPED
```

8. Destroy the container

```
root@s32g274aevb:~# lxc-destroy -n foo
```

24.2 Docker Containers

24.2.1 What is Docker?

Docker is a lightweight virtualization solution, which extends the capabilities of LXC. It is an Open-Source containerization technology which focuses on running applications in isolated environments.

Docker containers rely on Docker Engine, which is composed of a daemon process (dockerd), an API that applications can use to interact with the Docker daemon, and a CLI interface. The daemon process manages the underlying OS infrastructure to run the containers, which are based on templates named Docker Images.

24.2.2 Enabling Docker in Auto Linux BSP

To enable Docker in a Linux BSP build, the Kernel Image needs to be correctly configured to include all the required support, and the Docker Engine package needs to be installed into the root file system. This ensures basic Docker support for running containers.

For the Ubuntu target image, Docker support is included by default when building `fsl-image-ubuntu`. The next steps should only be done when enabling Docker in images based on Yocto root file system (e.g. `fsl-image-auto`).
The required Kernel support can be enabled in a Yocto build by adding the following line to the *conf/local.conf* in the build directory:

```
DISTRO_FEATURES:append = " docker"
```

Also, the Docker Container Engine should be installed into the built image, by adding the following line to the image recipe (e.g. meta-alb-sdk/recipes-fsl/images/fsl-image-auto.bb):

```
IMAGE_INSTALL:append = " docker-ce"
```



Depending on the size of the Docker Images that are fetched from Docker Repository, the available free space in root file system may not be enough and should be extended. For example, increasing the extra space to 2GB can be done by adding the following lines to the *conf/local.conf* in the build directory:

```
# Leave 2GB extra space
IMAGE_ROOTFS_EXTRA_SPACE = "2097152"
```

Build the *.sdcard* image, boot to Linux prompt, ensure that the board has Internet connectivity and the date is correctly set. Then, run the following command to test that Docker is working:

```
root@s32g274aevb:~# docker run hello-world
```

24.2.3 Docker Compose

Docker Compose is a Docker tool used to describe and run multi-container Docker applications. The configuration of the entire application is written in a YAML file, which should contain details about all the containers and what services they need from the Docker Engine. All the containers can be launched using the "`docker-compose up`" command.

To enable Docker Compose for images based on Yocto root file system, in addition to Docker Engine, also add the following line to the image recipe (e.g. meta-alb-sdk/recipes-fsl/images/fsl-image-auto.bb):

```
IMAGE_INSTALL:append = " python3-docker-compose"
```

For the Ubuntu target image (`fsl-image-ubuntu`), Docker Compose can be installed:

- before building the Ubuntu image, by adding the following line to the image recipe (e.g. meta-alb-sdk/recipes-fsl/images/fsl-image-ubuntu.bb):

  ```
  APTGET_EXTRA_PACKAGES += "docker-compose"
  ```

- after booting, by using the Ubuntu Package Manager:

  ```
  root@ubuntu-s32g274aevb:~# apt-get install docker-compose
  ```

In order to use Docker Compose, a multi-container application is needed. A useful resource is the official Docker Compose Getting Started Guide.

After getting all the necessary code (Python App, Dockerfile, requirements.txt and Docker Compose YAML) for the example application, the application can be built and run inside Docker Containers as following:

```
root@s32g274aevb:~# docker-compose up -d
```

24.3 Xen Hypervisor

24.3.1 What is Xen?

Xen is an Open-Source Type-1 (Bare-metal) hypervisor. Type-1 hypervisors run directly on the hardware, having one or more Operating Systems running on top of it. It is hosted by The Linux Foundation.

Xen on Arm supports Arm devices using the virtualization extensions of the Arm CPUs.

There are 2 types of Virtual Machines (VMs) that run on top of Xen:

- Privileged:
  - *Dom0* (the first virtual machine)
- Unprivileged:
  - *DomUs* (booted from Dom0 via the xl toolstack)

Xen virtualizes CPU, memory, interrupts and timers, providing virtual machines with virtualized resources. All hardware devices are assigned to Dom0 by default, that runs the native device drivers for them.

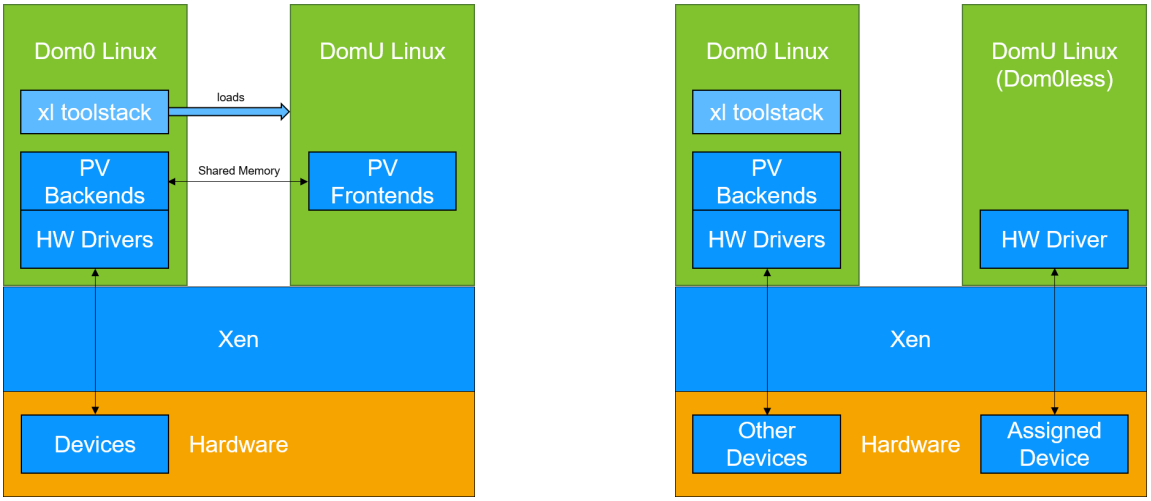
Disk, network, console and other I/O devices are shared between VMs using a paravirtualized backend/frontend approach. Dom0 runs all the backend drivers and services one or more frontends.

24.3.2 Dom0less Virtual Machines

Dom0less VMs are a special kind of DomUs, since they cannot be managed via *xl toolstack*, and therefore cannot benefit from any paravirtualized drivers. They have access to the devices that Xen virtualizes for them (CPU, memory, interrupts and timers), and also to an emulated vPL011 console.

Access to other peripherals can be granted using Device Passthrough, a virtualization technique for assigning various system peripherals to an unprivileged VM. S32 Platforms do not have an IOMMU and this adds some limitations regarding virtualization. For S32 Platforms, Dom0less VMs need to have 1:1 mapping from intermediate physical address (IPA) to physical address (PA) in order to support Device Passthrough.

The configuration for Dom0less VMs is done through Device Tree, making it easy to deploy a statically-configured system.



(a) Dom0 and DomU Configuration

(b) Dom0 and Dom0less DomU Configuration

Figure 3: Normal and Dom0less Use Cases

The difference between normal DomUs and Dom0less DomUs is illustrated in Figure 3. Subfigure 3a shows the interaction between Dom0 and a DomU VM. The DomU is loaded from Dom0 using *xl toolstack*, and DomU's access to hardware is granted by the paravirtualized backend-frontend drivers approach. [10]

In contrast, Subfigure 3b shows that a Dom0less DomU has no interaction with Dom0. Xen boots the Dom0less DomUs and directly assigns devices to them. Advantages of Dom0less DomUs are:

- Faster boot time (no need to wait for Dom0 to boot first)
- Static configuration (no need to dynamically configure anything after booting)

24.3.3 How to enable Xen in S32 Linux BSP build

Xen support can be enabled by adding the following line in the *conf/local.conf* file of your Yocto build directory:

```
DISTRO_FEATURES:append = " xen"
```

This will build Xen and all its dependencies, along with Xen tools (xl toolstack) that are needed to manage the DomUs from Dom0.

After building a *fsl-image-auto* or *fsl-image-base* image, the following files are deployed in the */boot* folder of the rootfs partitions:

[10] For platforms with an IOMMU, direct assignment of devices to normal DomUs is also available.

- Xen Image (*xen*)
- Linux Kernel Image (*Image*)
- Linux Device Tree binary (*fsl-<board_name>.dtb*)
- Boot script and its text source (*boot.scr*, *boot.source*)
- *optional:* Ramdisk image for Dom0less-DomUs (*fsl-image-dom0less-<board_name>.cpio.gz*)
- *optional:* Passthrough Device Tree binary (*passthrough.dtb*)
- xl toolstack (installed in the rootfs partitions, in */usr/sbin*)

The deployed boot script (*boot.scr*) is executed by U-Boot. It contains all the necessary commands for setting up the environment to boot Xen.

The optional files are deployed depending on the custom Xen setup chosen through the *XEN_EXAMPLES* variable. The setups are described in the section - Building Custom Xen Setups.



- The generated SDCard image has an additional partition that contains a root filesystem that can be used by a DomU.
- Another extra rootfs partition (i.e. for another DomU) can be enabled by adding the following lines to *conf/local.conf* file, and then *domU2_mmc.cfg* configuration file can be used to boot DomU2, as described in subchapter "Booting Xen, Dom0 and DomU":

```
SDCARD_ROOTFS_EXTRA2_FILE = "${SDCARD_ROOTFS_EXTRA1_FILE}"
SDCARD_ROOTFS_EXTRA2_SIZE = "${SDCARD_ROOTFS_EXTRA1_SIZE}"
```

After the build is complete, write the generated SDCard image as in an usual S32 BSP build.

24.3.4 Building Custom Xen Setups

With the increased complexity of setting up a statically-configured system, the user can also select a pre-configured Xen configuration, defined by the *xen-examples-*.bb* recipes within Yocto BSP layer. This can be done by setting the *XEN_EXAMPLE* variable in the *conf/local.conf* to one of the values mentioned in Table 12.

XEN_EXAMPLE value	Dom0	Dom0less-DomU1	Device Passthrough
"xen-examples-default"	512MB of RAM 1 vCPU RootFS on SD part.	-	-
"xen-examples-dom0less"		1024MB of RAM 1 vCPU RootFS on initrd	-
"xen-examples-dom0less-passthrough"			Manually configurable
"xen-examples-dom0less-passthrough-gmac"			GMAC controller assigned to DomU1
"xen-examples-dom0less-passthrough-pfe"	+ PFE Master controller		PFE Slave controller assigned to DomU1

Table 12: Xen Custom Setups

The default value for the *XEN_EXAMPLE* variable is *"xen-examples-default"*. This means that the default Xen setup is built in case the variable is not explicitly set.
For example, to build a basic setup with only Dom0 booting and no Dom0less VMs, the user can add the following to the *conf/local.conf* file:

```
XEN_EXAMPLE = "xen-examples-default"
```

For each value assigned to *XEN_EXAMPLE*, there is a corresponding recipe that is selected, built and deployed along with its runtime dependencies. These recipes can be found in meta-alb-bsp/recipes-extended/xen-examples and meta-alb-sdk/recipes-extended/xen-examples, and each has an associated config file, which contains the parameters of the system that can be changed.

All the recipes automatically build a complete Xen setup as per their description in Table 12. Automatically configured device passthrough for the GMAC controller is supported, through the *"xen-examples-dom0less-passthrough-gmac"* recipe. The *"xen-examples-dom0less-passthrough-pfe"* recipe automatically configures device passthrough for the PFE Slave controller. Refer to the PFE Linux Driver User Manual for more details.

24.3.5 Booting Xen, Dom0 and DomU

After having a complete image on the SDCard, the */boot* folder in each rootfs partition contains the items mentioned in section How to enable Xen in S32 Linux BSP build.

When the *boot* command is run in U-Boot, the boot script (*boot.scr*) is executed. This loads all the required files into memory and sets up the */chosen* node for providing the system settings to Xen. Then, U-Boot boots Xen (instead of the Linux Image), and Xen loads the Kernel Image. This is the image for Dom0. The same image is also used for Dom0less-DomUs, that boot in parallel with Dom0.

After Dom0 is up and running, the user can manage the DomUs from Dom0 userspace, using the *xl toolstack* installed in the rootfs. DomUs are configured via config files, written using the xl configuration syntax. Example configuration files can be found in Dom0's rootfs in */etc/xen/* folder.

Example for booting a DomU as described by tools/examples/domU1_mmc.cfg (384MB of RAM, 1 vCPU pinned to a pCPU, rootfs mounted directly on sdcard):

```
# Starting the DomU
root@s32g274aevb:~# xl create /etc/xen/domU1_mmc.cfg
# Switching to DomU's console
root@s32g274aevb:~# xl console domu1
...
# Switching back to Dom0's console
root@s32g274aevb:~# CTRL-]
```

When booting a Dom0less VM, its console cannot be reached using the *xl console* command, but by pressing *"CTRL-AAA"*, which cycles between Dom0's console, the Dom0less VMs' consoles and the Xen debug console.

Information about the system and the running VMs can be accessed using other xl commands:

```
# Info about system (RAM, CPUs etc.)
root@s32g274aevb:~# xl info

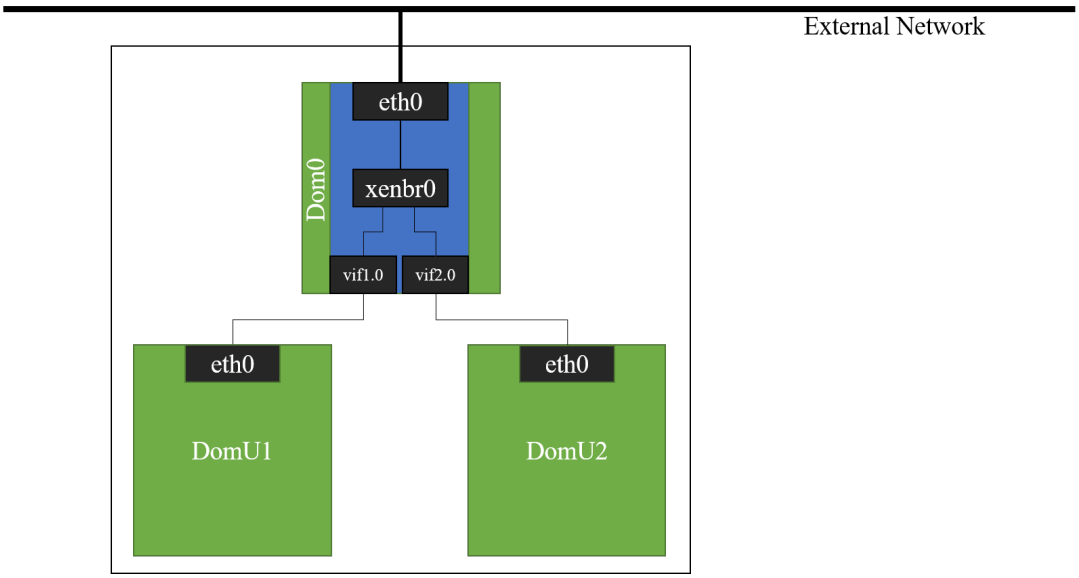
# Info about running VMs (ID, state, resources)
root@s32g274aevb:~# xl list

# Info about resource usage
root@s32g274aevb:~# xl top

# Shutdown a DomU
root@s32g274aevb:~# xl destroy <domU_ID>
```

24.3.6 Networking in DomUs

In order to have networking access in DomUs, Xen enables paravirtualized network interfaces (ethY) in the guests which are then linked to virtual interfaces created in Dom0. The virtual interfaces in Dom0 (vifX.Y) are connected to the physical host interface (eth0) via a bridge (xenbrX), which must be explicitly created and will act as a virtual switch. The virtual interfaces are created automatically by Xen at guest creation time. The bridge will take packets from the virtual machines and forward them on to the physical network.



Therefore, for a conventional networking setup, in which the DomU is connected to the external network, the user needs to setup the bridge by running the following commands in Dom0:

```
# Remove all IPs from physical interface
ip addr flush dev eth0

# Create bridge
brctl addbr xenbr0

# Add physical interface to bridge
brctl addif xenbr0 eth0

# Set bridge and physical interface up
ip link set dev xenbr0 up
ip link set dev eth0 up
```

```
# Request IP from DHCP Server on bridge
udhcpc -i xenbr0

# Or manually assign an IP Address
ip address add 192.168.0.2/24 dev xenbr0
```

Now, for Xen to attach the PV interface in DomU (eth0) to a virtual interface in Dom0, the following line must be added to the DomU config file used:

```
vif = [ 'bridge=xenbr0' ]
```

Additionally, the MAC and IP addresses of the DomU network interface can be specified in the same above-mentioned line, by specifying configuration keys 'mac=...,ip=...'.



- The physical host interface in Dom0 (e.g. "*eth0*", "*pfe0*") must not have any assigned IP address. Only "*xenbr0*" should have an assigned IP. Otherwise, the DomU will have no external network access.
- When the GMAC controller is assigned to a Dom0less-DomU, it cannot be used by Dom0. Thus, Dom0 cannot share the access to it through PV drivers.

24.3.7 VirtIO PV Drivers

VirtIO is a standardized common framework for IO virtualization in systems running hypervisors, based on the para-virtualization (PV) approach. It is similar to Xen PV device drivers, but VirtIO defines a protocol for communication between many types of drivers and devices, compared to the limited number supported in the Xen PV set. The latest version of the VirtIO specification is v1.2.

Currently, there are two approaches for using VirtIO PV drivers with a DomU Guest VM:

1. Using a `virtio-disk` standalone backend application for PV storage, running in Dom0; the DomU Guest VM will use a generic `virtio-blk` frontend driver to get access to a physical SDCard partition.
2. Using the `virtio-disk` and `virtio-net` backends integrated in QEMU System Emulator, which will be running in Dom0; the DomU Guest VM will use the generic `virtio-disk` and `virtio-net` frontend drivers to access a physical SDCard partition and a networking interface.

VirtIO support in Linux Kernel and Xen Hypervisor can be enabled by adding the `"virtio"` option besides `"xen"` in *conf/local.conf*:

```
DISTRO_FEATURES:append = " xen virtio"
```

For the first VirtIO approach described above, after booting the image with these options enabled, a DomU with PV storage based on the standalone `virtio-disk` rootfs can be spawned using the *domU1_mmc_virtio-disk.cfg* file, after having run the *virtio-disk* standalone backend application:

```
# Spawn the virtio-disk daemon process in Dom0
./usr/bin/virtio-disk &

# Create the DomU using domU1_mmc_virtio-disk.cfg
xl create /etc/xen/domU1_mmc_virtio-disk.cfg
```

For the second VirtIO approach described above, to use the VirtIO backends hosted in QEMU (`virtio-blk` and `virtio-net`) for PV storage and networking, the DomU Guest VM can be spawned using the *domU1_mmc_virtio-qemu.cfg* file, after having setup the `xenbr0` bridge in Dom0. An example setup:

```
# Setup bridge, attach to network interface (e.g., eth0, pfe0, pfe1, pfe2)
brctl addbr xenbr0
brctl addif xenbr0 eth0

# Make sure the network interface attached to the bridge has no IP
ip address flush dev eth0
ip link set dev xenbr0 up

# Add IP to the bridge (manually or via DHCP)
ip address add 10.0.0.100/16 dev xenbr0

# Create the DomU using domU1_mmc_virtio-qemu.cfg
xl create /etc/xen/domU1_mmc_virtio-qemu.cfg
```

For more information about the PV networking features supported by Xen, please check the "Networking in DomUs" section.

25 Arm Trusted Firmware

The Arm Trusted Firmware (TF-A) is a software component of the Linux BSP with a double role as a bootloader and Secure Monitor. As a bootloader, it runs before U-Boot on the A53 boot core and takes over some of the core and SoC initializations such as clocks or MC_ME partitions (mostly for those peripherals and subsystems that the TF-A needs). As a Secure Monitor, the TF-A provides power management services to other software components, via PSCI and SCMI.

The TF-A is divided into several stages, called Boot Levels (BLs). Arm TF-A specification includes BL1 (Trusted ROM), BL2 (Trusted Boot Firmware), BL31 (Secure Monitor), BL32 (Trusted OS) and BL33 (non-trusted world, e.g. U-Boot or UEFI). The BSP 42.0 TF-A implements the BL2 and BL31 stages:

- BL2 is the component pointed to by the boot IVT, see chapter TF-A SMP Boot. It is deployed in SRAM at a statically-defined address. Its main role is to initialize the platform and in particular the DDR, and load BL31. BL2 is a transient component. After the the next stage (BL33/U-Boot) has booted, BL2 is no longer used and its SRAM space can be reclaimed.
- BL31 is loaded by BL2 in DDR, at a statically-defined address. Currently, BL31 is deployed at 0xFF800000 in DDR with a size of 2MB (0x200000) ending at 0xFF9FFFFF . These are configured in TF-A sources at: plat/nxp/s32/include/s32_platform_def.h. BL31 is a resident component. It occupies the DDR region for the entire uptime of the platform, being involved in runtime power management, platform reset, suspend to RAM, etc. See Power Management chapter for details. It is to be noted that U-Boot cooperates with the TF-A in actually reserving the 0xFF800000-0xFF9FFFFF and hiding that region from the Linux kernel. To do so, U-Boot alters the Linux device tree before loading the kernel. See the SMP Information chapter SMP Boot for more details.

26 OP-TEE

The OP-TEE software component will provide Linux BSP users with a Trusted Execution Environment (TEE) so that critical applications working with sensitive data can leverage the hardware isolation of Arm TrustZone. Such applications are called Trusted Applications (TAs), and they run at Secure EL0.

The OP-TEE project is mainly split in two modules:

- **The optee_client module**, residing entirely in the user space of Linux, facilitates the communication between normal world applications (running at Non-Secure EL0 level) and secure world through the TEE Client API and the tee-supplicant daemon. When a normal world application invokes a TA via an API call, behind the scenes the normal world - the OP-TEE driver in the Linux Kernel - executes a Secure Monitor Call (SMC) instruction. The Secure Monitor (SM) traps the exception and will switch execution to optee_os in the secure world. The process of switching to the normal world from the secure world uses the same mechanisms.

  Note: The tee-supplicant daemon is started automatically by systemd's init process.

- **The optee_os module**, which runs at Secure EL1, ensures that TAs have all the necessary resources to be executed safely and securely. It has its minimal kernel, drivers and libraries, and implements the TEE Internal Core API that can leverage the TrustZone (TZ) hardware isolation. While optee_client implementation is platform-independent, the optee_os module is configured according to the platform specifications. Trusted memory zones, UART communication and GIC initialization are configured to ensure proper usage. TF-A is responsible for loading and executing the optee_os binaries. Moreover, as mentioned above, OP-TEE also relies on the Secure Monitor implemented in the TF-A.

Besides the optee_os and optee_client components, which are mandatory for creating a functional Trusted Execution Environment, the Linux BSP comes with another two modules offered by the OP-TEE project: optee_examples and optee_test. The contents of both these modules reside in the user space of the Linux BSP.

As the name implies, **the optee_examples module's** purpose is to give a glimpse into the basic building blocks and behavior of some TAs by exposing typical use-cases. There are a number of normal applications in the rootfs, all of them prefixed with `optee_example_` under the path */usr/bin/*, each corresponding to and calling a different TA. As an example, for running the `optee_example_hello_world` application, one must enter the following command:

```
optee_example_hello_world
```

The source code for these applications can be found on the official GitHub page: OP-TEE Sample Applications.

The optee_test module contains several thousands of tests checking that the OP-TEE integration has been done with no regression. The easiest way of running all the default tests is by entering the following command:

```
xtest
```

More information about running `xtest` can be found in the official OP-TEE documentation: optee_test.

26.1 RPMB Secure Storage

RPMB stands for Replay-Protected Memory Block. It is a partition on the eMMC where the host can read/write blocks of data, but cannot delete them. Every block must be authenticated and signed with a key. This specific key is One-Time Programmable (OTP): it can be programmed only once into eMMC's RPMB partition, in plain text. Thus, this action must take place in a secure environment. The protection against replay attacks is granted through a read-only counter

that is incremented after every write operation and cannot be decremented. This way, an attacker cannot tamper with previously written blocks and only new blocks can be written up to the limit of the counter.

OP-TEE does not have an eMMC driver and instead relies on the Linux Kernel driver to read/write blocks from/to eMMC. These requests are managed by the `tee-supplicant` which further uses the `ioctl` interface to communicate with the device. Each block of data is encrypted using the AES-CBC algorithm to prevent exposure of sensitive data to the normal world. Requesting to program the RPMB Key into the eMMC is possible, though the key would show up in plain text in the normal world - take caution in performing this operation, otherwise the key might leak.

OP-TEE derives the RPMB key from the Hardware Unique Key (HUK). The Hardware Security Engine (HSE) generates the HUK through a key derivation service. Starting from a secret key embedded within HSE and using a Key Derivation Function (KDF), HSE exports a new key that is used as the HUK in OP-TEE. This secret key is unique for each SoC, meaning that HUKs and RPMB Keys are also unique. As such, once a RPMB Key has been generated and programmed into the RPMB partition, the eMMC is tied to that specific SoC. If the SoC is replaced on the board containing the eMMC chip, then the existing RPMB partition will no longer be accessible.

To demonstrate RPMB Secure Storage capabilities, the Linux filesystem hosts a Trusted Application called `optee_example_secure_storage`. The application performs basic I/O operations with objects stored in the RPMB Secure Storage.



At the moment, the RPMB demo is not fully configured for a completely secure application, but mainly as an example of HSE's HUK capabilities. Further development is needed to restrict the availability of such a HUK to only specific execution environments on the SoC.



By default, `tee-supplicant` emulates a RPMB partition and, for safety considerations, the Linux BSP uses this emulation. Disabling the emulation translates to a commitment of permanently programming the RPMB Key into the eMMC partition, assuming the corresponding consequences that come with it.

Since the RPMB partition is emulated by default, it is not mandatory to boot from an eMMC to use this feature. Any boot medium supported in the Linux BSP does the job. To use the actual RPMB partition on the eMMC, please refer to the Setup eMMC section for instructions on using the eMMC.

Enable the RPMB Secure Storage by compiling optee_os with the following build parameters:

- `CFG_RPMB_FS=y`
  Enables the RPMB File System in optee_os
- `CFG_RPMB_WRITE_KEY=y`
  Write the generated RPMB Key into the RPMB partition
- `CFG_REE_FS=n`
  By default, Secure Storage is enabled using the normal world file system. Disable it since we're using RPMB
- `CFG_NXP_HSE=y`
  Enable the HSE Crypto Driver to get the HUK and offload AES-CBC operations to the crypto engine
- `CFG_NXP_HSE_FWDIR=/<path>/<to>/HSE_FW_<platform>_<premium>_<vmajor>_<vminor>_<vpatch>`
  This value should point to the folder that contains the HSE FW *interface* folder

Command example:

```
make CROSS_COMPILE64=/path/to/your/toolchain/dir/bin/aarch64-none-linux-gnu- ARCH=arm \
        PLATFORM=s32 PLATFORM_FLAVOR=s32g2 CFG_RPMB_FS=y CFG_RPMB_WRITE_KEY=y CFG_REE_FS=y\
        CFG_NXP_HSE=y CFG_NXP_HSE_FWDIR=/<path>/<to>/HSE_FW_<platform>_<premium>_<vmajor>_<vminor>_<vpatch>
```

Including the HSE Firmware in the fip image is a prerequisite to using the RPMB Secure Storage feature. Please refer to the HSE Crypto Driver in OP-TEE section for instructions on how to build a fip image with HSE Firmware & optee_os included.

26.2 TrustZone Key Provisioning

The demo's purpose is to showcase the provisioning of symmetric keys into HSE's NVM Catalog in a secure manner, leveraging the TrustZone capabilities. The demo has two distinct phases, Manufacturing Time & Runtime, and two separate actors: Host & Target. At runtime, the keys are neither exposed to the normal world, nor to the secure world. The keys are encrypted & authenticated in an earlier manufacturing stage, on the host, which is considered a trusted environment. A unique key is used for encrypting all the other keys: the Key Encryption Key (KEK). Like the other keys, the KEK is only exposed in plaintext during the manufacturing stage. It is imported into HSE's NVM Catalog and is used at runtime when decryption & authentication is requested for key provisioning.

26.2.1 Terminology

- Host: Yocto Build Machine
  - Considered a Trusted Environment
- Target: S32 Platform
- KEK: Key Encryption Key
  - Shared between Host & Target
  - Cannot be changed on Target; can be erased at runtime
  - Kept encrypted & authenticated inside HSE's SysImage
- Plain Key
  - Generated on the Host (e.g. during manufacturing, or for key renewal)
- Provisioned Key
  - A blobbed key such that the TrustZone software never sees it in clear
  - Generated from Plain Key and KEK
  - HSE can import the provisioned key and use it internally
  - Kept encrypted & authenticated inside HSE's SysImage
- Trusted Code
  - During Manufacturing, all code running on Target is considered trusted (even if not in TrustZone)
  - TrustZone code at runtime

26.2.2 Timeline of Actions

	Host	Target

Manufacturing	1. Generate KEK. 2. Transport KEK in Target's rootfs.	3. Format HSE Key Catalogs. 4. Import KEK into a pre-defined HSE NVM Key Slot. 5. Publish SysImage & write SysImage to persistent storage.
Runtime	1. Generate Plain Key. 2. Generate Provisioned Key: encrypt & authenticate the 'plain key' using AES-GCM with KEK as the encryption key and a random IV. Key Information must also be authenticated and will be included in the AAD. 3. Transport Provisioned Key in Target's rootfs.	4. TrustZone software reads Provisioned Key from Normal World's rootfs. 5. Imports Provisioned Key into HSE NVM catalog using pre-defined key slot IDs. 6. Publish SysImage & write SysImage to persistent storage.

26.2.3 Build and Run Instructions

The demo is implemented in two separate repositories:

1. optee: The TEE Internal Core API has been extended through a Pseudo Trusted Application to support the Key Provisioning operation. See file *lib/libutee/include/pta_hse_kp.h* for the complete interface.
2. pkcs11-hse: Host's & Target's userspace applications are implemented in *examples/trustzone-key-provision*

To demonstrate the TrustZone Key Provisioning, a dummy KEK and Plain Key have been defined in `pkcs11-hse's` *examples/trustzone-key-provision/host/aes-gcm-key-encryption.c* The file is meant to be compiled using the Host's compiler and then run on the Host machine. Both actions are executed during a Yocto Build when the following lines are added in *conf/local.conf* file of the current build machine:

```
DISTRO_FEATURES:append = " hse optee"
NXP_FIRMWARE_LOCAL_DIR = "/path/to/firmware/folder"
```

`NXP_FIRMWARE_LOCAL_DIR` must point to the the location of the folder containing the HSE Firmware deliverables.

As a result of the build, the files */usr/share/kek.bin* and */usr/share/enckey.bin*, corresponding to the KEK and Provisioned Key respectively, are added in the Target's rootfs. Also, the build will have added all the necessary userspace tools which are used for provisioning the KEK and the Provisioned Key.

Format the catalog, import the KEK into HSE's NVM Catalog and publish the new contents of the catalog in the SysImage. Finally, reboot the board so that the OP-TEE HSE Crypto Driver is initialized:

```
root@s32g274aevb: hse-secboot -f -d <device>
root@s32g274aevb: hse-kek-provision /usr/share/kek.bin
root@s32g274aevb: hse-sysimg <device>
root@s32g274aevb: reboot -n
```

`<device>` is the SD Card device, usually */dev/mmcblk0*.

Once the board has rebooted, import the Provisioned Key into HSE's NVM Catalog in the TrustZone and publish the new contents of the catalog in the SysImage:

```
root@s32g274aevb: hse-symkey-provision /usr/share/enckey.bin <key-group> <key-slot>
root@s32g274aevb: hse-sysimg <device>
```

For a complete list of available Key Groups and Key Slots for the provisioning operations, please consult the file *core/drivers/crypto/hse/include/hse_kp.h* in `optee_os` repository.

26.3 Opaque Keys

This feature aims to enhance OP-TEE's security by leveraging HSE's capability of storing the key material in an encrypted key storage - HSE Key Catalog - that is accessible only to HSE. The keys in the catalog are referenced through unique IDs called Key Handles. Instead of passing the plain key material when performing crypto operations, host applications or TAs will provide the Key Handle. HSE will then access the key material corresponding to the handle and will perform the operation. This way, the key material is never exposed outside of the key storage, making it 'opaque' to all the parties in the trust chain.

> Please note that the "opaque keys" or "embedded key handles" definitions as used by the HSE Driver are not the same as the "Opaque Handles" type in the TEE Internal Core API specification. Aside from the naming, there is no overlap between the two notions

26.3.1 Plain Keys vs Opaque Keys

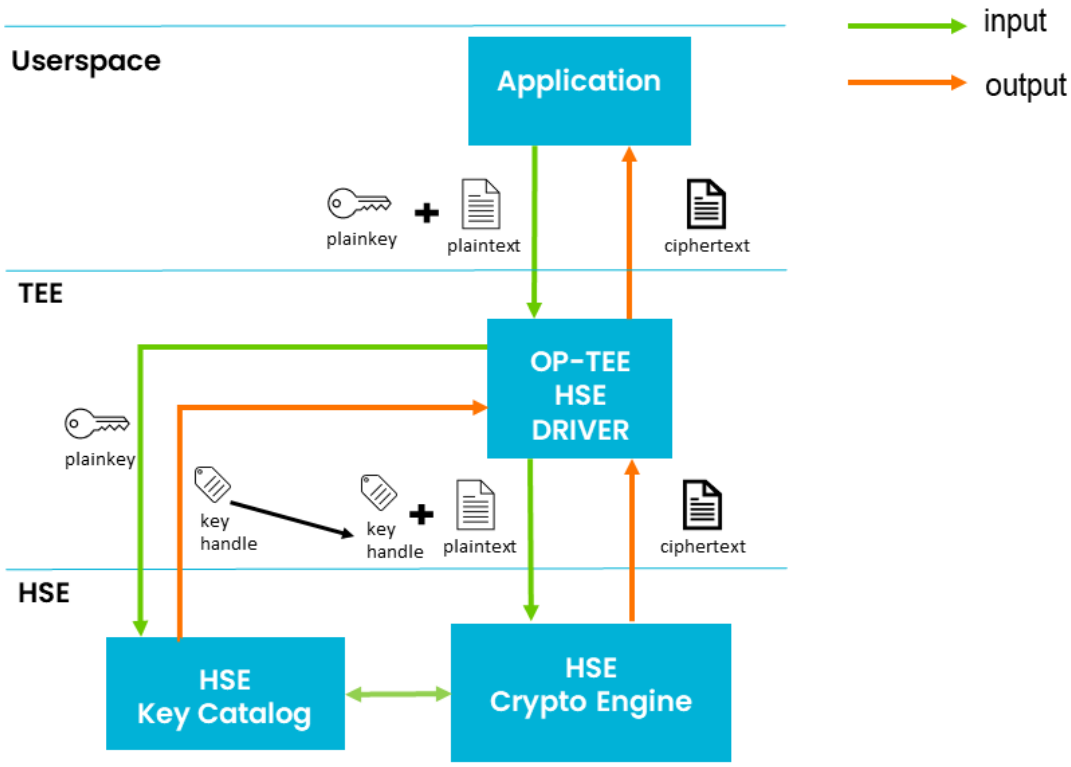


Figure 4: Plain Key Cipher Operation Flow

The TEE Internal Core API implemented in OP-TEE mandates that keys are passed in plaintext format in the buffers that will eventually be processed in the optee_os' HSE Crypto Driver. Figure 4 depicts the flow of a symmetric cipher operation in which a plainkey, along with the plaintext, are provided as inputs for this operation. The inputs are passed through the TEE stack to the optee_os' HSE Crypto Driver. By design HSE can only work with keys that are part of the

HSE Key Catalog. Therefore, the driver will first import the plainkey in an empty slot of the catalog and HSE will return the Key Handle of the respective slot. The driver will then issue a symmetric cipher service request to HSE, providing the previously returned Key Handle and the plaintext. Upon the successful execution of the cipher operation, HSE will return the ciphertext. The ciphertext is then sent through the TEE layers all the way back to the application.

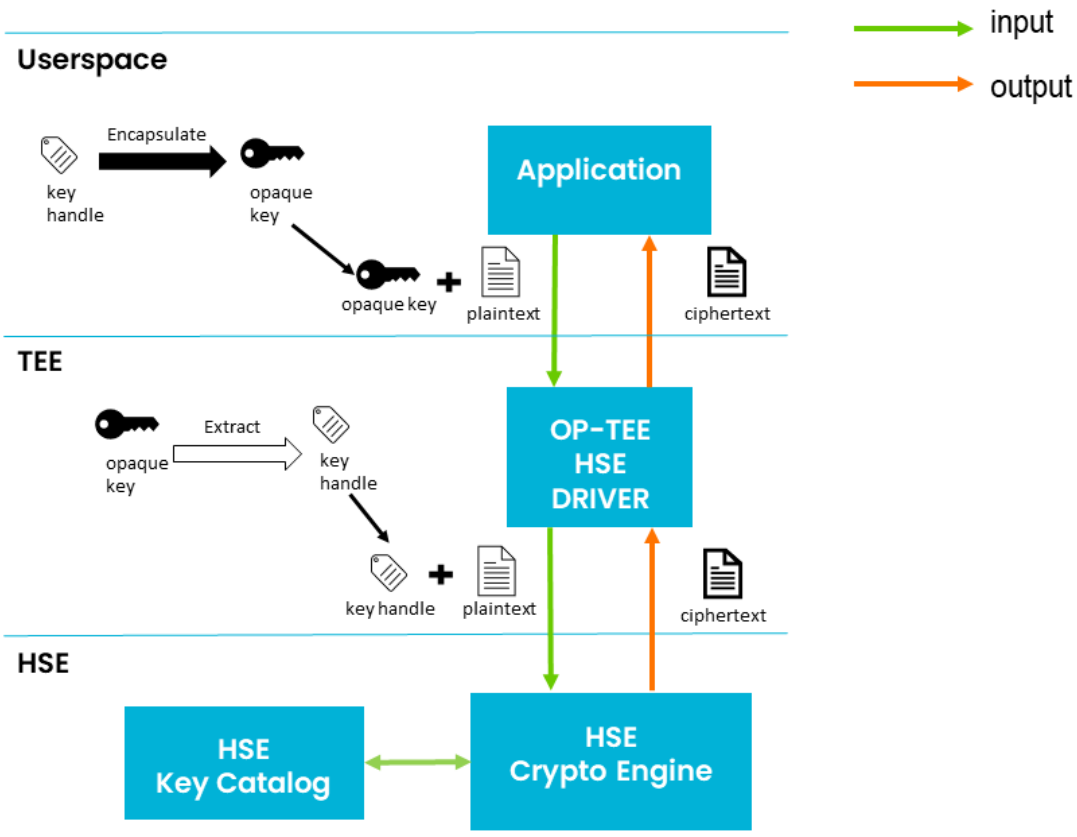


Figure 5: Opaque Key Cipher Operation Flow

The same symmetric cipher operation flow is depicted in Figure 5, but this time using an opaque key instead of a plainkey. The opaque key is a custom structure which has an HSE Key Handle embedded within it. The opaque key is put in the same buffer that is normally used for plainkeys and follows the same path through the TEE stack as any other plainkey would. The TEE layers have no knowledge about the opaque key and treat it as any other plainkey. The only component in OP-TEE which identifies the buffer as an opaque key is the HSE Driver – it will extract the Key Handle from this structure, check that it is valid, and will continue with performing the symmetric cipher service request, as explained in the previous example. The key material corresponding to the input Key Handle is expected to be provisioned into the catalog before the crypto operation is performed.
The design of the opaque keys feature allows one to migrate opaque keys-aware TAs/Host Apps from one TEE implementation to another because the feature does not require TEE modifications or cooperation; only TAs/Host Apps and the HSE Driver need to be aware of the Opaque Keys.

Opaque keys work with all cryptographic operations supported by the HSE Driver. The symmetric cipher example was chosen for simplicity.

26.3.2 Opaque Key Format

The opaque key is made of two distinct series of bytes which are concatenated: the 'MAGIC' and the HSE Key Handle. The 'MAGIC' is a 28-byte long constant value prepended to the Key Handle so that the HSE Driver in optee_os can detect the opaque key. Taking into consideration that the Opaque Key feature is using the same mechanism of transmitting the key as the usual plainkey case does, there are two reasons for using the 'MAGIC' value instead of simply putting the Key Handle in the key buffer:

1. All algorithm types (e.g. Cipher, RSA, ECC etc.) do some sort of input size checking. The 32-byte opaque key is a one-size-fits-all value. If only the 4-byte Key Handle were provided as input, the buffer would be rejected by most algorithms due to size considerations.
2. The 'MAGIC' is the sole differentiator between a plainkey and an opaque key. Without it, the optee_os HSE Driver cannot tell if the key buffer contains a plainkey or an opaque key.

$$\underbrace{CABF001CCABF541C397804A35A11249F64EE8923908CF5649A8C7E35}_{MAGIC}\ \underbrace{01080100}_{KeyHandle}$$

Figure 6: Opaque Key Format



Even though the 'MAGIC' value allows the use of plainkeys and opaque keys concomitantly, it comes at the cost of reducing the key space. Any plainkey that might match a valid 'MAGIC + Key Handle' value will be treated as an opaque key by the driver. This design choice assists with backward compatibility of legacy TAs/Host Apps which might not need to switch to using Opaque Keys. However, given the major security advantages that this feature brings, and also because some crypto operations supported by HSE will *only* work with opaque keys when deployed in the field, one might consider replacing runtime detection of opaque keys with a hard-coded approach, thus also regaining the original key space. The hard-coded option is not currently available in the BSP. Only run-time detection of opaque keys is supported.



Any Key Handle can be embedded in the opaque key, with the exception of Key Handles that correspond to the Key Groups used by the optee_os HSE Driver. The keys in the groups defined in core/drivers/crypto/hse/crypto.mk are subject to overwriting/erasing and, by default, the Opaque Key Feature implementation assumes that the provided Key Handles correspond to key slots which are not altered during an operation. As such, the responsibility of key slots management falls in the hands of the application which uses opaque keys.

26.3.3 Configuration and Examples

There are two prerequisites for using the opaque keys feature:

1. Knowing the configuration of the HSE Key Catalog so that key handles can be computed based on the type of catalog, key group and key slot. In Linux BSP the catalog is defined at examples/hse-secboot/keys-config.h.
2. The key material must be provisioned in the catalog. One can do this using the PTA core/pta/pta_hse_kp.c. Please note that the key material can be provisioned only in Key Groups that are not used by the optee_os' HSE driver and the provisioning must be done before the start of a crypto operation using Opaque Keys.

The optee_os' build parameter `CFG_HSE_EMBED_KEYHANDLES` is used to enable/disable the Opaque Key Feature. Please note that it is enabled by default. The build parameter `CFG_HSE_KP_PTA` enables/disables the plain key



provisioning PTA that is used to populate the HSE Key Catalog with key material. `CFG_HSE_KP_PTA` is enabled by default. All HSE Crypto drivers in optee_os support the use of Opaque Keys.

The demo application in tee_opaque_keys showcases the use of Opaque Keys in OP-TEE using a Host Application and a TA. It is a good starting point for understanding how to format an Opaque Key (or an Embedded Key Handle as it is referenced in the code) and use it in a crypto operation. The applications also provide key provisioning examples. Please consult the README file for build and run instructions, as well as information about the flow of the demo.

26.3.4 PBKDF2 Support for Opaque Keys

HSE's key derivation service has two particularities that make PBKDF2 support for opaque keys differ from the general support explained in the paragraphs before.

First thing to note is that any generated key can *only* be exported back to host memory wrapped (encrypted, optionally also signed) in a pre-provisioned key from HSE's NVM catalog. The caller application can never receive a plaintext generated key, not even for backward compatibility or for testing/debugging reasons. This is a *strong security feature* that HSE provides, but at the same time it is incompatible with the TEE Internal Core API for PBKDF2, which expects plaintext derived keys to be returned to the caller. The immediate consequence is that a sound security setup for PBKDF2 will *only* work if support for opaque keys is enabled and opaque keys are used.

Second thing to note is that key derivation algorithms supported by HSE always produce an `HSE_KEY_TYPE_SHARED_SECRET`. This key format cannot, in general, be directly used to further perform crypto operations in the HSE. HSE does provide a DeriveCopy service through which keys can be, in appropriately parametrized conditions, transformed from one type to another. TEE Internal Core API, however, does not have a direct equivalent of HSE's DeriveCopy.

To address these peculiarities, the PBKDF2 implementation (HSE offload) with opaque keys support:

- departs from the 'MAGIC'-based buffer format previously described in this chapter. That would be superfluous because, as explained, key generation would only work with opaque keys anyway. The immediate implication is that the user must, upon invocation of the PBKDF2 TEE Internal Core API interface, specify a user-managed key slot in the HSE key catalog where the derived key would be placed after generation. Both RAM and NVM catalogs are supported as the destination for the derived key. Caller application is responsible for managing its own key slots. Key slots managed by the HSE Driver cannot be used in this operation.
- allows the user to specify, upon invocation, the actual type (in HSE terms) of the derived key. This means that, when calling the PBKDF2 interface, the user/TA must specify whether the derived key should be an `HSE_KEY_TYPE_AES` or an `HSE_KEY_TYPE_HMAC`. The HSE Driver will automatically and transparently perform conversion of the derived key to this requested type and will return to the caller the key handle (opaque key) of the appropriate AES or HMAC key. Note that the specified, user-managed, target key handle must reside in a key slot of the appropriate type, AES or HMAC. A mismatch between the target key handle and the requested key type will result in a runtime error.

The HSE PBKDF2 driver in *core/drivers/crypto/hse_pbkdf2.c* specifies two structs which implement this modified interface:

- `struct pbkdf2_password_buf` which is a surrogate for the original input 'password' buffer. Now in addition to the actual password, it must also contain a metadata structure that specifies the destination key handle (managed by the user/TA) and the derived key type (AES or HSE), as explained above. The 'password length' parameter to PBKDF2 must correctly reflect the structure of this buffer, i.e. it must be the length of the actual password plus the length of the metadata, in order for the TEE core and intermediate layers to allocate, copy and propagate correctly this entire buffer. The TEE has no real reason to look *into* the contents of this buffer, which it should regard as the raw password. Only the host/TA and the HSE Driver are required to know the real format of the 'password' input buffer, while the TEE Client API and the TEE Internal Core API are used for 'tunneling' the request down to the HSE Driver.

- `struct pbkdf2_derived_key_buf` which is a surrogate for the original 'derived key' output buffer. Also for the purposes of TEE handling it (in other words, to hide its actual content from the TEE internals), this buffer must be allocated of the actual size of the requested derived key, even though upon return it will only contain a 4-byte key handle (opaque key), the rest being allocated, but unused.

27 Optional Features

27.1 Unrestricted userspace access to hardware registers

For debugging purposes, unrestricted access to hardware registers can be enabled in Linux. Invoke menuconfig before building the kernel (see "Invoke menuconfig to setup specific configuration for Linux".), and enable 'S32_DEBUG_REGACCESS' from 'Device Drivers: SOC (System On Chip) specific Drivers: NXP SoC drivers'.

After booting, the files in */sys/kernel/s32_regaccess* can be used to access hardware registers. Usage:

```
echo '0x<address> <access_width>' > /sys/kernel/s32_regaccess/read
echo '0x<address> <access_width> 0x<value>' > /sys/kernel/s32_regaccess/write
```

Use with caution. An incorrect access width, an incorrect address or an access to a device without an enabled clock may lead to a kernel crash.

27.2 SysVInit setup

Currently we are using the SystemD manager, by default, as inherited from Yocto, but Yocto Community also supports SysVInit manager. It is a matter of configuration on the Linux BSP to select Systemd or SysVInit manager. This configuration is based on Yocto upstream community support.



SysVInit is not part of the testing and validation of Linux BSP releases.

SysVInit manager can be enabled by adding to conf/local.conf the following setting:

```
INIT_MANAGER = "sysvinit"
```

27.3 DM-Verity

DM-Verity is a security feature meant to provide block integrity checking for Linux's rootfs. Each block of data is hashed and subsequently added in a hash-tree. The root node (or the root hash) is calculated based on every block hash. One change in a single device block leads to a different root hash overall. This way, only the root hash needs to be verified to guarantee the integrity of the whole hash tree.

The verification data (the hashes) are appended at the end of the rootfs. At boot time, the Linux kernel will verify the integrity of the rootfs by checking the appended hash values against values computed at boot time. The hash function used in this process is SHA-256 and the Linux kernel will use the sha256 implementation with the highest crypto algorithm priority. If hardware acceleration is not available, 'sha256-generic' will be used.

DM-Verity can be enabled in Yocto by adding the following distro feature in *conf/local.conf*:

```
DISTRO_FEATURES:append = " dm-verity"
```

By default, dm-verity is enabled for the `fsl-image-base` image. For other images, override the `DM_VERITY_IMAGE` Yocto variable with the desired value in *conf/local.conf*. For example, dm-verity for `fsl-image-auto` can be enabled by appending the following line to the *conf/local.conf*:

```
DM_VERITY_IMAGE = "fsl-image-auto"
```



The newly generated image will have a `.wic` extension instead of the usual `.sdcard` one. Still, the image will boot from an SD Card. Currently, only `fsl-image-base` and `fsl-image-auto` image types support dm-verity.



After writing a `.wic` image onto an SD card, it is imperative to check the SD card content against the content of the `.wic` image. If the SD card content differs from that of the image, the verity check will fail. Please refer to the Setup an SD/MMC card section for instructions on writing an image and verifying its contents.

For more information on dm-verity support in Linux, please consult https://www.kernel.org/doc/html/v6.6/admin-guide/device-mapper/verity.html

27.4 Root Hash Signing

Even if `dm-verity` is enabled, the file system is still vulnerable to attacks: a bad actor can replace the root hash, metadata of the file system and the root hash input, thus breaking the secure boot chain of trust at the file system level. Root hash signing is the process which completes the chain of trust for the file system. The root hash is signed using an RSA private key and, at boot time, the signature is verified using the corresponding RSA public key.

To enable the signing process, add `dm-verity` and `root-hash-sign` to `DISTRO_FEATURES`:

```
DISTRO_FEATURES:append = " dm-verity root-hash-sign"
```

The build system generates an RSA key pair which, at the end of the build process, is available in the directory (`tmp/deploy/images/<machine>`). The key pair consists of a private key and an corresponding public key in the .PEM format. If you want to use other specific key pairs, add the path to your keys in `conf/local.conf`:

```
DM_VERITY_PRIVATE_KEY = "/path/to/private_key.pem"
DM_VERITY_PUBLIC_KEY = "/path/to/public_key.pem"
```

27.5 Changing UART baud rate

27.5.1 Changing UART baud rate from Yocto

The default console for all components is LINFlexD UART0 and it is configured at a 115200 bits/s baud rate. The default baud rate can be changed by adding the following line in `conf/local.conf`:

```
SERIAL_CONSOLES = "<new_baudrate>;ttyLF0"
```

where <new_baudrate> is the desired baud rate value. This change will be applied to TF-A, OP-TEE, U-Boot, Linux Kernel and its filesystem.



The assumption is that the first entry in SERIAL_CONSOLES is the one used as a serial boot console. At this point only LINFlexD UART0 is supported as a serial boot console, therefore the first entry should refer to ttyLF0.

Currently, one can choose from the following list of baud rates: 9600, 19200, 38400, 57600, 115200, 921600, 1000000, 1500000, 2000000, 3000000. Additional changes are required if the available baud rates do not meet one's needs. The high-level steps are explained below:

- Update conf/local.conf.
- Add a new entry in meta-alb-bsp/recipes-bsp/u-boot/u-boot-s32/build/serial_configs/, named baudrate_<new_baudrate>.cfg which should contain "CONFIG_BAUDRATE=<new_baudrate>".
- Update CONFIG_SYS_BAUDRATE_TABLE from U-Boot source code and re-build:

```
bitbake u-boot -c devshell
# Open include/configs/s32-cc.h and add the new value in
# CONFIG_SYS_BAUDRATE_TABLE list.
exit
bitbake -c compile -f u-boot
bitbake u-boot
```

- Build the desired BSP image.

27.5.2 Changing UART baud rate per component

The default baud rate can be changed per component as follows:

- For TF-A, S32_LINFLEX_BAUDRATE=<new_baudrate> must be added to the compile command that is detailed in Building the Arm Trusted Firmware section.
- For OP-TEE, CFG_S32_UART_BAUDRATE=<new_baudrate> must be added to the compile command that is detailed in Building the optee_os component of OP-TEE section.
- For U-Boot, change CONFIG_BAUDRATE from menuconfig and recompile it. The new value should be added in CONFIG_SYS_BAUDRATE_TABLE if not present already. Since U-Boot will switch to the value defined by baudrate environment variable, which may be loaded from an external memory, one more step should be done after U-Boot boots:

```
## Change serial terminal speed to <old_baudrate> and stop
## at U-Boot prompt. Run the following one-line command:
=> setenv baudrate <new_value>; saveenv;
## Change serial terminal speed to <new_baudrate>.
```

- Linux kernel will use the U-Boot environment variable baudrate.
- Once reached to login, SystemD serial-getty service will use the baud rate mentioned in /etc/systemd/system/getty.target.wants/serial-getty@ttyLF0.service script, so will switch back to the <old_baudrate>. Changing it can be achieved by the following steps:

```
## Change serial terminal speed to <old_baudrate> and login.
## Open /etc/systemd/system/getty.target.wants/serial-getty\@ttyLF0.service,
## modify UART speed as below and save the file:
[...]
[Service]
Environment="TERM=linux"
ExecStart=-/sbin/agetty -8 -L %I <new_baudrate> $TERM
Type=idle
Restart=always
[...]
## Restart and change serial terminal speed to <new_baudrate>.
```

28 Demo Applications

The demo applications illustrate some basic capabilities over the Linux BSP.
These samples had been structured in a modular way in order to be not just easy to read but even easy to understand by the users of our product.

Because the processor of the S32G2 platform is an Arm Cortex-A53, we use the Arm compiler to generate the binary files of our code examples.



For PCIe Demos, please see section 28.4.

The demos can be automatically built from Yocto or manually using a cross-compiler (see section 3.2.1).

Building the Demo Applications from Yocto:

In order to build the demos and add them to the *.sdcard* image, simply add the following lines to *conf/local.conf*:

```
COMPATIBLE_MACHINE_pn-demo-samples = "${MACHINE}"
CORE_IMAGE_EXTRA_INSTALL += "demo-samples"
```

Then build any image, e.g. *fsl-image-auto*. The demo applications will be deployed on target in directory */opt/samples*.

Manually Building the Demo Applications:

In the root directory will be a global *Makefile* which will recursively call each sample's *Makefile*. The common variables will be defined in *common.mk* file.
In order to build all samples you can simply call "make" and "make clean" to delete all the already compiled object files. To personalize the build operation you can call "make sample_name" and will be executed just the sample's specified Makefile.
Some examples of building the samples:

- `make CFLAGS="-I" CROSS_COMPILE="aarch64-none-linux-gnu-"`

- `make SYSROOT="/opt/rootfs"`

28.1 Multicore Sample Application

This C sample illustrates the benefits of multicore processing using kernel level threads. It is known that the naive multiplication algorithm of two bidimensional arrays has a $\Theta(n^3)$ time complexity because of its three 'for' structures. To push this case to the limits, we simultaneously take five different bidimensional arrays from five different files in order to multiply them several times.

The loop number is given as an argument for this sample. For example you can run ./multicore 1000000 and the array will be multiplied by 1000000 times.

We consider that our machine has a processor with 4 cores. Cores 0 to 3 have assigned the first four threads while the fifth one will go on the first core that will become free (almost a random choice).

In *functions.c* are the functions that will be used.

- void make_matrix(char path[], int matrix[max_size][max_size]) is a function that reads from the file a bidimensional array. This function receives as parameters the path of the file and the resulting matrix.
- void display_matrix(int matrix[max_size][max_size], int size) writes to stdout the matrix whose address will be the parameters of this function
- void multi_matrix(int matrix[max_size][max_size], int size) deals with multiplying a matrix with a given size
- void* thread_multi_matrix(void *params) is the multi_matrix() alternative for threads implementation; parameters are given through a structure that contains the size of the matrix, the matrix itself and the id of the thread associate.

Output of this sample:

```
root@s32g274aevb:/opt/samples# ./multicore 1000000

loop_number = 1000000

The number of cores: 4
The online cores :
0-3

The offline cores :



Without threads- - - > 12933ms

With threads- - - > 4297ms

root@s32g274aevb:/opt/samples#
```

For details on this demo, please refer to the Multicore sample application README file.

28.2 Networking Sample Applications

28.2.1 One to One Chat Application

The one to one chat application will simulate a TCP server-client chat. The connection will be established through sockets and the communication is basically a stop-and-wait sequence: both server and client wait for a response after sending a message.

The messages for both the client and the server are received as input from the console.

In order to run this demo on target, it is recommended to have networking enabled and use two different consoles, e.g. the serial console for the server and a ssh session for the client. Otherwise there will be issues when writing the input messages for the two processes.

Execution flow:

- From the server compilation will result an executable file that receives as parameter the port.
  Practically, after the "make"/"make server"/"make client" command, the server must be turned on and receive number for the port. For example, in order to use the port with the number 1923, the right command will be "./server 1923"
- In order to find the server(if it is available), the client should know the name or its IP address. There are two arguments that should be specified at the execution: the IP address and the and the port number used for communication.
- Once the server is turned on, the client can connect to it. Assuming that the IP address of the server is 192.168.0.1, the client will connect with the command "./client 192.168.0.1 1923". After that, the client can start the conversation.

Example:

- Start server using port 1923:

```
./server 1923
```

- Start client on the same host:

```
./client 127.0.0.1 1923
```

- Write messages for the client then the server, alternately.

For details on this demo, please refer to the Networking sample application README file.

Output of this demo:

- Server side console:

```
root@s32g274aevb:/opt/samples# ./server 1923
Client: test1
Server: reply1
Client: test2
Server: reply2
Client closed the connection
root@s32g274aevb:/opt/samples#
```

- Client side console:

```
root@s32g274aevb:/opt/samples# ./client 127.0.0.1 1923
Client: test1
Server: reply1
Client: test2
Server: reply2
Client: quit
root@s32g274aevb:/opt/samples#
```

28.2.2 One to Many Echo Application

The one to many echo application will simulate a TCP server-multiclient chat. The connection will be established through sockets and the communication is basically a single sequence: The client sends some data to the server and the server ping it back with a standard package of data.

Execution flow:

- From the server compilation will result an executable file that receives as parameter the port for communication.
- In order to find the server (if it is available), the client should know the name or its IP address. There are three arguments that should be specified at the execution, the IP address, the number of the port that the socket will use and an identification name.
- Once the server is turned on, the clients can connect to it. Assuming that the IP address of the server is 192.168.0.1, the clients will connect with the command "./client 192.168.0.1 1923".

Example:

- Start server using port 1923:

```
./server 1923
```

- Start clients on the same host, each on a different console/ssh session:

```
./client 127.0.0.1 1923
```

- Write messages for the clients, alternately. The server will echo them.

For details on this demo, please refer to the Networking-extended sample application README file.

Output of this demo:

- Server and 1st client console:

```
root@s32g274aevb:/opt/samples# ./server_multiclient 1923 &
[1] 477
root@s32g274aevb:/opt/samples#
root@s32g274aevb:/opt/samples# ./client 127.0.0.1 1923
Client: Client4 connected
Client5 connected

Client: test1
Client4: test1
Server: test1
Client: Client5: test2

Client: test3
Client4: test3
Server: test3
Client: Client5: test4

Client: quit
Client4 disconnected!
root@s32g274aevb:/opt/samples#
root@s32g274aevb:/opt/samples# Client5 disconnected!

root@s32g274aevb:/opt/samples# jobs
[1]+  Running                 ./server_multiclient 1923 &
root@s32g274aevb:/opt/samples# fg
```

```
./server_multiclient 1923

^C
root@s32g274aevb:/opt/samples#
```

- 2nd client console:

```
root@s32g274aevb:/opt/samples# ./client 127.0.0.1 1923
Client: test2
Server: test2
Client: test4
Server: test4
Client:
Client: quit
root@s32g274aevb:/opt/samples#
```

28.3 GPIO Libgpiod Application

This demo uses the "Libgpiod Library" C API for interacting with GPIOs. This application can be used to manipulate GPIO pins, that is to set outputs and read inputs.

Apart from identifying 2 GPIO pins to be configured as input and output, the GPIO chip which they are part of should be also identified and passed as the first argument to the application. To list the available GPIO chips use the 'gpiodetect' command.

Let's consider that IPIN is the input pin and OPIN is the output pin, both being part of 'gpiochip0'. Launch the demo instances:

```
root@s32g274aevb:/opt/samples# ./gpio_libgpiod gpiochip0 OPIN 1 &
root@s32g274aevb:/opt/samples# ./gpio_libgpiod gpiochip0 IPIN
```

The former command exports OPIN, sets its direction to "out", writes 1 as the initial pin value, then toggles it for 50 times. The latter command reads and prints each second the value of IPIN. The values of IPIN should alternate between 0 and 1 since the pins are connected.

For more details on this demo, please refer to the gpio_libgpiod/README file.

28.4 PCIe Sample Applications

There are two PCIe-based sample applications, "PCIe Shared Memory" and "pcitest". The first one uses a shared memory mechanism for communication between the End Point (EP) and the Root Complex (RC), while the other is an application that comes with the Linux kernel. Both applications can be used to test the PCIe connection between two S32 boards.

The two demos are added to the root filesystem for the Yocto image *fsl-image-auto* by adding the following line in *local.conf*:

```
DISTRO_FEATURES:append = " pcie-demos"
```

To select the proper working mode for the PCIe controllers involved in the demos, U-Boot variable `hwconfig` must be configured appropriately.

`hwconfig` examples:

PCIe0 set as RC:

```
serdes0:mode=pcie,clock=ext;pcie0:mode=rc
```

PCIe1 set as EP:

```
serdes1:mode=pcie,clock=ext;pcie1:mode=ep
```



Only one PCIe controller of the same board should be configured as EP for the tests.
Before booting Linux on the RC, the EP must be already booted to Linux and the EP configuration already done.

28.4.1 PCIe Shared Memory Sample Application

This demo is a basic, Linux-based, example of a pair of applications sharing a buffer in each other's memory and communicating over PCIe. The two apps can then read/write from/to their own or the other's shared memory, using either the CPU or DMA.

The demo consists of two applications, one running on the PCIe Root Complex (RC) and one running on the PCIe Endpoint (EP). The assumption is that both of them will run on Linux.

For other details on this demo, please also refer to the PCIe Shared Memory sample application README file.

28.4.1.1 Command line

EP side:

```
./test_ep -b <pcie_base_address> -a <local_ddr_addr_hex> [-n <BAR_index>][-m <memsize>][-w <count>][-i]
                                                         [-o][-c <commands>]
```

RC side:

```
./test_rc -a <rc_local_ddr_addr_hex> -e <ep_bar_addr_hex> [-m memsize][-w count][-c <commands>]
```

where:

- `<local_ddr_addr_hex>` is the local shared buffer, for both EP and RC.
- `<pcie_base_address>` is the base address of the PCIe controller address space for EP.
  (For S32G/R: 0x5800000000 for PCIe0 and 0x4800000000 for PCIe1).
- `<ep_bar_addr_hex>` is the EP BAR address shown during enumeration on RC.
- `<BAR_index>` is the EP BAR to be used for communication.
  By default, BAR0 is used for EP and is recommended for S32G/R.
  Make sure `<ep_bar_addr_hex>` matches the EP BAR for the correct RC PCIe controller.

- <memsize> is the size of the shared memory buffer and of the PCIe mapped window.
- <count> is the number of double words displayed from the start and the end of the memory buffer for command '5'.
- <commands> is a string composed of one-letter commands to be executed in a batch mode.
  It should contain the command letters '1' to '8' for EP, '1' to '6' for RC, plus 0 which means 5s delay. The exit commands are not required, as the applications will exit automatically after executing the batch.
- -i (config inbound) Setup inbound window for receiving data into local shared buffer. If 'auto-config-bars' is set in the Linux Kernel pcie device-tree node, then the inbound window has been already configured during PCIe Linux Kernel driver probing. Hence '-i' option is not needed in this scenario (using PCIe RC & PCIe EP on the same board).
- -o (config outbound) Setup outbound window for accessing RC mem.



By setting the property 'auto-config-bars' in the pcie node of the PCIe controller running as EndPoint (EP), then the EP BARs are configured during boot, and the Root Complex (if probed first) will re-enumerate it on-the-fly using the hot-plug mechanism. This removes the necessity to boot first the EP then the RC. The two connected controllers running as EP and RC will be probed correctly, and the RC will enumerate the EP no matter if the EP and the RC are physically located on the same board or on two distinct boards. The property 'auto-config-bars' can be set in U-Boot from the variable 'fdt_override', as follows:

```
=> setenv fdt_override 'fdt addr ${fdt_addr}; fdt resize;
fdt set pci1 "auto-config-bars"'
```

Examples:
1. Two Boards, default device tree, 'auto-config-bars' not set: *For S32G/R EP (PCIe1, BAR0):*

```
# ./test_ep -a 0xC0000000 -b 0x4800000000 -i -o
```

For S32G/R RC (PCIe0, EP using BAR0):

```
# ./test_rc -a 0xC0000000 -e 0x5800100000
```

2. Single board, PCIe0 as RC, PCIe1 as EP, connected via an extension cable, 'auto-config-bars' property set in pcie1 node:

```
# ./test_ep -a 0xC0400000 -b 0x4800000000 -o
# # next command was run on a separate SSH terminal connection
# ./test_rc -a 0xc0000000 -e 0x5800200000
```



<ep_bar_addr_hex> for the selected EP BAR can be found from the kernel boot log, with the dmesg command. E.g. for BAR 0:

```
# dmesg | grep "BAR 0"
[ 5.163953] pci 0000:00:00.0 BAR 0: assigned [mem 0x5800000000-0x58000fffff]
[ 5.581444] pci 0001:00:00.0 BAR 0: assigned [mem 0x4800000000-0x48000fffff]
[ 5.581506] pci 0001:01:00.0 BAR 0: assigned [mem 0x4800100000-0x48001fffff]
```

Although on Linux boot the PCIe controllers are already configured as EP or RC, depending on U-Boot environment variable hwconfig, the BAR configuration and other communication settings are only done when test_ep is started. Hence, test_ep needs to be executed before RC boots (if 'auto-config-bars' property is not used).

28.4.1.2 Usage

After the handshake between `test_ep` and `test_rc` is done, the following commands are available inside the application:

Left: `test_ep`; Right: `test_rc`

```
Select test (press 'h' to show all tests): h

Test cases :
1. Single 1M Write transfer from local buffer to RC DDR mem (pattern=0x12)
2. Single 1M Read transfer from RC DDR buffer to local DDR buffer
3. Variable size throughput test Write(pattern = 0x67) + Read to/from RC DDR mem
4. Fill local DDR buffer (1M) with pattern 0xdeadbeef
5. Read and print first and last 2 DW(32bytes) bytes in local DDR
6. Write 1M from local DDR to RC DDR mem through DMA single transfer
7. Read 1M from RC DDR mem to local DDR through DMA single transfer
8. Multiple 1M Write transfers from local buffer to RC DDR mem (pattern=0x12)
   This is essentially the same as #1, only looped and multithreaded.
   Usable for performance tests and some errata validation.
9. Exit app

Select test (press 'h' to show all tests): []
```

```
Select test (press 'h' to show all tests): h

Test cases :
1. Single 1M Write transfer from local buffer to EP mem (pattern = 0x42)
2. Single 1M Read  transfer from EP mem to local DDR buffer
3. Variable size throughput test Write(pattern = 0xc8) + Read to/from EP mem
4. Fill local DDR buffer (1M) with DW pattern 0x55AA55AA
5. Read and print first and last 2 DW(32bytes) in local DDR
6. Multiple 1M Write transfers from local buffer to EP mem (pattern = 0x42)
   This is essentially the same as #1, only looped and multithreaded.
   Usable for performance tests and some errata validation.
7. Exit app

Select test (press 'h' to show all tests): []
```

28.4.1.3 Examples

Left: `test_ep`; Right: `test_rc`; Write from RC to EP (no DMA)

```
Command code  = 4
Select test (press 'h' to show all tests): 5

Command code  = 5
First 2 DWORDS(1DW = 4bytes) from local mapped DDR
*(buf +       0) = 0xdeadbeef
*(buf +     0x1) = 0xdeadbeef
Last 2 DWORDS(1DW = 4bytes) from local mapped DDR
*(buff +  0x3fffc) = 0xdeadbeef
*(buff +  0x3fffd) = 0xdeadbeef
Select test (press 'h' to show all tests): 5

Command code  = 5
First 2 DWORDS(1DW = 4bytes) from local mapped DDR
*(buf +       0) = 0x42424242
*(buf +     0x1) = 0x42424242
Last 2 DWORDS(1DW = 4bytes) from local mapped DDR
*(buff +  0x3fffc) = 0x42424242
*(buff +  0x3fffd) = 0x42424242
Select test (press 'h' to show all tests): []
```

```
Command code  = 1

1 : 1MB Write => Size: 1048568 bytes     DMA: NO        Time: 0.002529800 seconds      Rate: 404771 KB/s

Select test (press 'h' to show all tests):
```

Left: `test_ep`; Right: `test_rc`; DMA write from EP to RC

```
Select test (press 'h' to show all tests): 4

Command code  = 4
Select test (press 'h' to show all tests): 5

Command code  = 5
First 2 DWORDS(1DW = 4bytes) from local mapped DDR
*(buf +       0) = 0xdeadbeef
*(buf +     0x1) = 0xdeadbeef
Last 2 DWORDS(1DW = 4bytes) from local mapped DDR
*(buff +  0x3fffc) = 0xdeadbeef
*(buff +  0x3fffd) = 0xdeadbeef
Select test (press 'h' to show all tests): 6

Command code  = 6
DMA transfer completed
6 : 1MB Write => Size: 1048568 bytes     DMA: YES       Time: 0.001308800 seconds      Rate: 782390 KB/s

DMA Signal Handler count = 1
Select test (press 'h' to show all tests):
```

```
Select test (press 'h' to show all tests): 4

Command code  = 4
Select test (press 'h' to show all tests): 5

Command code  = 5

First 2 DWORDS(1DW = 4bytes) from local mapped DDR
*(buf +       0) = 0x55aa55aa
*(buf +     0x1) = 0x55aa55aa
Last 2 DWORDS(1DW = 4bytes) from local mapped DDR
*(buff +  0x3fffc) = 0x55aa55aa
*(buff +  0x3fffd) = 0x55aa55aa
Select test (press 'h' to show all tests): 5

Command code  = 5

First 2 DWORDS(1DW = 4bytes) from local mapped DDR
*(buf +       0) = 0xdeadbeef
*(buf +     0x1) = 0xdeadbeef
Last 2 DWORDS(1DW = 4bytes) from local mapped DDR
*(buff +  0x3fffc) = 0xdeadbeef
*(buff +  0x3fffd) = 0xdeadbeef
Select test (press 'h' to show all tests):
```

28.4.2 *pcitest* Application

This is the standard PCI test application from the Linux kernel, based on the End Point Function (EPF) Test Framework. It is described in the Linux Kernel documentation (https://www.kernel.org/doc/html/v6.6/PCI/endpoint/pci-test-howto.html).

28.4.2.1 EP side configuration

This can be done with the convenience script *pcitest-ep-config.sh*, located on target root filesystem in */usr/bin*. It sets the PCIe Device ID to an S32 device and enables 8 MSI and 16 MSI-X interrupts. The EP configuration must be done before Linux on the RC boots, as the script also sets the PCIe Device ID which is recognized by the EPF Test kernel module.

28.4.2.2 Command Line

`pcitest` can only be run on the RC side, after the EP has been configured:

```
root@s32g274ardb2:~# pcitest -h
usage: pcitest [options]
Options:
        -D <dev>                PCI endpoint test device {default: /dev/pci-endpoint-test.0}
        -b <bar num>            BAR test (bar number between 0..5)
        -m <msi num>            MSI test (msi number between 1..32)
        -x <msix num>           MSI-X test (msix number between 1..2048)
        -i <irq type>           Set IRQ type (0 - Legacy, 1 - MSI, 2 - MSI-X)
        -e                      Clear IRQ
        -I                      Get current IRQ type configured
        -d                      Use DMA Engine
        -S                      Use Simple Single DMA transfer
        -l                      Legacy IRQ test
        -r                      Read buffer test
        -w                      Write buffer test
        -c                      Copy buffer test
        -s <size>               Size of buffer {default: 100KB}
        -h                      Print this help message
```

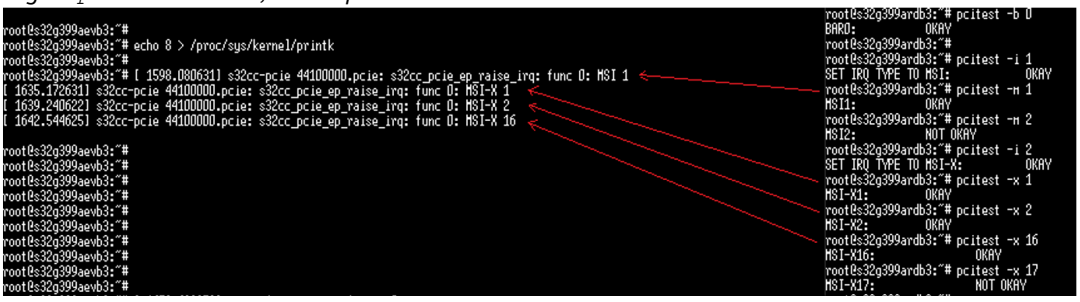
`pcitest` can be run directly from a shell, one command at a time, or from a script, like *pcitest.sh*, found in the same location as *pcitest*.



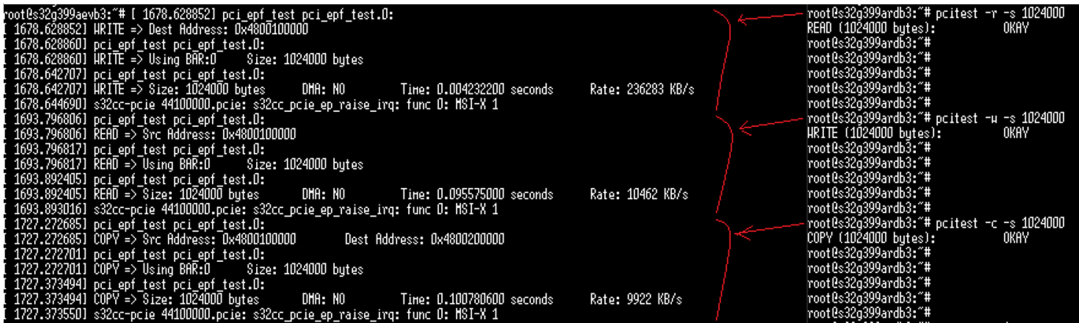
DMA support is not enabled for `pcitest`.
`pcitest.sh` has a very large output, as it loops over all theoretical configurations (e.g. tests all 2048 possible MSI-X interrupts).

28.4.2.3 Examples

Left: EP; Right: `pcitest` on RC; Interrupt tests



Left: EP; Right: `pcitest` on RC; Data transfer tests


root@s32g399aevb3:~# [1678.628852] pci_epf_test pci_epf_test.0:
[1678.628852] WRITE => Dest Address: 0x4800100000
[1678.628860] pci_epf_test pci_epf_test.0:
[1678.628860] WRITE => Using BAR:0 Size: 1024000 bytes
[1678.642707] pci_epf_test pci_epf_test.0:
[1678.642707] WRITE => Size: 1024000 bytes DMA: NO Time: 0.004232200 seconds Rate: 236283 KB/s
[1678.644690] s32cc-pcie 44100000.pcie: s32cc_pcie_ep_raise_irq: func 0: MSI-X 1
[1693.796806] pci_epf_test pci_epf_test.0:
[1693.796806] READ => Src Address: 0x4800100000
[1693.796817] pci_epf_test pci_epf_test.0:
[1693.796817] READ => Using BAR:0 Size: 1024000 bytes
[1693.892405] pci_epf_test pci_epf_test.0:
[1693.892405] READ => Size: 1024000 bytes DMA: NO Time: 0.095575000 seconds Rate: 10462 KB/s
[1693.893016] s32cc-pcie 44100000.pcie: s32cc_pcie_ep_raise_irq: func 0: MSI-X 1
[1727.272685] pci_epf_test pci_epf_test.0:
[1727.272685] COPY => Src Address: 0x4800100000 Dest Address: 0x4800200000
[1727.272701] pci_epf_test pci_epf_test.0:
[1727.272701] COPY => Using BAR:0 Size: 1024000 bytes
[1727.373494] pci_epf_test pci_epf_test.0:
[1727.373494] COPY => Size: 1024000 bytes DMA: NO Time: 0.100780600 seconds Rate: 9922 KB/s
[1727.373550] s32cc-pcie 44100000.pcie: s32cc_pcie_ep_raise_irq: func 0: MSI-X 1
root@s32g399ardb3:~# pcitest -r -s 1024000
READ (1024000 bytes): OKAY
root@s32g399ardb3:~#
root@s32g399ardb3:~#
root@s32g399ardb3:~#
root@s32g399ardb3:~#
root@s32g399ardb3:~#
root@s32g399ardb3:~# pcitest -w -s 1024000
WRITE (1024000 bytes): OKAY
root@s32g399ardb3:~#
root@s32g399ardb3:~#
root@s32g399ardb3:~#
root@s32g399ardb3:~#
root@s32g399ardb3:~#
root@s32g399ardb3:~# pcitest -c -s 1024000
COPY (1024000 bytes): OKAY
root@s32g399ardb3:~#
root@s32g399ardb3:~#
root@s32g399ardb3:~#
root@s32g399ardb3:~#
root@s32g399ardb3:~#

29 SoC Errata

29.1 Thermal Monitoring Unit (TMU)

29.1.1 ERR052243

According to ERR052243, invalid temperature measurements may be observed across the temperature range specified in the device data sheet.

As a workaround, the Linux TMU driver monitors the differences between two consecutive samples. If the difference is greater than a defined acceptable change, the new temperature could be erroneous, and the driver will wait for the next temperature sample. In case of an erroneous measurement, the temperature will rise and fall (or the other way around) at approximately the same rate.

In the Linux driver implementation, if the absolute value of the difference between the rising and falling rate is less or equal with two, the initial temperature measurement is considered invalid and discarded. Waiting for one more sample avoids thermal mitigation actions based on an incorrect temperature value. The workaround applies only for the site monitored with interrupts.

The acceptable change is defined as the property of the `tmu` node in the Linux Device Tree, named `tmu-rate-filter`, and should contain the maximum acceptable difference, in degrees, between two consecutive TMU samples.

29.2 Ultra Secured Digital Host Controller (uSDHC)

29.2.1 ERR052198

To facilitate command queuing in eMMC, uSDHC supports Command Queue Engine (CQE) so that the host can queue data transfer tasks. According to ERR052198, in some cases where the CQDPT (Device Pending Task) bit is being set and cleared at the same time, the CQDPT bit is not cleared even after the task has completed execution, resulting in a CQE timeout. The impact is that such CQE timeouts could lead to an OS crash.

Erratum ERR052198 applies to eMMC High Speed Modes, such as HS200, HS400 and HS400ES, when CQHCI support is enabled. CQHCI support is currently available only in the Linux Kernel.

There are two possible workarounds:

1. Disable CQE feature from uSDHC Linux Kernel driver.
2. Before sending a DCMD (Direct Command) request, software should manually clear the CQDPT[bit n] by setting CQTCLR[bit n] if DPT[bit n] != CQTDBR[bit n], in order to clear the pending task.

By default in Linux Kernel, if eMMC High Speed Modes are enabled the first workaround is applied. Additionally, CQHCI feature can be enabled by following this section. In this scenario, the second workaround will be applied and CQHCI support will remain enabled. Please note that its stability cannot be guaranteed.

How to Reach Us:

Home Page:
nxp.com

Web Support:
nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including “typicals,” must be validated for each customer application by customer’s technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

While NXP has implemented advanced security features, all products may be subject to unidentified vulnerabilities. Customers are responsible for the design and operation of their applications and products to reduce the effect of these vulnerabilities on customer’s applications and products, and NXP accepts no liability for any vulnerability that is discovered. Customers should implement appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRANCE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, AltiVec, C-5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C-Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. AMBA, Arm, Arm7, Arm7TDMI, Arm9, Arm11, Artisan, big.LITTLE, Cordio, CoreLink, CoreSight, Cortex, DesignStart, DynamIQ, Jazelle, Keil, Mali, Mbed, Mbed Enabled, NEON, POP, RealView, SecurCore, Socrates, Thumb, TrustZone, ULINK, ULINK2, ULINK-ME, ULINK-PLUS, ULINKpro, μVision, Versatile are trademarks or registered trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere. The related technology may be protected by any or all of patents, copyrights, designs and trade secrets. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

©2024 NXP B.V.